

Introduction

This book is intended as an introduction to the [Leptos](#) Web framework. It will walk through the fundamental concepts you need to build applications, beginning with a simple application rendered in the browser, and building toward a full-stack application with server-side rendering and hydration.

The guide doesn't assume you know anything about fine-grained reactivity or the details of modern Web frameworks. It does assume you are familiar with the Rust programming language, HTML, CSS, the DOM, and basic Web APIs.

Leptos is most similar to frameworks like [Solid](#) (JavaScript) and [Sycamore](#) (Rust). There are some similarities to other frameworks like React (JavaScript), Svelte (JavaScript), Yew (Rust), and Dioxus (Rust), so knowledge of one of those frameworks may also make it easier to understand Leptos.

You can find more detailed docs for each part of the API at [Docs.rs](#).

The source code for the book is available [here](#). PRs for typos or clarification are always welcome.

Getting Started

There are two basic paths to getting started with Leptos:

1. **Client-side rendering (CSR) with [Trunk](#)** - a great option if you just want to make a snappy website with Leptos, or work with a pre-existing server or API. In CSR mode, Trunk compiles your Leptos app to WebAssembly (WASM) and runs it in the browser like a typical Javascript single-page app (SPA). The advantages of Leptos CSR include faster build times and a quicker iterative development cycle, as well as a simpler mental model and more options for deploying your app. CSR apps do come with some disadvantages: initial load times for your end users are slower compared to a server-side rendering approach, and the usual SEO challenges that come along with using a JS single-page app model apply to Leptos CSR apps as well. Also note that, under the hood, an auto-generated snippet of JS is used to load the Leptos WASM bundle, so JS *must* be enabled on the client device for your CSR app to display properly. As with all software engineering, there are trade-offs here you'll need to consider.
2. **Full-stack, server-side rendering (SSR) with [cargo-leptos](#)** - SSR is a great option for building CRUD-style websites and custom web apps if you want Rust powering both your frontend and backend. With the Leptos SSR option, your app is rendered to HTML on the server and sent down to the browser; then, WebAssembly is used to instrument the HTML so your app becomes interactive - this process is called 'hydration'. On the server side, Leptos SSR apps integrate closely with your choice of either [Actix-web](#) or [Axum](#) server libraries, so you can leverage those communities' crates to help build out your Leptos server. The advantages of taking the SSR route with Leptos include helping you get the best initial load times and optimal SEO scores for your web app. SSR apps can also dramatically simplify working across the server/client boundary via a Leptos feature called "server functions", which lets you transparently call functions on the server from your client code (more on this feature later). Full-stack SSR isn't all rainbows and butterflies, though - disadvantages include a slower developer iteration loop (because you need to recompile both the server and client when making Rust code changes), as well as some added complexity that comes along with hydration.

By the end of the book, you should have a good idea of which trade-offs to make and which route to take - CSR or SSR - depending on your project's requirements.

In Part 1 of this book, we'll start with client-side rendering Leptos sites and building reactive UIs using `Trunk` to serve our JS and WASM bundle to the browser.

We'll introduce `cargo-leptos` in Part 2 of this book, which is all about working with the full power of Leptos in its full-stack, SSR mode.

 Note

If you're coming from the Javascript world and terms like client-side rendering (CSR) and server-side rendering (SSR) are unfamiliar to you, the easiest way to understand the difference is by analogy:

Leptos' CSR mode is similar to working with React (or a 'signals'-based framework like SolidJS), and focuses on producing a client-side UI which you can use with any tech stack on the server.

Using Leptos' SSR mode is similar to working with a full-stack framework like Next.js in the React world (or Solid's "SolidStart" framework) - SSR helps you build sites and apps that are rendered on the server then sent down to the client. SSR can help to improve your site's loading performance and accessibility as well as make it easier for one person to work on *both* client- and server-side without needing to context-switch between different languages for frontend and backend.

The Leptos framework can be used either in CSR mode to just make a UI (like React), or you can use Leptos in full-stack SSR mode (like Next.js) so that you can build both your UI and your server with one language: Rust.

Hello World! Getting Set up for Leptos CSR Development

First up, make sure Rust is installed and up-to-date ([see here if you need instructions](#)).

If you don't have it installed already, you can install the "Trunk" tool for running Leptos CSR sites by running the following on the command-line:

```
cargo install --locked trunk
```

And then create a basic Rust project

```
cargo init leptos-tutorial
```

cd into your new `leptos-tutorial` project and add `leptos` as a dependency

```
cargo add leptos --features=csr
```

Make sure you've added the `wasm32-unknown-unknown` target so that Rust can compile your code to WebAssembly to run in the browser.

```
rustup target add wasm32-unknown-unknown
```

Create a simple `index.html` in the root of the `leptos-tutorial` directory

```
<!DOCTYPE html>
<html>
  <head></head>
  <body></body>
</html>
```

And add a simple "Hello, world!" to your `main.rs`

```
use leptos::prelude::*;

fn main() {
  leptos::mount::mount_to_body(|| view! { <p>"Hello, world!"</p> })
}
```

Your directory structure should now look something like this

```
leptos_tutorial
├── src
│   └── main.rs
├── Cargo.toml
└── index.html
```

Now run `trunk serve --open` from the root of the `leptos-tutorial` directory. Trunk should automatically compile your app and open it in your default browser. If you make edits to `main.rs`, Trunk will recompile your source code and live-reload the page.

Welcome to the world of UI development with Rust and WebAssembly (WASM), powered by Leptos and Trunk!

Note

If you are using Windows, note that `trunk serve --open` may not work. If you have issues with `--open`, simply use `trunk serve` and open a browser tab manually.

Now before we get started building your first real applications with Leptos, there are a couple of things you might want to know to help make your experience with Leptos just a little bit easier.

Leptos Developer Experience Improvements

There are a couple of things you can do to improve your experience of developing websites and apps with Leptos. You may want to take a few minutes and set up your environment to optimize your development experience, especially if you want to code along with the examples in this book.

1) Set up `console_error_panic_hook`

By default, panics that happen while running your WASM code in the browser just throw an error in the browser with an unhelpful message like `Unreachable executed` and a stack trace that points into your WASM binary.

With `console_error_panic_hook`, you get an actual Rust stack trace that includes a line in your Rust source code.

It's very easy to set up:

1. Run `cargo add console_error_panic_hook` in your project
2. In your main function, add `console_error_panic_hook::set_once();`

If this is unclear, [click here for an example](#).

Now you should have much better panic messages in the browser console!

2) Editor Autocompletion inside `#[component]` and `#[server]`

Because of the nature of macros (they can expand from anything to anything, but only if the input is exactly correct at that instant) it can be hard for rust-analyzer to do proper autocompletion and other support.

If you run into issues using these macros in your editor, you can explicitly tell rust-analyzer to ignore certain proc macros. For the `#[server]` macro especially, which annotates function bodies but doesn't actually transform anything inside the body of your function, this can be really helpful.

 Note

Starting in Leptos version 0.5.3, rust-analyzer support was added for the `#[component]` macro, but if you run into issues, you may want to add `#[component]` to the macro ignore list as well (see below). Note that this means that rust-analyzer doesn't know about your component props, which may generate its own set of errors or warnings in the IDE.

VSCode `settings.json`:

```
"rust-analyzer.procMacro.ignored": {
  "leptos_macro": [
    // optional:
    // "component",
    "server"
  ],
}
```

VSCode with cargo-leptos `settings.json`:

```
"rust-analyzer.procMacro.ignored": {
  "leptos_macro": [
    // optional:
    // "component",
    "server"
  ],
},
// if code that is cfg-gated for the `ssr` feature is shown as inactive,
// you may want to tell rust-analyzer to enable the `ssr` feature by default
//
// you can also use `rust-analyzer.cargo.allFeatures` to enable all features
"rust-analyzer.cargo.features": ["ssr"]
```

Neovim:

```
vim.lsp.config('rust_analyzer', {
  -- Other Confgs ...
  settings = {
    ["rust-analyzer"] = {
      -- Other Settings ...
      procMacro = {
        ignored = {
          leptos_macro = {
            -- optional: --
            -- "component",
            "server",
          },
        },
      },
    },
  },
})
```

Helix, in `.helix/languages.toml`:

```
[[language]]
name = "rust"

[language-server.rust-analyzer]
config = { procMacro = { ignored = { leptos_macro = [
  # Optional:
  # "component",
  "server"
] } } }
```

Zed, in `settings.json`:

```
{
  -- Other Settings ...
  "lsp": {
    "rust-analyzer": {
      "initialization_options": {
        "procMacro": {
          "ignored": {
            "leptos_macro": [
              // Optional:
              // "component",
              "server"
            ],
          },
        },
      },
    },
  },
}
```

SublimeText 3, under `LSP-rust-analyzer.sublime-settings` in `Goto Anything...` menu:

```
// Settings in here override those in "LSP-rust-analyzer/LSP-rust-analyzer.sublime-settings"
{
  "rust-analyzer.procMacro.ignored": {
    "leptos_macro": [
      // optional:
      // "component",
      "server"
    ],
  },
}
```

3) Enable features in Rust-Analyzer for your Editor (optional)

By default, rust-analyzer will only run against the default features in your Rust project. Leptos uses different features to control compilation. For client side rendered projects, we use `csr` in different places, for server side rendered apps they can include `ssr` for server code and `hydrate` for code that we'll only run in the browser.

How to enable these features varies by your IDE, we've listed some common ones below. If your IDE is not listed, you can usually find the setting by searching for `rust-analyzer.cargo.features` OR `rust-analyzer.cargo.allFeatures`.

VSCode, in `settings.json`:

```
{
  "rust-analyzer.cargo.features": "all", // Enable all features
}
```

Neovim, in `init.lua`:

```
vim.lsp.config('rust_analyzer', {
  settings = {
    ["rust-analyzer"] = {
      cargo = {
        features = "all", -- Enable all features
      },
    },
  },
})
```

helix, in `.helix/languages.toml` or per project in `.helix/config.toml`:

```
[[language]]
name = "rust"

[language-server.rust-analyzer.config.cargo]
allFeatures = true
```

Zed, in `settings.json`:

```
{
  -- Other Settings ...
  "lsp": {
    "rust-analyzer": {
      "initialization_options": {
        "cargo": {
          "allFeatures": true // Enable all features
        }
      }
    }
  }
}
```

SublimeText 3, in the user settings for `LSP-rust-analyzer-settings.json`

```
{
  "settings": {
    "LSP": {
      "rust-analyzer": {
        "settings": {
          "cargo": {
            "features": "all"
          }
        }
      }
    }
  }
}
```

4) Set up `leptosfmt` (optional)

`leptosfmt` is a formatter for the Leptos `view!` macro (inside of which you'll typically write your UI code). Because the `view!` macro enables an 'RSX' (like JSX) style of writing your UI's, `cargo-fmt` has a harder time auto-formatting your code that's inside the `view!` macro.

`leptosfmt` is a crate that solves your formatting issues and keeps your RSX-style UI code looking nice and tidy!

`leptosfmt` can be installed and used via the command line or from within your code editor:

First, install the tool with `cargo install leptosfmt`.

If you just want to use the default options from the command line, just run `leptosfmt ./**/*.rs` from the root of your project to format all the rust files using `leptosfmt`.

Run automatically in Rust Analyzer IDEs

If you wish to set up your editor to work with `leptosfmt`, or if you wish to customize your `leptosfmt` experience, please see the instructions available on the [leptosfmt github repo's README.md page](#).

Just note that it's recommended to set up your editor with `leptosfmt` on a per-workspace basis for best results.

Run automatically in RustRover

Unfortunately, RustRover does not support Rust Analyzer, so a different approach is required in order to automatically run `leptosfmt`. One way is to use the [FileWatchers](#) plugin with the below configuration:

- Name: Leptosfmt
- File type: Rust files
- Program: `/path/to/leptosfmt` (can simply be `leptosfmt` if it's in your `$PATH` environment variable)
- Arguments: `$FilePath$`
- Output paths to refresh: `$FilePath$`

5) Use `--cfg=erase_components` during development

Leptos 0.7 made a number of changes to the renderer that relied more heavily on the type system. For larger projects, this can lead to slower compile times. Most of the slowdown in compile times can be alleviated by using the custom configuration flag `--cfg=erase_components` during development. (This erases some of that type information to reduce the amount of work done and debug info emitted by the compiler, at the expense of additional binary size and runtime cost, so it's best not to use it in release mode.)

As of cargo-leptos v0.2.40, this is automatically enabled for you in development mode. If you are using trunk, not using cargo-leptos, or want to enable it for non-dev uses, you can set this easily in the command line (`RUSTFLAGS="--cfg erase_components" trunk serve` or `RUSTFLAGS="--cfg erase_components" cargo leptos watch`), or in your `.cargo/config.toml`:

```
# use your own native target
[target.aarch64-apple-darwin]
rustflags = [
    "--cfg",
    "erase_components",
]

[target.wasm32-unknown-unknown]
rustflags = [
    "--cfg",
    "erase_components",
]
```

The Leptos Community and leptos-* Crates

The Community

One final note before we get to building with Leptos: if you haven't already, feel free to join the growing community on the Leptos [Discord](#) and on [Github](#). Our Discord channel in particular is very active and friendly - we'd love to have you there!

Note

If you find a chapter or an explanation that isn't clear while you're working your way through the Leptos book, just mention it in the "docs-and-education" channel or ask a question in "help" so we can clear things up and update the book for others.

As you get further along in your Leptos journey and find that you have questions about "how to do 'x' with Leptos", then search the Discord "help" channel to see if a similar question has been asked before, or feel free to post your own question - the community is quite helpful and very responsive.

The "[Discussions](#)" on Github are also a great place for asking questions and keeping up with Leptos announcements.

And of course, if you run into any bugs while developing with Leptos or would like to make a feature request (or contribute a bug fix / new feature), open up an issue on the [Github issue tracker](#).

Leptos-* Crates

The community has built a growing number of Leptos-related crates that will help you get productive with Leptos projects more quickly - check out the list of crates built on top of Leptos and contributed by the community on the [Awesome Leptos](#) repo on Github.

If you want to find the newest, up-and-coming Leptos-related crates, check out the "Tools and Libraries" section of the Leptos Discord. In that section, there are channels for the Leptos `view!` macro formatter (in the "leptosfmt" channel); there's a channel for the utility library "leptos-use"; another channel for the UI component library "thaw-ui"; and a

"libraries" channel where new `leptos-*` crates are discussed before making their way into the growing list of crates and resources available on [Awesome Leptos](#).

Part 1: Building User Interfaces

In the first part of the book, we're going to look at building user interfaces on the client-side using Leptos. Under the hood, Leptos and Trunk are bundling up a snippet of Javascript which will load up the Leptos UI, which has been compiled to WebAssembly to drive the interactivity in your CSR (client-side rendered) website.

Part 1 will introduce you to the basic tools you need to build a reactive user interface powered by Leptos and Rust. By the end of Part 1, you should be able to build a snappy synchronous website that's rendered in the browser and which you can deploy on any static-site hosting service, like Github Pages or Vercel.

Info

To get the most out of this book, we encourage you to code along with the examples provided. In the [Getting Started](#) and [Leptos DX](#) chapters, we showed you how to set up a basic project with Leptos and Trunk, including WASM error handling in the browser. That basic setup is enough to get you started developing with Leptos.

If you'd prefer to get started using a more full-featured template which demonstrates how to set up a few of the basics you'd see in a real Leptos project, such as routing, (covered later in the book), injecting `<Title>` and `<Meta>` tags into the page head, and a few other niceties, then feel free to utilize [the leptos-rs start-trunk](#) template repo to get up and running.

The `start-trunk` template requires that you have Trunk and `cargo-generate` installed, which you can get by running `cargo install trunk` and `cargo install cargo-generate`.

To use the template to set up your project, just run

```
cargo generate --git https://github.com/leptos-rs/start-trunk
```

then run

```
trunk serve --port 3000 --open
```

in the newly created app's directory to start developing your app. The Trunk server will reload your app on file changes, making development relatively seamless.

A Basic Component

That “Hello, world!” was a *very* simple example. Let’s move on to something a little more like an ordinary app.

First, let’s edit the `main` function so that, instead of rendering the whole app, it just renders an `<App/>` component. Components are the basic unit of composition and design in most web frameworks, and Leptos is no exception. Conceptually, they are similar to HTML elements: they represent a section of the DOM, with self-contained, defined behavior. Unlike HTML elements, they are in `PascalCase`, so most Leptos applications will start with something like an `<App/>` component.

```
use leptos::mount::mount_to_body;

fn main() {
    mount_to_body(App);
}
```

Now let’s define our `App` component itself. Because it’s relatively simple, I’ll give you the whole thing up front, then walk through it line by line.

```
use leptos::prelude::*;

#[component]
fn App() -> impl IntoView {
    let (count, set_count) = signal(0);

    view! {
        <button
            on:click=move |_| set_count.set(3)
        >
            "Click me: "
            {count}
        </button>
        <p>
            "Double count: "
            {move || count.get() * 2}
        </p>
    }
}
```

Importing the Prelude

```
use leptos::prelude::*;
```

Leptos provides a prelude which includes commonly-used traits and functions. If you'd prefer to use individual imports, feel free to do that; the compiler will provide helpful recommendations for each import.

The Component Signature

```
#[component]
```

Like all component definitions, this begins with the `#[component]` macro. `#[component]` annotates a function so it can be used as a component in your Leptos application. We'll see some of the other features of this macro in a couple chapters.

```
fn App() -> impl IntoView
```

Every component is a function with the following characteristics

1. It takes zero or more arguments of any type.
2. It returns `impl IntoView`, which is an opaque type that includes anything you could return from a Leptos `view`.

Component function arguments are gathered together into a single props struct which is built by the `view` macro as needed.

The Component Body

The body of the component function is a set-up function that runs once, not a render function that reruns multiple times. You'll typically use it to create a few reactive variables, define any side effects that run in response to those values changing, and describe the user interface.

```
let (count, set_count) = signal(0);
```

`signal` creates a signal, the basic unit of reactive change and state management in Leptos. This returns a `(getter, setter)` tuple. To access the current value, you'll use `count.get()` (or, on nightly Rust, the shorthand `count()`). To set the current value, you'll call `set_count.set(...)` (or, on nightly, `set_count(...)`).

`.get()` clones the value and `.set()` overwrites it. In many cases, it's more efficient to use `.with()` or `.update()`; check out the docs for [ReadSignal](#) and [WriteSignal](#) if

you'd like to learn more about those trade-offs at this point.

The View

Leptos defines user interfaces using a JSX-like format via the `view` macro.

```
view! {  
  <button  
    // define an event listener with on:  
    on:click=move |_| set_count.set(3)  
  >  
    // text nodes are wrapped in quotation marks  
    "Click me: "  
  
    // blocks include Rust code  
    // in this case, it renders the value of the signal  
    {count}  
  </button>  
  <p>  
    "Double count: "  
    {move || count.get() * 2}  
  </p>  
}
```

This should mostly be easy to understand: it mostly looks like HTML, with a special `on:click` syntax to define a `click` event listener and a few text nodes that look like Rust strings. All HTML elements are supported, including both built-in elements (like `<p>`) and custom elements/web components (like `<my-custom-element>`).

Info

Unquoted text: The `view` macro does have some support for unquoted text nodes, which are the norm in HTML or JSX (i.e., `<p>Hello!</p>` rather than `<p>"Hello!"</p>`). Due to limitations of Rust proc macros, using unquoted text can occasionally cause spacing issues around punctuation, and does not support all Unicode strings. You can use unquoted text if it's your preference; note that if you encounter any issues with it, they can always be resolved by quoting the text node as an ordinary Rust string.

Then there are two values in braces: one, `{count}`, seems pretty easy to understand (it's just the value of our signal), and then...

```
{move || count.get() * 2}
```

whatever that is.

People sometimes joke that they use more closures in their first Leptos application than they've ever used in their lives. And fair enough.

Passing a function into the view tells the framework: "Hey, this is something that might change."

When we click the button and call `set_count`, the `count` signal is updated. This `move || count.get() * 2` closure, whose value depends on the value of `count`, reruns, and the framework makes a targeted update to that specific text node, touching nothing else in your application. This is what allows for extremely efficient updates to the DOM.

Remember—and this is *very important*—only signals and functions are treated as reactive values in the view.

This means that `{count}` and `{count.get()}` do very different things in your view.

`{count}` passes in a signal, telling the framework to update the view every time `count` changes. `{count.get()}` accesses the value of `count` once, and passes an `i32` into the view, rendering it once, unreactively.

In the same way, `{move || count.get() * 2}` and `{count.get() * 2}` behave differently. The first one is a function, so it's rendered reactively. The second is a value, so it's just rendered once, and won't update when `count` changes.

You can see the difference in the CodeSandbox below!

Let's make one final change. `set_count.set(3)` is a pretty useless thing for a click handler to do. Let's replace "set this value to 3" with "increment this value by 1":

```
move |_| {  
    *set_count.write() += 1;  
}
```

You can see here that while `set_count` just sets the value, `set_count.write()` gives us a mutable reference and mutates the value in place. Either one will trigger a reactive update in our UI.

Throughout this tutorial, we'll use CodeSandbox to show interactive examples. Hover over any of the variables to show Rust-Analyzer details and docs for what's going on. Feel free to fork the examples to play with them yourself!

[□ Live example](#)



► [CodeSandbox Source](#)

view: Dynamic Classes, Styles and Attributes

So far we've seen how to use the `view` macro to create event listeners and to create dynamic text by passing a function (such as a signal) into the view.

But of course there are other things you might want to update in your user interface. In this section, we'll look at how to update classes, styles and attributes dynamically, and we'll introduce the concept of a **derived signal**.

Let's start with a simple component that should be familiar: click a button to increment a counter.

```
#[component]
fn App() -> impl IntoView {
    let (count, set_count) = signal(0);

    view! {
        <button
            on:click=move |_| {
                *set_count.write() += 1;
            }
        >
            "Click me: "
            {count}
        </button>
    }
}
```

So far, we've covered all of this in the previous chapter.

Dynamic Classes

Now let's say I'd like to update the list of CSS classes on this element dynamically. For example, let's say I want to add the class `red` when the count is odd. I can do this using the `class:` syntax.

```
class:red=move || count.get() % 2 == 1
```

`class:` attributes take

1. the class name, following the colon (`red`)
2. a value, which can be a `bool` or a function that returns a `bool`

When the value is `true`, the class is added. When the value is `false`, the class is removed. And if the value is a function that accesses a signal, the class will reactively update when the signal changes.

Now every time I click the button, the text should toggle between red and black as the number switches between even and odd.

```
<button
  on:click=move |_| {
    *set_count.write() += 1;
  }
  // the class: syntax reactively updates a single class
  // here, we'll set the `red` class when `count` is odd
  class:red=move || count.get() % 2 == 1
>
  "Click me"
</button>
```

If you're following along, make sure you go into your `index.html` and add something like this:

```
<style>
  .red {
    color: red;
  }
</style>
```

Some CSS class names can't be directly parsed by the `view` macro, especially if they include a mix of dashes and numbers or other characters. In that case, you can use a tuple syntax: `class=("name", value)` still directly updates a single class.

```
class=("button-20", move || count.get() % 2 == 1)
```

The tuple syntax also allows specifying multiple classes under a single condition using an array as the first tuple element.

```
class=(["button-20", "rounded"], move || count.get() % 2 == 1)
```

Dynamic Styles

Individual CSS properties can be directly updated with a similar `style:` syntax.

```

let (count, set_count) = signal(0);

view! {
  <button
    on:click=move |_| {
      *set_count.write() += 10;
    }
    // set the `style` attribute
    style="position: absolute"
    // and toggle individual CSS properties with `style:`
    style:left=move || format!("{}", count.get() + 100)
    style:background-color=move || format!("rgb({}, {}, 100)", count.get(),
100)
    style:max-width="400px"
    // Set a CSS variable for stylesheet use
    style=( "--columns", move || count.get().to_string())
  >
    "Click to Move"
  </button>
}

```

Dynamic Attributes

The same applies to plain attributes. Passing a plain string or primitive value to an attribute gives it a static value. Passing a function (including a signal) to an attribute causes it to update its value reactively. Let's add another element to our view:

```

<progress
  max="50"
  // signals are functions, so `value=count` and `value=move || count.get()`
  // are interchangeable.
  value=count
/>

```

Now every time we set the count, not only will the `class` of the `<button>` be toggled, but the `value` of the `<progress>` bar will increase, which means that our progress bar will move forward.

Derived Signals

Let's go one layer deeper, just for fun.

You already know that we create reactive interfaces just by passing functions into the `view`. This means that we can easily change our progress bar. For example, suppose we want it to move twice as fast:

```
<progress
  max="50"
  value=move || count.get() * 2
/>
```

But imagine we want to reuse that calculation in more than one place. You can do this using a **derived signal**: a closure that accesses a signal.

```
let double_count = move || count.get() * 2;

/* insert the rest of the view */
<progress
  max="50"
  // we use it once here
  value=double_count
/>
<p>
  "Double Count: "
  // and again here
  {double_count}
</p>
```

Derived signals let you create reactive computed values that can be used in multiple places in your application with minimal overhead.

Note: Using a derived signal like this means that the calculation runs once per signal change (when `count()` changes) and once per place we access `double_count`; in other words, twice. This is a very cheap calculation, so that's fine. We'll look at memos in a later chapter, which were designed to solve this problem for expensive calculations.

Advanced Topic: Injecting Raw HTML

The `view` macro provides support for an additional attribute, `inner_html`, which can be used to directly set the HTML contents of any element, wiping out any other children you've given it. Note that this does *not* escape the HTML you provide. You should make sure that it only contains trusted input or that any HTML entities are escaped, to prevent cross-site scripting (XSS) attacks.

```
let html = "<p>This HTML will be injected.</p>";
view! {
  <div inner_html=html/>
}
```

[Click here for the full `view` macros docs.](#)

□ [Live example](#)



► [CodeSandbox Source](#)

Components and Props

So far, we've been building our whole application in a single component. This is fine for really tiny examples, but in any real application you'll need to break the user interface out into multiple components, so you can break your interface down into smaller, reusable, composable chunks.

Let's take our progress bar example. Imagine that you want two progress bars instead of one: one that advances one tick per click, one that advances two ticks per click.

You *could* do this by just creating two `<progress>` elements:

```
let (count, set_count) = signal(0);
let double_count = move || count.get() * 2;

view! {
  <progress
    max="50"
    value=count
  />
  <progress
    max="50"
    value=double_count
  />
}
```

But of course, this doesn't scale very well. If you want to add a third progress bar, you need to add this code another time. And if you want to edit anything about it, you need to edit it in triplicate.

Instead, let's create a `<ProgressBar/>` component.

```
#[component]
fn ProgressBar() -> impl IntoView {
  view! {
    <progress
      max="50"
      // hmm... where will we get this from?
      value=progress
    />
  }
}
```

There's just one problem: `progress` is not defined. Where should it come from? When we were defining everything manually, we just used the local variable names. Now we need some way to pass an argument into the component.

Component Props

We do this using component properties, or “props.” If you’ve used another frontend framework, this is probably a familiar idea. Basically, properties are to components as attributes are to HTML elements: they let you pass additional information into the component.

In Leptos, you define props by giving additional arguments to the component function.

```
#[component]
fn ProgressBar(
    progress: ReadSignal<i32>
) -> impl IntoView {
    view! {
        <progress
            max="50"
            // now this works
            value=progress
        />
    }
}
```

Now we can use our component in the main `<App/>` component’s view.

```
#[component]
fn App() -> impl IntoView {
    let (count, set_count) = signal(0);
    view! {
        <button on:click=move |_| *set_count.write() += 1>
            "Click me"
        </button>
        // now we use our component!
        <ProgressBar progress=count/>
    }
}
```

Using a component in the view looks a lot like using an HTML element. You’ll notice that you can easily tell the difference between an element and a component because components always have `PascalCase` names. You pass the `progress` prop in as if it were an HTML element attribute. Simple.

Reactive and Static Props

You’ll notice that throughout this example, `progress` takes a reactive `ReadSignal<i32>`, and not a plain `i32`. This is **very important**.

Component props have no special meaning attached to them. A component is simply a function that runs once to set up the user interface. The only way to tell the interface to

respond to changes is to pass it a signal type. So if you have a component property that will change over time, like our `progress`, it should be a signal.

optional Props

Right now the `max` setting is hard-coded. Let's take that as a prop too. But let's make this prop optional. We can do that by annotating it with `#[prop(optional)]`.

```
#[component]
fn ProgressBar(
    // mark this prop optional
    // you can specify it or not when you use <ProgressBar/>
    #[prop(optional)]
    max: u16,
    progress: ReadSignal<i32>
) -> impl IntoView {
    view! {
        <progress
            max=max
            value=progress
        />
    }
}
```

Now, we can use `<ProgressBar max=50 progress=count/>`, or we can omit `max` to use the default value (i.e., `<ProgressBar progress=count/>`). The default value on an `optional` is its `Default::default()` value, which for a `u16` is going to be `0`. In the case of a progress bar, a `max` value of `0` is not very useful.

So let's give it a particular default value instead.

default props

You can specify a default value other than `Default::default()` pretty simply with `#[prop(default = ...)]`.

```
#[component]
fn ProgressBar(
    #[prop(default = 100)]
    max: u16,
    progress: ReadSignal<i32>
) -> impl IntoView {
    view! {
        <progress
            max=max
            value=progress
        />
    }
}
```

Generic Props

This is great. But we began with two counters, one driven by `count`, and one by the derived signal `double_count`. Let's recreate that by using `double_count` as the `progress` prop on another `<ProgressBar/>`.

```
#[component]
fn App() -> impl IntoView {
    let (count, set_count) = signal(0);
    let double_count = move || count.get() * 2;

    view! {
        <button on:click=move |_| { set_count.update(|n| *n += 1); }>
            "Click me"
        </button>
        <ProgressBar progress=count/>
        // add a second progress bar
        <ProgressBar progress=double_count/>
    }
}
```

Hm... this won't compile. It should be pretty easy to understand why: we've declared that the `progress` prop takes `ReadSignal<i32>`, and `double_count` is not `ReadSignal<i32>`. As rust-analyzer will tell you, its type is `|| -> i32`, i.e., it's a closure that returns an `i32`.

There are a couple ways to handle this. One would be to say: "Well, I know that for the view to be reactive, it needs to take a function or a signal. I can always turn a signal into a function by wrapping it in a closure... Maybe I could just take any function?"

If you're using nightly Rust with the `nightly` feature, signals are functions, so you could use a generic component and take any `Fn() -> i32`:

```
#[component]
fn ProgressBar(
    #[prop(default = 100)]
    max: u16,
    progress: impl Fn() -> i32 + Send + Sync + 'static
) -> impl IntoView {
    view! {
        <progress
            max=max
            value=progress
        />
        // Add a line-break to avoid overlap
        <br/>
    }
}
```

Generic props can also be specified using a `where` clause, or using inline generics like `ProgressBar<F: Fn() -> i32 + 'static>`.

You can specify a generic in the view using the syntax for expressing types:

`<Component<T>/>` (not with the turbofish-style `<Component::<T>/>`).

```
#[component]
fn SizeOf<T: Sized>(&[prop(marker)] _ty: PhantomData<T>) -> impl IntoView {
    std::mem::size_of::<T>()
}

#[component]
pub fn App() -> impl IntoView {
    view! {
        <SizeOf<usize>/>
        <SizeOf<String>/>
    }
}
```

Note that there are some limitations. For example, our view macro parser can't handle nested generics like `<SizeOf<Vec<T>>/>`.

marker Props

Generics need to be used somewhere in the component props. This is because props are built into a struct, so all generic types must be used somewhere in the struct. This is easily accomplished using an optional `PhantomData` prop: `#[prop(optional)] _ty: PhantomData<T>`, but this generates some docs and a setter for the `_ty` field, you can use `#[prop(marker)]` for types that are always defaulted. This will remove the props from the docs and the builder, and will `#[serde(skip)]` the field for islands.

into Props

If you're on stable Rust, signals don't directly implement `Fn()`. We could wrap the signal in a closure `(move || progress.get())` but that's a bit messy.

There's another way we could implement this, and it would be to use `#[prop(into)]`. This attribute automatically calls `.into()` on the values you pass as props, which allows you to easily pass props with different values.

In this case, it's helpful to know about the `Signal` type. `Signal` is an enumerated type that represents any kind of readable reactive signal, or a plain value. It can be useful when defining APIs for components you'll want to reuse while passing different sorts of signals.

```
#[component]
fn ProgressBar(
    #[prop(default = 100)]
    max: u16,
    #[prop(into)]
    progress: Signal<i32>
) -> impl IntoView
{
    view! {
        <progress
            max=max
            value=progress
        />
        <br/>
    }
}

#[component]
fn App() -> impl IntoView {
    let (count, set_count) = signal(0);
    let double_count = move || count.get() * 2;

    view! {
        <button on:click=move |_| *set_count.write() += 1>
            "Click me"
        </button>
        // .into() converts `ReadSignal` to `Signal`
        <ProgressBar progress=count/>
        // use `Signal::derive()` to wrap a derived signal with the `Signal`
    }
}

type
    <ProgressBar progress=Signal::derive(double_count)/>
}
```

Optional Generic Props

Note that you can't specify optional generic props for a component. Let's see what would happen if you try:

```

#[component]
fn ProgressBar<F: Fn() -> i32 + Send + Sync + 'static>(
    #[prop(optional)] progress: Option<F>,
) -> impl IntoView {
    progress.map(|progress| {
        view! {
            <progress
                max=100
                value=progress
            />
            <br/>
        }
    })
}

#[component]
pub fn App() -> impl IntoView {
    view! {
        <ProgressBar/>
    }
}

```

Rust helpfully gives the error

```

xx |           <ProgressBar/>
    |           ^^^^^^^^^^^^^ cannot infer type of the type parameter `F` declared
on the function `ProgressBar`
    |
help: consider specifying the generic argument
    |
xx |           <ProgressBar::<F>/>
    |                       ++++++

```

You can specify generics on components with a `<ProgressBar<F>/>` syntax (no turbofish in the `view` macro). Specifying the correct type here is not possible; closures and functions in general are unnameable types. The compiler can display them with a shorthand, but you can't specify them.

However, you can get around this by providing a concrete type using `Box<dyn _>` or `&dyn _`:

```

#[component]
fn ProgressBar(
    #[prop(optional)] progress: Option<Box<dyn Fn() -> i32 + Send + Sync>>,
) -> impl IntoView {
    progress.map(|progress| {
        view! {
            <progress
                max=100
                value=progress
            />
            <br/>
        }
    })
}

#[component]
pub fn App() -> impl IntoView {
    view! {
        <ProgressBar/>
    }
}

```

Because the Rust compiler now knows the concrete type of the prop, and therefore its size in memory even in the `None` case, this compiles fine.

In this particular case, `&dyn Fn() -> i32` will cause lifetime issues, but in other cases, it may be a possibility.

Documenting Components

This is one of the least essential but most important sections of this book. It's not strictly necessary to document your components and their props. It may be very important, depending on the size of your team and your app. But it's very easy, and bears immediate fruit.

To document a component and its props, you can simply add doc comments on the component function, and each one of the props:

```
/// Shows progress toward a goal.
#[component]
fn ProgressBar(
    /// The maximum value of the progress bar.
    #[prop(default = 100)]
    max: u16,
    /// How much progress should be displayed.
    #[prop(into)]
    progress: Signal<i32>,
) -> impl IntoView {
    /* ... */
}
```

That's all you need to do. These behave like ordinary Rust doc comments, except that you can document individual component props, which can't be done with Rust function arguments.

This will automatically generate documentation for your component, its `Props` type, and each of the fields used to add props. It can be a little hard to understand how powerful this is until you hover over the component name or props and see the power of the `#[component]` macro combined with rust-analyzer [here](#).

Spreading Attributes onto Components

Sometimes you want users to be able to add additional attributes to a component. For example, you might want users to be able to add their own `class` or `id` attributes for styling or other purposes.

You *could* do this by creating `class` or `id` props that you then apply to the appropriate element. But Leptos also supports “spreading” additional attributes onto components. Attributes added to a component will be applied to all top-level HTML elements returned from its view.

```

// you can create attribute lists by using the view macro with a spread {...} as
the tag name
let spread_onto_component = view! {
  <{...} aria-label="a component with attribute spreading"/>
};

view! {
  // attributes that are spread onto a component will be applied to *all*
  elements returned as part of
  // the component's view. to apply attributes to a subset of the component,
  pass them via a component prop
  <ComponentThatTakesSpread
    // plain identifiers are for props
    some_prop="foo"
    another_prop=42

    // the class:, style:, prop:, on: syntaxes work just as they do on
    elements
    class:foo=true
    style:font-weight="bold"
    prop:cool=42
    on:click=move |_| alert("clicked ComponentThatTakesSpread")

    // to pass a plain HTML attribute, prefix it with attr:
    attr:id="foo"

    // or, if you want to include multiple attributes, rather than
    prefixing each with
    // attr:, you can separate them from component props with the spread
    {...}
    {...} // everything after this is treated as an HTML attribute
    title="ooh, a title!"

    // we can add the whole list of attributes defined above
    {...spread_onto_component}
  />
}

```

 Note

If you would like to extract the attributes into a function so you can use it in multiple components, you can do so by implementing a function that returns `impl Attribute`.

That would make the example above look like this:

```
fn spread_onto_component() -> impl Attribute {  
    view!{  
        <{..} aria-label="a component with attribute spreading"/>  
    }  
}  
  
view!{  
    <SomeComponent {..spread_onto_component()} />  
}
```

If you want to spread attributes onto a component, but want to apply the attributes to something other than all top-level elements, use [AttributeInterceptor](#).

See the [spread example](#) for more examples.

[□ Live example](#)[▶ CodeSandbox Source](#)

Iteration

Whether you're listing todos, displaying a table, or showing product images, iterating over a list of items is a common task in web applications. Reconciling the differences between changing sets of items can also be one of the trickiest tasks for a framework to handle well.

Leptos supports two different patterns for iterating over items:

1. For static views: `Vec<_>`
2. For dynamic lists: `<For/>`

Static Views with `Vec<_>`

Sometimes you need to show an item repeatedly, but the list you're drawing from does not often change. In this case, it's important to know that you can insert any `Vec<IV>` where `IV: IntoView` into your view. In other words, if you can render `T`, you can render `Vec<T>`.

```
let values = vec![0, 1, 2];
view! {
    // this will just render "012"
    <p>{values.clone()}</p>
    // or we can wrap them in <li>
    <ul>
        {values.into_iter()
            .map(|n| view! { <li>{n}</li>})
            .collect::<Vec<_>>()}
    </ul>
}
```

Leptos also provides a `.collect_view()` helper function that allows you to collect any iterator of `T: IntoView` into `Vec<View>`.

```
let values = vec![0, 1, 2];
view! {
    // this will just render "012"
    <p>{values.clone()}</p>
    // or we can wrap them in <li>
    <ul>
        {values.into_iter()
            .map(|n| view! { <li>{n}</li>})
            .collect_view()}
    </ul>
}
```

The fact that the *list* is static doesn't mean the interface needs to be static. You can render dynamic items as part of a static list.

```
// create a list of 5 signals
let length = 5;
let counters = (1..=length).map(|idx| RwSignal::new(idx));
```

Note here that instead of calling `signal()` to get a tuple with a reader and a writer, here we use `RwSignal::new()` to get a single, read-write signal. This is just more convenient for a situation where we'd otherwise be passing the tuples around.

```
// each item manages a reactive view
// but the list itself will never change
let counter_buttons = counters
    .map(|count| {
        view! {
            <li>
                <button
                    on:click=move |_| *count.write() += 1
                >
                    {count}
                </button>
            </li>
        }
    })
    .collect_view();

view! {
    <ul>{counter_buttons}</ul>
}
```

You *can* render a `Fn() -> Vec<_>` reactively as well. But note that this is an unkeyed list update: it will reuse the existing DOM elements, and update them with the new values, according to their order in the new `Vec<_>`. If you're just adding and removing items at the end of the list, this works well, but if you are moving items around or inserting items into the middle of the list, this will cause the browser to do more work than it needs to, and may have surprising effects on things like input state and CSS animations. (For more on the "keyed" vs. "unkeyed" distinction, and some practical examples, you can read [this article](#).)

Luckily, there's an efficient way to do keyed list iteration, as well.

Dynamic Rendering with the `<For/>` Component

The `<For/>` component is a keyed dynamic list. It takes three props:

- `each`: a reactive function that returns the items `T` to be iterated over
- `key`: a key function that takes `&T` and returns a stable, unique key or ID
- `children`: renders each `T` into a view

`key` is, well, the key. You can add, remove, and move items within the list. As long as each item's key is stable over time, the framework does not need to rerender any of the items, unless they are new additions, and it can very efficiently add, remove, and move items as they change. This allows for extremely efficient updates to the list as it changes, with minimal additional work.

Creating a good `key` can be a little tricky. You generally do *not* want to use an index for this purpose, as it is not stable—if you remove or move items, their indices change.

But it's a great idea to do something like generating a unique ID for each row as it is generated, and using that as an ID for the key function.

Check out the `<DynamicList/>` component below for an example.

□ [Live example](#)



► [CodeSandbox Source](#)

Accessing an index while iterating with `<ForEnumerate/>`

For the cases where you need to access the real-time index while iterating, Leptos provides a `<ForEnumerate/>` component.

The props are identical to the `<For/>` component, but when rendering `children` it additionally provides a `ReadSignal<usize>` parameter as the index:

```
#[derive(Copy, Clone, Debug, PartialEq, Eq)]
struct Counter {
    id: usize,
    count: RwSignal<i32>
}

<ForEnumerate
    each=move || counters.get() // Same as <For/>
    key=|counter| counter.id     // Same as <For/>
    // Provides the index as a signal and the child T
    children={move |index: ReadSignal<usize>, counter: Counter| {
        view! {
            <button>{move || index.get()} ". Value: " {move ||
counter.count.get()}</button>
        }
    }}
/>
```

or it could also be used with the more convenient `let` syntax:

```
<ForEnumerate
  each=move || counters.get() // Same as <For/>
  key=|counter| counter.id    // Same as <For/>
  let(idx, counter)           // let syntax
>
  <button>{move || idx.get()} ". Value: " {move || counter.count.get()}
</button>
</ ForEnumerate>
```

Iterating over More Complex Data with `<For/>`

This chapter goes into iteration over nested data structures in a bit more depth. It belongs here with the other chapter on iteration, but feel free to skip it and come back if you'd like to stick with simpler subjects for now.

The Problem

I just said that the framework does not rerender any of the items in one of the rows, unless the key has changed. This probably makes sense at first, but it can easily trip you up.

Let's consider an example in which each of the items in our row is some data structure. Imagine, for example, that the items come from some JSON array of keys and values:

```
#[derive(Debug, Clone)]
struct DatabaseEntry {
    key: String,
    value: i32,
}
```

Let's define a simple component that will iterate over the rows and display each one:

```
#[component]
pub fn App() -> impl IntoView {
    // start with a set of three rows
    let (data, set_data) = signal(vec![
        DatabaseEntry {
            key: "foo".to_string(),
            value: 10,
        },
        DatabaseEntry {
            key: "bar".to_string(),
            value: 20,
        },
        DatabaseEntry {
            key: "baz".to_string(),
            value: 15,
        },
    ]);
    view! {
        // when we click, update each row,
        // doubling its value
        <button on:click=move |_| {
            set_data.update(|data| {
                for row in data {
                    row.value *= 2;
                }
            });
            // log the new value of the signal
            leptos::logging::log!("{:?}", data.get());
        }>
        "Update Values"
    </button>
    // iterate over the rows and display each value
    <For
        each=move || data.get()
        key=|state| state.key.clone()
        let(child)
    >
        <p>{child.value}</p>
    </For>
    }
}
```

Note the `let(child)` syntax here. In the previous chapter we introduced `<For/>` with a `children` prop. We can actually create this value directly in the children of the `<For/>` component, without breaking out of the `view` macro: the `let(child)` combined with `<p>{child.value}</p>` above is the equivalent of

```
children=|child| view! { <p>{child.value}</p> }
```

For convenience, you can also choose to destructure the pattern of your data:

```
<For
  each=move || data.get()
  key=|state| state.key.clone()
  let(DatabaseEntry { key, value })
>
```

When you click the `update values` button... nothing happens. Or rather: the signal is updated, the new value is logged, but the `{child.value}` for each row doesn't update.

Let's see: is that because we forgot to add a closure to make it reactive? Let's try `{move || child.value}`.

...Nope. Still nothing.

Here's the problem: as I said, each row is only rerendered when the key changes. We've updated the value for each row, but not the key for any of the rows, so nothing has rerendered. And if you look at the type of `child.value`, it's a plain `i32`, not a reactive `ReadSignal<i32>` or something. This means that even if we wrap a closure around it, the value in this row will never update.

We have four possible solutions:

1. change the `key` so that it always updates when the data structure changes
2. change the `value` so that it's reactive
3. take a reactive slice of the data structure instead of using each row directly
4. use a `Store`

Option 1: Change the Key

Each row is only rerendered when the key changes. Our rows above didn't rerender, because the key didn't change. So: why not just force the key to change?

```
<For
  each=move || data.get()
  key=|state| (state.key.clone(), state.value)
  let(child)
>
  <p>{child.value}</p>
</For>
```

Now we include both the key and the value in the `key`. This means that whenever the value of a row changes, `<For/>` will treat it as if it's an entirely new row, and replace the previous one.

Pros

This is very easy. We can make it even easier by deriving `PartialEq`, `Eq`, and `Hash` on `DatabaseEntry`, in which case we could just `key=|state| state.clone()`.

Cons

This is the least efficient of the four options. Every time the value of a row changes, it throws out the previous `<p>` element and replaces it with an entirely new one. Rather than making a fine-grained update to the text node, in other words, it really does rerender the entire row on every change, and this is expensive in proportion to how complex the UI of the row is.

You'll notice we also end up cloning the whole data structure so that `<For/>` can hold onto a copy of the key. For more complex structures, this can become a bad idea fast!

Option 2: Nested Signals

If we do want that fine-grained reactivity for the value, one option is to wrap the `value` of each row in a signal.

```
#[derive(Debug, Clone)]
struct DatabaseEntry {
    key: String,
    value: RwSignal<i32>,
}
```

`RwSignal<_>` is a “read-write signal,” which combines the getter and setter in one object. I’m using it here because it’s a little easier to store in a struct than separate getters and setters.

```
#[component]
pub fn App() -> impl IntoView {
    // start with a set of three rows
    let (data, _set_data) = signal(vec![
        DatabaseEntry {
            key: "foo".to_string(),
            value: RwSignal::new(10),
        },
        DatabaseEntry {
            key: "bar".to_string(),
            value: RwSignal::new(20),
        },
        DatabaseEntry {
            key: "baz".to_string(),
            value: RwSignal::new(15),
        },
    ]);
    view! {
        // when we click, update each row,
        // doubling its value
        <button on:click=move |_| {
            for row in &*data.read() {
                row.value.update(|value| *value *= 2);
            }
            // log the new value of the signal
            leptos::logging::log!("{:?}", data.get());
        }>
        "Update Values"
    </button>
    // iterate over the rows and display each value
    <For
        each=move || data.get()
        key=|state| state.key.clone()
        let(child)
    >
        <p>{child.value}</p>
    </For>
    }
}
```

This version works! And if you look in the DOM inspector in your browser, you'll see that unlike in the previous version, in this version only the individual text nodes are updated. Passing the signal directly into `{child.value}` works, as signals do keep their reactivity if you pass them into the view.

Note that I changed the `set_data.update()` to a `data.read()`. `.read()` is a non-cloning way of accessing a signal's value. In this case, we are only updating the inner values, not updating the list of values: because signals maintain their own state, we don't actually need to update the `data` signal at all, so the immutable `.read()` is fine here.

In fact, this version doesn't update `data`, so the `<For/>` is essentially a static list as in the last chapter, and this could just be a plain iterator. But the `<For/>` is useful if we

want to add or remove rows in the future.

Pros

This is the most efficient option, and fits directly with the rest of the mental model of the framework: values that change over time are wrapped in signals so the interface can respond to them.

Cons

Nested reactivity can be cumbersome if you're receiving data from an API or another data source you don't control, and you don't want to create a different struct wrapping each field in a signal.

Option 3: Memoized Slices

Leptos provides a primitive called a `Memo`, which creates a derived computation that only triggers a reactive update when its value has changed.

This allows you to create reactive values for subfields of a larger data structure, without needing to wrap the fields of that structure in signals. In combination with `<ForEnumerate/>`, this will allow us to rerender only changed data values.

Most of the application can remain the same as the initial (broken) version, but the `<For/>` will be updated to this:

```
<ForEnumerate
  each=move || data.get()
  key=|state| state.key.clone()
  children=move |index, _| {
    let value = Memo::new(move |_| {
      data.with(|data| data.get(index.get()).map(|d|
d.value).unwrap_or(0))
    });
    view! {
      <p>{value}</p>
    }
  }
/>
```

You'll notice a few differences here:

- we use `ForEnumerate` rather than `For`, so we have access to an `index` signal
- we use the `children` prop explicitly, to make it easier to run some non-`view` code

- we define a `value` memo and use that in the view. This `value` field doesn't actually use the `child` being passed into each row. Instead, it uses the index and reaches back into the original `data` to get the value.

Now every time `data` changes, each memo will be recalculated. If its value has changed, it will update its text node, without rerendering the whole row.

Note: It is not safe to use `For` for this with an enumerated iterator, as in an earlier version of this example:

```
<For
  each=move || data.get().into_iter().enumerate()
  key=|(_, state)| state.key.clone()
  children=move |(index, _)| {
    let value = Memo::new(move |_| {
      data.with(|data| data.get(index).map(|d| d.value).unwrap_or(0))
    });
    view! {
      <p>{value}</p>
    }
  }
/>
```

In this case, changes to values in `data` will be reacted to, but changes to ordering will not, as the Memo will always use the `index` it was initially created with. This will result in duplicate entries in the rendered output if any items are moved.

Pros

We get the same fine-grained reactivity of the signal-wrapped version, without needing to wrap the data in signals.

Cons

It's a bit more complex to set up this memo-per-row inside the `<ForEnumerate/>` loop rather than using nested signals. For example, you'll notice that we have to guard against the possibility that the `data[index.get()]` would panic by using `data.get(index.get())`, because this memo may be triggered to re-run once just after the row is removed. (This is because the memo for each row and the whole `<ForEnumerate/>` both depend on the same `data` signal, and the order of execution for multiple reactive values that depend on the same signal isn't guaranteed.)

Note also that while memos memoize their reactive changes, the same calculation does need to re-run to check the value every time, so nested reactive signals will still be more efficient for pinpoint updates here.

Option 4: Stores

Some of this content is duplicated in the section on global state management with stores [here](#). Both sections are intermediate/optional content, so I thought some duplication couldn't hurt.

Leptos 0.7 introduces a new reactive primitive called “stores.” Stores are designed to address the issues described in this chapter so far. They're a bit experimental, so they require an additional dependency called `reactive_stores` in your `Cargo.toml`.

Stores give you fine-grained reactive access to the individual fields of a struct, and to individual items in collections like `Vec<_>`, without needing to create nested signals or memos manually, as in the options given above.

Stores are built on top of the `store` derive macro, which creates a getter for each field of a struct. Calling this getter gives reactive access to that particular field. Reading from it will track only that field and its parents/children, and updating it will only notify that field and its parents/children, but not siblings. In other words, mutating `value` will not notify `key`, and so on.

We can adapt the data types we used in the examples above.

The top level of a store always needs to be a struct, so we'll create a `Data` wrapper with a single `rows` field.

```
#[derive(Store, Debug, Clone)]
pub struct Data {
    #[store(key: String = |row| row.key.clone())]
    rows: Vec<DatabaseEntry>,
}

#[derive(Store, Debug, Clone)]
struct DatabaseEntry {
    key: String,
    value: i32,
}
```

Adding `#[store(key)]` to the `rows` field allows us to have keyed access to the fields of the store, which will be useful in the `<For/>` component below. We can simply use `key`, the same key that we'll use in `<For/>`.

The `<For/>` component is pretty straightforward:

```

<For
  each=move || data.rows()
  key=|row| row.read().key.clone()
  children=|child| {
    let value = child.value();
    view! { <p>{move || value.get()}</p> }
  }
/>

```

Because `rows` is a keyed field, it implements `IntoIterator`, and we can simply use `move || data.rows()` as the `each` prop. This will react to any changes to the `rows` list, just as `move || data.get()` did in our nested-signal version.

The `key` field calls `.read()` to get access to the current value of the row, then clones and returns the `key` field.

In `children` prop, calling `child.value()` gives us reactive access to the `value` field for the row with this key. If rows are reordered, added, or removed, the keyed store field will keep in sync so that this `value` is always associated with the correct key.

In the update button handler, we'll iterate over the entries in `rows`, updating each one:

```

for row in data.rows().iter_unkeyed() {
  *row.value().write() *= 2;
}

```

Pros

We get the fine-grained reactivity of the nested-signal and memo versions, without needing to manually create nested signals or memoized slices. We can work with plain data (a struct and `Vec<_>`), annotated with a `derive` macro, rather than special nested reactive types.

Personally, I think the stores version is the nicest one here. And no surprise, as it's the newest API. We've had a few years to think about these things and stores include some of the lessons we've learned!

Cons

On the other hand, it's the newest API. As of writing this sentence (December 2024), stores have only been released for a few weeks; I am sure that there are still some bugs or edge cases to be figured out.

Full Example

Here's the complete store example. You can find another, more complete example [here](#), and more discussion in the book [here](#).

```

use reactive_stores::Store;

#[derive(Store, Debug, Clone)]
pub struct Data {
    #[store(key: String = |row| row.key.clone())]
    rows: Vec<DatabaseEntry>,
}

#[derive(Store, Debug, Clone)]
struct DatabaseEntry {
    key: String,
    value: i32,
}

#[component]
pub fn App() -> impl IntoView {
    // instead of a signal with the rows, we create a store for Data
    let data = Store::new(Data {
        rows: vec![
            DatabaseEntry {
                key: "foo".to_string(),
                value: 10,
            },
            DatabaseEntry {
                key: "bar".to_string(),
                value: 20,
            },
            DatabaseEntry {
                key: "baz".to_string(),
                value: 15,
            },
        ],
    });

    view! {
        // when we click, update each row,
        // doubling its value
        <button on:click=move |_| {
            // allows iterating over the entries in an iterable store field
            use reactive_stores::StoreFieldIterator;

            // calling rows() gives us access to the rows
            for row in data.rows().iter_unkeyed() {
                *row.value().write() *= 2;
            }
            // log the new value of the signal
            leptos::logging::log!("{:?}", data.get());
        }>
        "Update Values"
    </button>
    // iterate over the rows and display each value
    <For
        each=move || data.rows()
        key=|row| row.read().key.clone()
        children=|child| {
            let value = child.value();
            view! { <p>{move || value.get()}</p> }
        }
    </For>

```

```
}  
  }  
  />  
}
```

Forms and Inputs

Forms and form inputs are an important part of interactive apps. There are two basic patterns for interacting with inputs in Leptos, which you may recognize if you're familiar with React, SolidJS, or a similar framework: using **controlled** or **uncontrolled** inputs.

Controlled Inputs

In a "controlled input," the framework controls the state of the input element. On every `input` event, it updates a local signal that holds the current state, which in turn updates the `value` prop of the input.

There are two important things to remember:

1. The `input` event fires on (almost) every change to the element, while the `change` event fires (more or less) when you unfocus the input. You probably want `on:input`, but we give you the freedom to choose.
2. The `value` *attribute* only sets the initial value of the input, i.e., it only updates the input up to the point that you begin typing. The `value` *property* continues updating the input after that. You usually want to set `prop:value` for this reason. (The same is true for `checked` and `prop:checked` on an `<input type="checkbox">`.)

```
let (name, set_name) = signal("Controlled".to_string());

view! {
  <input type="text"
    // adding :target gives us typed access to the element
    // that is the target of the event that fires
    on:input:target=move |ev| {
      // .value() returns the current value of an HTML input element
      set_name.set(ev.target().value());
    }

    // the `prop:` syntax lets you update a DOM property,
    // rather than an attribute.
    prop:value=name
  />
  <p>"Name is: " {name}</p>
}
```

Why do you need `prop:value`?

Web browsers are the most ubiquitous and stable platform for rendering graphical user interfaces in existence. They have also maintained an incredible backwards compatibility over their three decades of existence. Inevitably, this means there are some quirks.

One odd quirk is that there is a distinction between HTML attributes and DOM element properties, i.e., between something called an “attribute” which is parsed from HTML and can be set on a DOM element with `.setAttribute()`, and something called a “property” which is a field of the JavaScript class representation of that parsed HTML element.

In the case of an `<input value=...>`, setting the `value attribute` is defined as setting the initial value for the input, and setting `value property` sets its current value. It may be easier to understand this by opening `about:blank` and running the following JavaScript in the browser console, line by line:

```
// create an input and append it to the DOM
const el = document.createElement("input");
document.body.appendChild(el);

el.setAttribute("value", "test"); // updates the input
el.setAttribute("value", "another test"); // updates the input again

// now go and type into the input: delete some characters, etc.

el.setAttribute("value", "one more time?");
// nothing should have changed. Setting the "initial value" does nothing
now

// however...
el.value = "But this works";
```

Many other frontend frameworks conflate attributes and properties, or create a special case for inputs that sets the value correctly. Maybe Leptos should do this too, but for now, I prefer giving users the maximum amount of control over whether they’re setting an attribute or a property, and doing my best to educate people about the actual underlying browser behavior rather than obscuring it.

Simplifying Controlled Inputs with `bind`:

Adherence to Web standards and a clear division between “reading from a signal” and “writing to a signal” are good, but creating controlled inputs in this way can sometimes seem like more boilerplate than is really necessary.

Leptos also includes a special `bind:` syntax for inputs that allows you to automatically bind signals to inputs. They do exactly the same thing as the “controlled input” pattern above: create an event listener that updates the signal, and a dynamic property that reads from the signal. You can use `bind:value` for text inputs, `bind:checked` for checkboxes, and `bind:group` for radio button groups.

```
let (name, set_name) = signal("Controlled".to_string());
let email = RwSignal::new("").to_string();
let favorite_color = RwSignal::new("red".to_string());
let spam_me = RwSignal::new(true);

view! {
  <input type="text"
    bind:value=(name, set_name)
  />
  <input type="email"
    bind:value=email
  />
  <label>
    "Please send me lots of spam email."
    <input type="checkbox"
      bind:checked=spam_me
    />
  </label>
  <fieldset>
    <legend>"Favorite color"</legend>
    <label>
      "Red"
      <input
        type="radio"
        name="color"
        value="red"
        bind:group=favorite_color
      />
    </label>
    <label>
      "Green"
      <input
        type="radio"
        name="color"
        value="green"
        bind:group=favorite_color
      />
    </label>
    <label>
      "Blue"
      <input
        type="radio"
        name="color"
        value="blue"
        bind:group=favorite_color
      />
    </label>
  </fieldset>
  <p>"Your favorite color is " {favorite_color} "."</p>
  <p>"Name is: " {name}</p>
  <p>"Email is: " {email}</p>
  <Show when=move || spam_me.get()>
    <p>"You'll receive cool bonus content!"</p>
  </Show>
}
```

Uncontrolled Inputs

In an "uncontrolled input," the browser controls the state of the input element. Rather than continuously updating a signal to hold its value, we use a `NodeRef` to access the input when we want to get its value.

In this example, we only notify the framework when the `<form>` fires a `submit` event. Note the use of the `leptos::html` module, which provides a bunch of types for every HTML element.

```
let (name, set_name) = signal("Uncontrolled".to_string());

let input_element: NodeRef<html::Input> = NodeRef::new();

view! {
  <form on:submit=on_submit> // on_submit defined below
    <input type="text"
      value=name
      node_ref=input_element
    />
    <input type="submit" value="Submit"/>
  </form>
  <p>"Name is: " {name}</p>
}
```

The view should be pretty self-explanatory by now. Note two things:

1. Unlike in the controlled input example, we use `value` (not `prop:value`). This is because we're just setting the initial value of the input, and letting the browser control its state. (We could use `prop:value` instead.)
2. We use `node_ref=...` to fill the `NodeRef`. (Older examples sometimes use `_ref`. They are the same thing, but `node_ref` has better rust-analyzer support.)

`NodeRef` is a kind of reactive smart pointer: we can use it to access the underlying DOM node. Its value will be set when the element is rendered.

```

let on_submit = move |ev: SubmitEvent| {
    // stop the page from reloading!
    ev.prevent_default();

    // here, we'll extract the value from the input
    let value = input_element
        .get()
        // event handlers can only fire after the view
        // is mounted to the DOM, so the `NodeRef` will be `Some`
        .expect("<input> should be mounted")
        // `leptos::HtmlElement<html::Input>` implements `Deref`
        // to a `web_sys::HtmlInputElement`.
        // this means we can call `HtmlInputElement::value()`
        // to get the current value of the input
        .value();
    set_name.set(value);
};

```

Our `on_submit` handler will access the input's value and use it to call `set_name.set()`. To access the DOM node stored in the `NodeRef`, we can simply call it as a function (or using `.get()`). This will return `Option<leptos::HtmlElement<html::Input>>`, but we know that the element has already been mounted (how else did you fire this event!), so it's safe to unwrap here.

We can then call `.value()` to get the value out of the input, because `NodeRef` gives us access to a correctly-typed HTML element.

Take a look at [web_sys](#) and [HtmlElement](#) to learn more about using a `leptos::HtmlElement`. Also, see the full [CodeSandbox](#) example at the end of this page.

Special Cases: `<textarea>` and `<select>`

Two form elements tend to cause some confusion in different ways.

`<textarea>`

Unlike `<input>`, the `<textarea>` element does not support a `value` attribute in HTML. Instead, it receives its initial value as a plain text node in its HTML children.

So if you'd like to server render an initial value, and have the value also react in the browser, you can both pass it an initial text node as a child and use `prop:value` to set its current value.

```
view! {
  <textarea
    prop:value=move || some_value.get()
    on:input:target=move |ev| some_value.set(ev.target().value())
  >
    {some_value}
  </textarea>
}
```

<select>

The `<select>` element can likewise be controlled via a `value` property on the `<select>` itself, which will select whichever `<option>` has that value.

```
let (value, set_value) = signal(0i32);
view! {
  <select
    on:change:target=move |ev| {
      set_value.set(ev.target().value().parse().unwrap());
    }
    prop:value=move || value.get().to_string()
  >
    <option value="0">"0"</option>
    <option value="1">"1"</option>
    <option value="2">"2"</option>
  </select>
  // a button that will cycle through the options
  <button on:click=move |_| set_value.update(|n| {
    if *n == 2 {
      *n = 0;
    } else {
      *n += 1;
    }
  })>
    "Next Option"
  </button>
}
```

□ [Controlled vs uncontrolled forms CodeSandbox](#)



► [CodeSandbox Source](#)

Control Flow

In most applications, you sometimes need to make a decision: Should I render this part of the view, or not? Should I render `<ButtonA/>` or `<WidgetB/>`? This is **control flow**.

A Few Tips

When thinking about how to do this with Leptos, it's important to remember a few things:

1. Rust is an expression-oriented language: control-flow expressions like `if x() { y } else { z }` and `match x() { ... }` return their values. This makes them very useful for declarative user interfaces.
2. For any `T` that implements `IntoView`—in other words, for any type that Leptos knows how to render—`Option<T>` and `Result<T, impl Error>` also implement `IntoView`. And just as `Fn() -> T` renders a reactive `T`, `Fn() -> Option<T>` and `Fn() -> Result<T, impl Error>` are reactive.
3. Rust has lots of handy helpers like `Option::map`, `Option::and_then`, `Option::ok_or`, `Result::map`, `Result::ok`, and `bool::then` that allow you to convert, in a declarative way, between a few different standard types, all of which can be rendered. Spending time in the `Option` and `Result` docs in particular is one of the best ways to level up your Rust game.
4. And always remember: to be reactive, values must be functions. You'll see me constantly wrap things in a `move ||` closure, below. This is to ensure that they actually rerun when the signal they depend on changes, keeping the UI reactive.

So What?

To connect the dots a little: this means that you can actually implement most of your control flow with native Rust code, without any control-flow components or special knowledge.

For example, let's start with a simple signal and derived signal:

```
let (value, set_value) = signal(0);
let is_odd = move || value.get() % 2 != 0;
```

We can use these signals and ordinary Rust to build most control flow.

if statements

Let's say I want to render some text if the number is odd, and some other text if it's even. Well, how about this?

```
view! {
  <p>
    {move || if is_odd() {
      "Odd"
    } else {
      "Even"
    }}
  </p>
}
```

An `if` expression returns its value, and a `&str` implements `IntoView`, so a `Fn() -> &str` implements `IntoView`, so this... just works!

Option<T>

Let's say we want to render some text if it's odd, and nothing if it's even.

```
let message = move || {
  if is_odd() {
    Some("Ding ding ding!")
  } else {
    None
  }
};

view! {
  <p>{message}</p>
}
```

This works fine. We can make it a little shorter if we'd like, using `bool::then()`.

```
let message = move || is_odd().then(|| "Ding ding ding!");
view! {
  <p>{message}</p>
}
```

You could even inline this if you'd like, although personally I sometimes like the better `cargo fmt` and `rust-analyzer` support I get by pulling things out of the `view`.

match statements

We're still just writing ordinary Rust code, right? So you have all the power of Rust's pattern matching at your disposal.

```
let message = move || {  
    match value.get() {  
        0 => "Zero",  
        1 => "One",  
        n if is_odd() => "Odd",  
        _ => "Even"  
    }  
};  
view! {  
    <p>{message}</p>  
}
```

And why not? YOLO, right?

Preventing Over-Rendering

Not so YOLO.

Everything we've just done is basically fine. But there's one thing you should remember and try to be careful with. Each one of the control-flow functions we've created so far is basically a derived signal: it will rerun every time the value changes. In the examples above, where the value switches from even to odd on every change, this is fine.

But consider the following example:

```
let (value, set_value) = signal(0);  
  
let message = move || if value.get() > 5 {  
    "Big"  
} else {  
    "Small"  
};  
  
view! {  
    <p>{message}</p>  
}
```

This *works*, for sure. But if you added a log, you might be surprised

```
let message = move || if value.get() > 5 {
  logging::log!("{}", rendering Big", value.get());
  "Big"
} else {
  logging::log!("{}", rendering Small", value.get());
  "Small"
};
```

As a user repeatedly clicks a button incrementing `value`, you'd see something like this:

```
1: rendering Small
2: rendering Small
3: rendering Small
4: rendering Small
5: rendering Small
6: rendering Big
7: rendering Big
8: rendering Big
... ad infinitum
```

Every time `value` changes, it reruns the `if` statement. This makes sense, with how reactivity works. But it has a downside. For a simple text node, rerunning the `if` statement and rerendering isn't a big deal. But imagine it were like this:

```
let message = move || if value.get() > 5 {
  <Big/>
} else {
  <Small/>
};
```

This rerenders `<Small/>` five times, then `<Big/>` infinitely. If they're loading resources, creating signals, or even just creating DOM nodes, this is unnecessary work.

<Show/>

The `<Show/>` component is the answer. You pass it a `when` condition function, a `fallback` to be shown if the `when` function returns `false`, and children to be rendered if `when` is `true`.

```
let (value, set_value) = signal(0);

view! {
  <Show
    when=move || { value.get() > 5 }
    fallback=|| view! { <Small/> }
  >
  <Big/>
</Show>
}
```

`<Show/>` memoizes the `when` condition, so it only renders its `<Small/>` once, continuing to show the same component until `value` is greater than five; then it renders `<Big/>` once, continuing to show it indefinitely or until `value` goes below five and then renders `<Small/>` again.

This is a helpful tool to avoid rerendering when using dynamic `if` expressions. As always, there's some overhead: for a very simple node (like updating a single text node, or updating a class or attribute), a `move || if ...` will be more efficient. But if it's at all expensive to render either branch, reach for `<Show/>`.

Note: Type Conversions

There's one final thing it's important to say in this section.

Leptos uses a statically-typed view tree. The `view` macro returns different types for different kinds of view.

This won't compile, because the different HTML elements are different types.

```
view! {
    <main>
        {move || match is_odd() {
            true if value.get() == 1 => {
                view! { <pre>"One"</pre> }
            },
            false if value.get() == 2 => {
                view! { <p>"Two"</p> }
            }
            // returns HtmlElement<Textarea>
            _ => view! { <textarea>{value.get()}</textarea> }
        }}
    </main>
}
```

This strong typing is very powerful, because it enables all sorts of compile-time optimizations. But it can be a little annoying in conditional logic like this, because you can't return different types from different branches of a condition in Rust. There are two ways to get yourself out of this situation:

1. Use the enum `Either` (and `EitherOf3`, `EitherOf4`, etc.) to convert the different types to the same type.
2. Use `.into_any()` to convert multiple types into one typed-erased `AnyView`.

Here's the same example, with the conversion added:

```
view! {  
  <main>  
    {move || match is_odd() {  
      true if value.get() == 1 => {  
        // returns HtmlElement<Pre>  
        view! { <pre>"One"</pre> }.into_any()  
      },  
      false if value.get() == 2 => {  
        // returns HtmlElement<P>  
        view! { <p>"Two"</p> }.into_any()  
      }  
    }  
    // returns HtmlElement<Textarea>  
    _ => view! { <textarea>{value.get()}</textarea> }.into_any()  
  }  
}</main>  
}
```

[□ Live example](#)[▶ CodeSandbox Source](#)

Error Handling

In the last chapter, we saw that you can render `Option<T>`: in the `None` case, it will render nothing, and in the `Some(T)` case, it will render `T` (that is, if `T` implements `IntoView`). You can actually do something very similar with a `Result<T, E>`. In the `Err(_)` case, it will render nothing. In the `Ok(T)` case, it will render the `T`.

Let's start with a simple component to capture a number input.

```
#[component]
fn NumericInput() -> impl IntoView {
    let (value, set_value) = signal(Ok(0));

    view! {
        <label>
            "Type an integer (or not!)"
            <input type="number" on:input:target=move |ev| {
                // when input changes, try to parse a number from the input
                set_value.set(ev.target().value().parse::<i32>())
            }/>
            <p>
                "You entered "
                <strong>{value}</strong>
            </p>
        </label>
    }
}
```

Every time you change the input, `on_input` will attempt to parse its value into a 32-bit integer (`i32`), and store it in our `value` signal, which is a `Result<i32, _>`. If you type the number `42`, the UI will display

You entered 42

But if you type the string `foo`, it will display

You entered

This is not great. It saves us using `.unwrap_or_default()` or something, but it would be much nicer if we could catch the error and do something with it.

You can do that, with the `<ErrorBoundary/>` component.

 Note

People often try to point out that `<input type="number">` prevents you from typing a string like `foo`, or anything else that's not a number. This is true in some browsers, but not in all! Moreover, there are a variety of things that can be typed into a plain number input that are not an `i32`: a floating-point number, a larger-than-32-bit number, the letter `e`, and so on. The browser can be told to uphold some of these invariants, but browser behavior still varies: Parsing for yourself is important!

`<ErrorBoundary/>`

An `<ErrorBoundary/>` is a little like the `<Show/>` component we saw in the last chapter. If everything's okay—which is to say, if everything is `ok(_)`—it renders its children. But if there's an `Err(_)` rendered among those children, it will trigger the `<ErrorBoundary/>`'s `fallback`.

Let's add an `<ErrorBoundary/>` to this example.

```

#[component]
fn NumericInput() -> impl IntoView {
    let (value, set_value) = signal(Ok(0));

    view! {
        <h1>"Error Handling"</h1>
        <label>
            "Type a number (or something that's not a number!)"
            <input type="number" on:input:target=move |ev| {
                // when input changes, try to parse a number from the input
                set_value.set(ev.target().value().parse::<i32>())
            }/>
            // If an `Err(_)` had been rendered inside the <ErrorBoundary/>,
            // the fallback will be displayed. Otherwise, the children of the
            // <ErrorBoundary/> will be displayed.
            <ErrorBoundary
                // the fallback receives a signal containing current errors
                fallback=|errors| view! {
                    <div class="error">
                        <p>"Not a number! Errors: "</p>
                        // we can render a list of errors
                        // as strings, if we'd like
                        <ul>
                            {move || errors.get()
                                .into_iter()
                                .map(|(_, e)| view! { <li>{e.to_string()}
                                    </li>}})
                                .collect::<Vec<_>>()}
                            }
                        </ul>
                    </div>
                }
            >
            <p>
                "You entered "
                // because `value` is `Result<i32, _>`,
                // it will render the `i32` if it is `Ok`,
                // and render nothing and trigger the error boundary
                // if it is `Err`. It's a signal, so this will dynamically
                // update when `value` changes
                <strong>{value}</strong>
            </p>
        </ErrorBoundary>
    </label>
    }
}

```

Now, if you type 42, value is `Ok(42)` and you'll see

You entered 42

If you type `foo`, value is `Err(_)` and the `fallback` will render. We've chosen to render the list of errors as a `String`, so you'll see something like

Not a number! Errors:

- cannot parse integer from empty string

If you fix the error, the error message will disappear and the content you're wrapping in an `<ErrorBoundary/>` will appear again.

[□ Live example](#)



► [CodeSandbox Source](#)

Parent-Child Communication

You can think of your application as a nested tree of components. Each component handles its own local state and manages a section of the user interface, so components tend to be relatively self-contained.

Sometimes, though, you'll want to communicate between a parent component and its child. For example, imagine you've defined a `<FancyButton/>` component that adds some styling, logging, or something else to a `<button/>`. You want to use a `<FancyButton/>` in your `<App/>` component. But how can you communicate between the two?

It's easy to communicate state from a parent component to a child component. We covered some of this in the material on [components and props](#). Basically if you want the parent to communicate to the child, you can pass either a `ReadSignal` or `Signal` as a prop.

But what about the other direction? How can a child send notifications about events or state changes back up to the parent?

There are four basic patterns of parent-child communication in Leptos.

1. Pass a `WriteSignal`

One approach is simply to pass a `WriteSignal` from the parent down to the child, and update it in the child. This lets you manipulate the state of the parent from the child.

```
#[component]
pub fn App() -> impl IntoView {
    let (toggled, set_toggled) = signal(false);
    view! {
        <p>"Toggled? " {toggled}</p>
        <ButtonA setter=set_toggled/>
    }
}

#[component]
pub fn ButtonA(setter: WriteSignal<bool>) -> impl IntoView {
    view! {
        <button
            on:click=move |_| setter.update(|value| *value = !*value)
            >
            "Toggle"
        </button>
    }
}
```

This pattern is simple, but you should be careful with it: passing around a `WriteSignal` can make it hard to reason about your code. In this example, it's pretty clear when you read `<App/>` that you are handing off the ability to mutate `toggled`, but it's not at all clear when or how it will change. In this small, local example it's easy to understand, but if you find yourself passing around `WriteSignal`s like this throughout your code, you should really consider whether this is making it too easy to write spaghetti code.

2. Use a Callback

Another approach would be to pass a callback to the child: say, `on_click`.

```
#[component]
pub fn App() -> impl IntoView {
    let (toggled, set_toggled) = signal(false);
    view! {
        <p>"Toggled? " {toggled}</p>
        <ButtonB on_click=move |_| set_toggled.update(|value| *value =
!*value)/>
    }
}

#[component]
pub fn ButtonB(on_click: impl FnMut(MouseEvent) + 'static) -> impl IntoView {
    view! {
        <button on:click=on_click>
            "Toggle"
        </button>
    }
}
```

You'll notice that whereas `<ButtonA/>` was given a `WriteSignal` and decided how to mutate it, `<ButtonB/>` simply fires an event: the mutation happens back in `<App/>`. This has the advantage of keeping local state local, preventing the problem of spaghetti mutation. But it also means the logic to mutate that signal needs to exist up in `<App/>`, not down in `<ButtonB/>`. These are real trade-offs, not a simple right-or-wrong choice.

3. Use an Event Listener

You can actually write Option 2 in a slightly different way. If the callback maps directly onto a native DOM event, you can add an `on:` listener directly to the place you use the component in your `view` macro in `<App/>`.

```
#[component]
pub fn App() -> impl IntoView {
    let (toggled, set_toggled) = signal(false);
    view! {
        <p>"Toggled? " {toggled}</p>
        // note the on:click instead of on_click
        // this is the same syntax as an HTML element event listener
        <ButtonC on:click=move |_| set_toggled.update(|value| *value =
!*value)/>
    }
}

#[component]
pub fn ButtonC() -> impl IntoView {
    view! {
        <button>"Toggle"</button>
    }
}
```

This lets you write way less code in `<ButtonC/>` than you did for `<ButtonB/>`, and still gives a correctly-typed event to the listener. This works by adding an `on:` event listener to each element that `<ButtonC/>` returns: in this case, just the one `<button>`.

Of course, this only works for actual DOM events that you're passing directly through to the elements you're rendering in the component. For more complex logic that doesn't map directly onto an element (say you create `<ValidatedForm/>` and want an `on_valid_form_submit` callback) you should use Option 2.

4. Providing a Context

This version is actually a variant on Option 1. Say you have a deeply-nested component tree:

```

#[component]
pub fn App() -> impl IntoView {
    let (toggled, set_toggled) = signal(false);
    view! {
        <p>"Toggled? " {toggled}</p>
        <Layout/>
    }
}

#[component]
pub fn Layout() -> impl IntoView {
    view! {
        <header>
            <h1>"My Page"</h1>
        </header>
        <main>
            <Content/>
        </main>
    }
}

#[component]
pub fn Content() -> impl IntoView {
    view! {
        <div class="content">
            <ButtonD/>
        </div>
    }
}

#[component]
pub fn ButtonD() -> impl IntoView {
    todo!()
}

```

Now `<ButtonD/>` is no longer a direct child of `<App/>`, so you can't simply pass your `writeSignal` to its props. You could do what's sometimes called "prop drilling," adding a prop to each layer between the two:

```

#[component]
pub fn App() -> impl IntoView {
    let (toggled, set_toggled) = signal(false);
    view! {
        <p>"Toggled? " {toggled}</p>
        <Layout set_toggled/>
    }
}

#[component]
pub fn Layout(set_toggled: WriteSignal<bool>) -> impl IntoView {
    view! {
        <header>
            <h1>"My Page"</h1>
        </header>
        <main>
            <Content set_toggled/>
        </main>
    }
}

#[component]
pub fn Content(set_toggled: WriteSignal<bool>) -> impl IntoView {
    view! {
        <div class="content">
            <ButtonD set_toggled/>
        </div>
    }
}

#[component]
pub fn ButtonD(set_toggled: WriteSignal<bool>) -> impl IntoView {
    todo!()
}

```

This is a mess. `<Layout/>` and `<Content/>` don't need `set_toggled`; they just pass it through to `<ButtonD/>`. But I need to declare the prop in triplicate. This is not only annoying but hard to maintain: imagine we add a "half-toggled" option and the type of `set_toggled` needs to change to an `enum`. We have to change it in three places!

Isn't there some way to skip levels?

There is!

4.1 The Context API

You can provide data that skips levels by using `provide_context` and `use_context`. Contexts are identified by the type of the data you provide (in this example, `WriteSignal<bool>`), and they exist in a top-down tree that follows the contours of your UI tree. In this example, we can use context to skip the unnecessary prop drilling.

```
#[component]
pub fn App() -> impl IntoView {
    let (toggled, set_toggled) = signal(false);

    // share `set_toggled` with all children of this component
    provide_context(set_toggled);

    view! {
        <p>"Toggled? " {toggled}</p>
        <Layout/>
    }
}

// <Layout/> and <Content/> omitted
// To work in this version, drop the `set_toggled` parameter on each

#[component]
pub fn ButtonD() -> impl IntoView {
    // use_context searches up the context tree, hoping to
    // find a `WriteSignal<bool>`
    // in this case, I .expect() because I know I provided it
    let setter = use_context::<WriteSignal<bool>>().expect("to have found the
setter provided");

    view! {
        <button
            on:click=move |_| setter.update(|value| *value = !*value)
        >
            "Toggle"
        </button>
    }
}
```

The same caveats apply to this as to `<ButtonA/>`: passing a `WriteSignal` around should be done with caution, as it allows you to mutate state from arbitrary parts of your code. But when done carefully, this can be one of the most effective techniques for global state management in Leptos: simply provide the state at the highest level you'll need it, and use it wherever you need it lower down.

Note that there are no performance downsides to this approach. Because you are passing a fine-grained reactive signal, *nothing happens* in the intervening components (`<Layout/>` and `<Content/>`) when you update it. You are communicating directly between `<ButtonD/>` and `<App/>` . In fact—and this is the power of fine-grained reactivity—you are communicating directly between a button click in `<ButtonD/>` and a single text node in `<App/>` . It's as if the components themselves don't exist at all. And, well... at runtime, they don't. It's just signals and effects, all the way down.

Note that this approach makes an important tradeoff: You don't have type-safety anymore between `provide_context` and `use_context` . Receiving the right context in the child component is a runtime check (see `use_context.expect(...)`). The compiler won't guide you during a refactoring, as it does with the earlier approaches.

[□ Live example](#)[▶ CodeSandbox Source](#)

Component Children

It's pretty common to want to pass children into a component, just as you can pass children into an HTML element. For example, imagine I have a `<FancyForm/>` component that enhances an HTML `<form>`. I need some way to pass all its inputs.

```
view! {  
  <FancyForm>  
    <fieldset>  
      <label>  
        "Some Input"  
        <input type="text" name="something"/>  
      </label>  
    </fieldset>  
    <button>"Submit"</button>  
  </FancyForm>  
}
```

How can you do this in Leptos? There are basically two ways to pass components to other components:

1. **render props**: properties that are functions that return a view
2. the **children** prop: a special component property that includes anything you pass as a child to the component.

In fact, you've already seen these both in action in the `<Show/>` component:

```
view! {  
  <Show  
    // `when` is a normal prop  
    when=move || value.get() > 5  
    // `fallback` is a "render prop": a function that returns a view  
    fallback=|| view! { <Small/> }  
  >  
    // `    // will be given to the `children` prop  
    <Big/>  
  </Show>  
}
```

Let's define a component that takes some children and a render prop.

```

/// Displays a `render_prop` and some children within markup.
#[component]
pub fn TakesChildren<F, IV>(
    /// Takes a function (type F) that returns anything that can be
    /// converted into a View (type IV)
    render_prop: F,
    /// `children` can take one of several different types, each of which
    /// is a function that returns some view type
    children: Children,
) -> impl IntoView
where
    F: Fn() -> IV,
    IV: IntoView,
{
    view! {
        <h1><code>"<TakesChildren/>"</code></h1>
        <h2>"Render Prop"</h2>
        {render_prop()}
        <hr/>
        <h2>"Children"</h2>
        {children()}
    }
}

```

`render_prop` and `children` are both functions, so we can call them to generate the appropriate views. `children`, in particular, is an alias for `Box<dyn FnOnce() -> AnyView>`. (Aren't you glad we named it `children` instead?) The `AnyView` returned here is an opaque, type-erased view: you can't do anything to inspect it. There are a variety of other child types: for example, `ChildrenFragment` will return a `Fragment`, which is a collection whose children can be iterated over.

If you need a `Fn` or `FnMut` here because you need to call `children` more than once, we also provide `ChildrenFn` and `ChildrenMut` aliases.

We can use the component like this:

```

view! {
    <TakesChildren render_prop=|| view! { <p>"Hi, there!"</p> }>
    // these get passed to `children`
    "Some text"
    <span>"A span"</span>
    </TakesChildren>
}

```

Typed Children: Slots

So far, we have discussed components with a single `children` prop, but sometimes it is helpful to create a component with multiple children of different types. For example:

```
view! {
  <If condition=a_is_true>
    <Then>"Show content when a is true"</Then>
    <ElseIf condition=b_is_true>"b is true"</ElseIf>
    <ElseIf condition=c_is_true>"c is true"</ElseIf>
    <Else>"None of the above are true"</Else>
  </If>
}
```

The `If` component always expects a `Then` child, optionally multiple `ElseIf` children and an optional `Else` child. To handle this, Leptos provides [slots](#).

The `#[slot]` macro annotates a plain Rust struct as component slot:

```
// A simple struct annotated with `#[slot]`,
// which expects children
#[slot]
struct Then {
  children: ChildrenFn,
}
```

This slot can be used as a prop in a component:

```
#[component]
fn If(
  condition: Signal<bool>,
  // Component slot, should be passed through the <Then slot> syntax
  then_slot: Then,
) -> impl IntoView {
  move || {
    if condition.get() {
      (then_slot.children)().into_any()
    } else {
      ().into_any()
    }
  }
}
```

Now, the `If` component expects a child of type `Then`. You would need to annotate the used slot with `slot:<prop_name>`:

```
view! {
  <If condition=a_is_true>
    // The `If` component always expects a `Then` child for `then_slot`
    <Then slot:then_slot>"Show content when a is true"</Then>
  </If>
}
```

Specifying `slot` without a name will default the chosen slot as the snake case version of the struct name. So in this case `<Then slot>` would be equivalent to `<Then slot:then>`.

For the complete example, see [slots examples](#).

Event handlers on slots

Event handlers cannot be specified directly on slots like this:

```
<ComponentWithSlot>
  // ⚠ Event handler `on:click` directly on slot is not allowed
  <SlotWithChildren slot:slot on:click=move |_| {}>
    <h1>"Hello, World!"</h1>
  </SlotWithChildren>
</ComponentWithSlot>
```

Instead, wrap the slot content in a regular element and attach event handlers there:

```
<ComponentWithSlot>
  <SlotWithChildren slot:slot>
    // ✅ Event handler is not defined directly on slot
    <div on:click=move |_| {}>
      <h1>"Hello, World!"</h1>
    </div>
  </SlotWithChildren>
</ComponentWithSlot>
```

Manipulating Children

The `Fragment` type is basically a way of wrapping a `Vec<AnyView>`. You can insert it anywhere into your view.

But you can also access those inner views directly to manipulate them. For example, here's a component that takes its children and turns them into an unordered list.

```
/// Wraps each child in an `- ` and embeds them in a `
`.
#[component]
pub fn WrapsChildren(children: ChildrenFragment) -> impl IntoView {
    // children() returns a `Fragment`, which has a
    // `nodes` field that contains a Vec<View>
    // this means we can iterate over the children
    // to create something new!
    let children = children()
        .nodes
        .into_iter()
        .map(|child| view! { <li>{child}</li> })
        .collect::<Vec<_>>();

    view! {
        <h1><code>"<WrapsChildren/>"</code></h1>
        // wrap our wrapped children in a UL
        <ul>{children}</ul>
    }
}

```

Calling it like this will create a list:

```
view! {
    <WrapsChildren>
        "A"
        "B"
        "C"
    </WrapsChildren>
}
```

[□ Live example](#)



► [CodeSandbox Source](#)

No Macros: The View Builder Syntax

If you're perfectly happy with the `view!` macro syntax described so far, you're welcome to skip this chapter. The builder syntax described in this section is always available, but never required.

For one reason or another, many developers would prefer to avoid macros. Perhaps you don't like the limited `rustfmt` support. (Although, you should check out [leptosfmt](#), which is an excellent tool!) Perhaps you worry about the effect of macros on compile time. Perhaps you prefer the aesthetics of pure Rust syntax, or you have trouble context-switching between an HTML-like syntax and your Rust code. Or perhaps you want more flexibility in how you create and manipulate HTML elements than the `view` macro provides.

If you fall into any of those camps, the builder syntax may be for you.

The `view` macro expands an HTML-like syntax to a series of Rust functions and method calls. If you'd rather not use the `view` macro, you can simply use that expanded syntax yourself. And it's actually pretty nice!

First off, if you want you can even drop the `#[component]` macro: a component is just a setup function that creates your view, so you can define a component as a simple function call:

```
pub fn counter(initial_value: i32, step: u32) -> impl IntoView { }
```

Elements are created by calling a function with the same name as the HTML element:

```
p()
```

Custom elements/web components can be created by using the `custom()` function with their name:

```
custom("my-custom-element")
```

You can add children to the element with `.child()`, which takes a single child or a tuple or array of types that implement `IntoView`.

```
p().child((em().child("Big, "), strong().child("bold "), "text"))
```

Attributes are added with `.attr()`. This can take any of the same types that you could pass as an attribute into the view macro (types that implement `Attribute`).

```
p().attr("id", "foo")
    .attr("data-count", move || count.get().to_string())
```

They can also be added with attribute methods, which are available for any built-in HTML attribute name:

```
p().id("foo")
    .attr("data-count", move || count.get().to_string())
```

Similarly, the `class:`, `prop:`, and `style:` syntaxes map directly onto `.class()`, `.prop()`, and `.style()` methods.

Event listeners can be added with `.on()`. Typed events found in `leptos::ev` prevent typos in event names and allow for correct type inference in the callback function.

```
button()
    .on(ev::click, move |_| set_count.set(0))
    .child("Clear")
```

All of this adds up to a very Rusty syntax to build full-featured views, if you prefer this style.

```
/// A simple counter view.
// A component is really just a function call: it runs once to create the DOM
// and reactive system
pub fn counter(initial_value: i32, step: i32) -> impl IntoView {
    let (count, set_count) = signal(initial_value);
    div().child((
        button()
            // typed events found in leptos::ev
            // 1) prevent typos in event names
            // 2) allow for correct type inference in callbacks
            .on(ev::click, move |_| set_count.set(0))
            .child("Clear"),
        button()
            .on(ev::click, move |_| *set_count.write() -= step)
            .child("-1"),
        span().child(("Value: ", move || count.get(), "!")),
        button()
            .on(ev::click, move |_| *set_count.write() += step)
            .child("+1"),
    ))
}
```

Using Components with the Builder Syntax

To create your own components with the builder syntax, you can simply use plain functions (see above). To use other components (for example, the built-in `For` or `Show` control-flow

components), you can take advantage of the fact that each component is a function of one component props argument, and component props have their own builder.

You can either use the component props builder:

```
use leptos::html::p;

let (value, set_value) = signal(0);

Show(
  ShowProps::builder()
    .when(move || value.get() > 5)
    .fallback(|| p().child("I will appear if `value` is 5 or lower"))
    .children(ToChildren::to_children(|| {
      p().child("I will appear if `value` is above 5")
    }))
    .build(),
)
```

or you can directly build the props struct:

```
use leptos::html::p;

let (value, set_value) = signal(0);

Show(ShowProps {
  when: move || value.get() > 5,
  fallback: (|| p().child("I will appear if `value` is 5 or lower")).into(),
  children: ToChildren::to_children(|| p().child("I will appear if `value` is
above 5")),
})
```

Using the component builder correctly applies the various modifiers like `#[prop(into)]`; using the struct syntax, we've applied this manually by calling `.into()` ourselves.

Expanding Macros

Not every feature of the `view` macro or `component` macro syntax has been described here in detail. However, Rust provides you the tools you need to understand what is going on with any macro. Specifically, rust-analyzer's ["expand macro recursively" feature](#) allows you to expand any macro to show the code it generates, and [cargo-expand](#) expands all the macros in a project into regular Rust code. The rest of this book will continue using the `view` macro syntax, but if you're ever unsure how to translate this into the builder syntax, you can use these tools to explore the code that is generated.

Reactivity

Leptos is built on top of a fine-grained reactive system, designed to run expensive side effects (like rendering something in a browser, or making a network request) as infrequently as possible in response to change, reactive values.

So far we've seen signals in action. These chapters will go into a bit more depth, and look at effects, which are the other half of the story.

Working with Signals

So far we've used some simple examples of using `signal`, which returns a `ReadSignal` getter and a `WriteSignal` setter.

Getting and Setting

There are a few basic signal operations:

Getting

1. `.read()` returns a read guard which dereferences to the value of the signal, and tracks any future changes to the value of the signal reactively. Note that you cannot update the value of the signal until this guard is dropped, or it will cause a runtime error.
2. `.with()` takes a function, which receives the current value of the signal by reference (`&T`), and tracks the signal.
3. `.get()` clones the current value of the signal and tracks further changes to the value.

`.get()` is the most common method of accessing a signal. `.read()` is useful for methods that take an immutable reference, without cloning the value

(`my_vec_signal.read().len()`). `.with()` is useful if you need to do more with that reference, but want to make sure you don't hold onto the lock longer than you need.

Setting

1. `.write()` returns a write guard which is a mutable reference to the value of the signal, and notifies any subscribers that they need to update. Note that you cannot read from the value of the signal until this guard is dropped, or it will cause a runtime error.
2. `.update()` takes a function, which receives a mutable reference to the current value of the signal (`&mut T`), and notifies subscribers. (`.update()` doesn't return the value returned by the closure, but you can use `.try_update()` if you need to; for example, if you're removing an item from a `Vec<_>` and want the removed item.)
3. `.set()` replaces the current value of the signal and notifies subscribers.

`.set()` is most common for setting a new value; `.write()` is very useful for updating a value in place. Just as is the case with `.read()` and `.with()`, `.update()` can be useful when you want to avoid the possibility of holding on the write lock longer than you intended to.

 Note

These traits are based on trait composition and provided by blanket implementations. For example, `Read` is implemented for any type that implements `Track` and `ReadUntracked`. `With` is implemented for any type that implements `Read`. `Get` is implemented for any type that implements `With` and `Clone`. And so on.

Similar relationships exist for `Write`, `Update`, and `Set`.

This is worth noting when reading docs: if you only see `ReadUntracked` and `Track` as implemented traits, you will still be able to use `.with()`, `.get()` (if `T: Clone`), and so on.

Working with Signals

You might notice that `.get()` and `.set()` can be implemented in terms of `.read()` and `.write()`, or `.with()` and `.update()`. In other words, `count.get()` is identical to `count.with(|n| n.clone())` or `count.read().clone()`, and `count.set(1)` is implemented by doing `count.update(|n| *n = 1)` or `*count.write() = 1`.

But of course, `.get()` and `.set()` are nicer syntax.

However, there are some very good use cases for the other methods.

For example, consider a signal that holds a `Vec<String>`.

```
let (names, set_names) = signal(Vec::());
if names.get().is_empty() {
    set_names(vec!["Alice".to_string()]);
}
```

In terms of logic, this is simple enough, but it's hiding some significant inefficiencies.

Remember that `names.get().is_empty()` clones the value. This means we clone the whole `Vec<String>`, run `is_empty()`, and then immediately throw away the clone.

Likewise, `set_names` replaces the value with a whole new `Vec<_>`. This is fine, but we might as well just mutate the original `Vec<_>` in place.

```
let (names, set_names) = signal(Vec::());
if names.read().is_empty() {
    set_names.write().push("Alice".to_string());
}
```

Now our function simply takes `names` by reference to run `is_empty()` , avoiding that clone, and then mutates the `vec<_>` in place.

Thread Safety and Thread-Local Values

You may have noticed, either by reading the docs or by experimenting with your own applications, that the values that are stored in signals must be `Send + Sync` . This is because the reactive system actually supports multi-threading: signals can be sent across threads, and the whole reactive graph can work across multiple threads. (This is especially useful when doing [server-side rendering](#) with server frameworks like Axum, which use Tokio’s multi-threaded executor.) In most cases, this has no effect on what you do: ordinary Rust data types are `Send + Sync` by default.

However, the browser environment is only single-threaded unless you use a Web Worker, and the JavaScript types provided by `wasm-bindgen` and `web-sys` are all explicitly `!Send` . This mean they can’t be stored in ordinary signals.

As a result, we provide “local” alternatives for each of the signal primitives, which can be used to store `!Send` data. You should only reach for these when you have a `!Send` browser type you need to store in a signal.

Standard	Local
<code>signal</code>	<code>signal_local</code>
<code>RwSignal::new</code>	<code>RwSignal::new_local</code>
<code>Resource</code>	<code>LocalResource</code>
<code>Action::new</code>	<code>Action::new_local</code> , <code>Action::new_unsync</code>

Nightly Syntax

When using the `nightly` feature and `nightly` syntax, calling a `ReadSignal` as a function is syntax sugar for `.get()` . Calling a `WriteSignal` as a function is syntax sugar for `.set()` . So

```
let (count, set_count) = signal(0);
set_count(1);
logging::log!(count());
```

is the same as

```
let (count, set_count) = signal(0);
set_count.set(1);
logging::log!(count.get());
```

This is not just syntax sugar, but makes for a more consistent API by making signals semantically the same thing as functions: see the [Interlude](#).

Making signals depend on each other

Often people ask about situations in which some signal needs to change based on some other signal's value. There are three good ways to do this, and one that's less than ideal but okay under controlled circumstances.

Good Options

1) B is a function of A. Create a signal for A and a derived signal or memo for B.

```
// A
let (count, set_count) = signal(1);
// B is a function of A
let derived_signal_double_count = move || count.get() * 2;
// B is a function of A
let memoized_double_count = Memo::new(move |_| count.get() * 2);
```

For guidance on whether to use a derived signal or a memo, see the docs for [Memo](#)

2) C is a function of A and some other thing B. Create signals for A and B and a derived signal or memo for C.

```
// A
let (first_name, set_first_name) = signal("Bridget".to_string());
// B
let (last_name, set_last_name) = signal("Jones".to_string());
// C is a function of A and B
let full_name = move || format!("{}", &*first_name.read(),
&*last_name.read());
```

3) A and B are independent signals, but sometimes updated at the same time. When you make the call to update A, make a separate call to update B.

```
// A
let (age, set_age) = signal(32);
// B
let (favorite_number, set_favorite_number) = signal(42);
// use this to handle a click on a `Clear` button
let clear_handler = move |_| {
    // update both A and B
    set_age.set(0);
    set_favorite_number.set(0);
};
```

If you really must...

4) Create an effect to write to B whenever A changes. This is officially discouraged, for several reasons: a) It will always be less efficient, as it means every time A updates you do two full trips through the reactive process. (You set A, which causes the effect to run, as well as any other effects that depend on A. Then you set B, which causes any effects that depend on B to run.) b) It increases your chances of accidentally creating things like infinite loops or over-re-running effects. This is the kind of ping-ponging, reactive spaghetti code that was common in the early 2010s and that we try to avoid with things like read-write segregation and discouraging writing to signals from effects.

In most situations, it's best to rewrite things such that there's a clear, top-down data flow based on derived signals or memos. But this isn't the end of the world.

I'm intentionally not providing an example here. Read the [Effect](#) docs to figure out how this would work.

Responding to Changes with Effects

We've made it this far without having mentioned half of the reactive system: effects.

Reactivity works in two halves: updating individual reactive values ("signals") notifies the pieces of code that depend on them ("effects") that they need to run again. These two halves of the reactive system are inter-dependent. Without effects, signals can change within the reactive system but never be observed in a way that interacts with the outside world. Without signals, effects run once but never again, as there's no observable value to subscribe to. Effects are quite literally "side effects" of the reactive system: they exist to synchronize the reactive system with the non-reactive world outside it.

The renderer uses effects to update parts of the DOM in response to changes in signals. You can create your own effects to synchronize the reactive system with the outside world in other ways.

`Effect::new` takes a function as its argument. It runs this function on the next "tick" of the reactive system. (So for example, if you use it in a component, it will run just *after* that component has been rendered.) If you access any reactive signal inside that function, it registers the fact that the effect depends on that signal. Whenever one of the signals that the effect depends on changes, the effect runs again.

```
let (a, set_a) = signal(0);
let (b, set_b) = signal(0);

Effect::new(move |_| {
    // immediately prints "Value: 0" and subscribes to `a`
    logging::log!("Value: {}", a.get());
});
```

The effect function is called with an argument containing whatever value it returned the last time it ran. On the initial run, this is `None`.

By default, effects **do not run on the server**. This means you can call browser-specific APIs within the effect function without causing issues. If you need an effect to run on the server, use `Effect::new_isomorphic`.

Auto-tracking and Dynamic Dependencies

If you're familiar with a framework like React, you might notice one key difference. React and similar frameworks typically require you to pass a "dependency array," an explicit set of variables that determine when the effect should rerun.

Because Leptos comes from the tradition of synchronous reactive programming, we don't need this explicit dependency list. Instead, we automatically track dependencies depending on which signals are accessed within the effect.

This has two effects (no pun intended). Dependencies are:

1. **Automatic:** You don't need to maintain a dependency list, or worry about what should or shouldn't be included. The framework simply tracks which signals might cause the effect to rerun, and handles it for you.
2. **Dynamic:** The dependency list is cleared and updated every time the effect runs. If your effect contains a conditional (for example), only signals that are used in the current branch are tracked. This means that effects rerun the absolute minimum number of times.

If this sounds like magic, and if you want a deep dive into how automatic dependency tracking works, [check out this video](#). (Apologies for the low volume!)

Effects as Zero-Cost-ish Abstraction

While they're not a "zero-cost abstraction" in the most technical sense—they require some additional memory use, exist at runtime, etc.—at a higher level, from the perspective of whatever expensive API calls or other work you're doing within them, effects are a zero-cost abstraction. They rerun the absolute minimum number of times necessary, given how you've described them.

Imagine that I'm creating some kind of chat software, and I want people to be able to display their full name, or just their first name, and to notify the server whenever their name changes:

```
let (first, set_first) = signal(String::new());
let (last, set_last) = signal(String::new());
let (use_last, set_use_last) = signal(true);

// this will add the name to the log
// any time one of the source signals changes
Effect::new(move |_| {
    logging::log!(
        "{}", if use_last.get() {
            format!("{}", first.get(), last.get())
        } else {
            first.get()
        },
    ),
})
});
```

If `use_last` is `true`, effect should rerun whenever `first`, `last`, or `use_last` changes. But if I toggle `use_last` to `false`, a change in `last` will never cause the full name to change. In fact, `last` will be removed from the dependency list until `use_last` toggles again. This saves us from sending multiple unnecessary requests to the API if I change `last` multiple times while `use_last` is still `false`.

To create an effect, or not to create an effect?

Effects are intended to synchronize the reactive system with the non-reactive world outside, not to synchronize between different reactive values. In other words: using an effect to read a value from one signal and set it in another is always sub-optimal.

If you need to define a signal that depends on the value of other signals, use a derived signal or a [Memo](#). Writing to a signal inside an effect isn't the end of the world, and it won't cause your computer to light on fire, but a derived signal or memo is always better—not only because the dataflow is clear, but because the performance is better.

```
let (a, set_a) = signal(0);

// ⚠️ not great
let (b, set_b) = signal(0);
Effect::new(move |_| {
    set_b.set(a.get() * 2);
});

// ✅ woo-hoo!
let b = move || a.get() * 2;
```

If you need to synchronize some reactive value with the non-reactive world outside—like a web API, the console, the filesystem, or the DOM—writing to a signal in an effect is a fine way to do that. In many cases, though, you'll find that you're really writing to a signal inside an event listener or something else, not inside an effect. In these cases, you should check out [leptos-use](#) to see if it already provides a reactive wrapping primitive to do that!

If you're curious for more information about when you should and shouldn't use `create_effect`, [check out this video](#) for a more in-depth consideration!

Effects and Rendering

We've managed to get this far without mentioning effects because they're built into the Leptos DOM renderer. We've seen that you can create a signal and pass it into the `view`

macro, and it will update the relevant DOM node whenever the signal changes:

```
let (count, set_count) = signal(0);

view! {
  <p>{count}</p>
}
```

This works because the framework essentially creates an effect wrapping this update. You can imagine Leptos translating this view into something like this:

```
let (count, set_count) = signal(0);

// create a DOM element
let document = leptos::document();
let p = document.create_element("p").unwrap();

// create an effect to reactively update the text
Effect::new(move |prev_value| {
  // first, access the signal's value and convert it to a string
  let text = count.get().to_string();

  // if this is different from the previous value, update the node
  if prev_value != Some(text) {
    p.set_text_content(&text);
  }

  // return this value so we can memoize the next update
  text
});
```

Every time `count` is updated, this effect will rerun. This is what allows reactive, fine-grained updates to the DOM.

Explicit Tracking with `Effect::watch()`

In addition to `Effect::new()`, Leptos provides an `Effect::watch()` function, which can be used to separate tracking and responding to changes by explicitly passing in a set of values to track.

`watch` takes three arguments. The `dependency_fn` argument is reactively tracked while `handler` and `immediate` are not. Whenever `dependency_fn` is changed, `handler` is run. If `immediate` is false, the `handler` will run only after the first change is detected of any signal that is accessed in `dependency_fn`. `watch` returns an `Effect`, which can be called with `.stop()` to stop tracking the dependencies.

```
let (num, set_num) = signal(0);

let effect = Effect::watch(
  move || num.get(),
  move |num, prev_num, _| {
    leptos::logging::log!("Number: {}; Prev: {:?}", num, prev_num);
  },
  false,
);

set_num.set(1); // > "Number: 1; Prev: Some(0)"

effect.stop(); // stop watching

set_num.set(2); // (nothing happens)
```

[□ Live example](#)[▶ CodeSandbox Source](#)

Interlude: Reactivity and Functions

One of our core contributors said to me recently: “I never used closures this often until I started using Leptos.” And it’s true. Closures are at the heart of any Leptos application. It sometimes looks a little silly:

```
// a signal holds a value, and can be updated
let (count, set_count) = signal(0);

// a derived signal is a function that accesses other signals
let double_count = move || count.get() * 2;
let count_is_odd = move || count.get() & 1 == 1;
let text = move || if count_is_odd() {
    "odd"
} else {
    "even"
};

// an effect automatically tracks the signals it depends on
// and reruns when they change
Effect::new(move |_| {
    logging::log!("text = {}", text());
});

view! {
    <p>{move || text().to_uppercase()}</p>
}
```

Closures, closures everywhere!

But why?

Functions and UI Frameworks

Functions are at the heart of every UI framework. And this makes perfect sense. Creating a user interface is basically divided into two phases:

1. initial rendering
2. updates

In a web framework, the framework does some kind of initial rendering. Then it hands control back over to the browser. When certain events fire (like a mouse click) or asynchronous tasks finish (like an HTTP request finishing), the browser wakes the framework back up to update something. The framework runs some kind of code to update your user interface, and goes back asleep until the browser wakes it up again.

The key phrase here is “runs some kind of code.” The natural way to “run some kind of code” at an arbitrary point in time—in Rust or in any other programming language—is to call a function. And in fact every UI framework is based on rerunning some kind of function over and over:

1. virtual DOM (VDOM) frameworks like React, Yew, or Dioxus rerun a component or render function over and over, to generate a virtual DOM tree that can be reconciled with the previous result to patch the DOM
2. compiled frameworks like Angular and Svelte divide your component templates into “create” and “update” functions, rerunning the update function when they detect a change to the component’s state
3. in fine-grained reactive frameworks like SolidJS, Sycamore, or Leptos, *you* define the functions that rerun

That’s what all our components are doing.

Take our typical `<SimpleCounter/>` example in its simplest form:

```
#[component]
pub fn SimpleCounter() -> impl IntoView {
    let (value, set_value) = signal(0);

    let increment = move |_| *set_value.write() += 1;

    view! {
        <button on:click=increment>
            {value}
        </button>
    }
}
```

The `SimpleCounter` function itself runs once. The `value` signal is created once. The framework hands off the `increment` function to the browser as an event listener. When you click the button, the browser calls `increment`, which updates `value` via `set_value`. And that updates the single text node represented in our view by `{value}`.

Functions are key to reactivity. They provide the framework with the ability to rerun the smallest possible unit of your application in response to a change.

So remember two things:

1. Your component function is a setup function, not a render function: it only runs once.
2. For values in your view template to be reactive, they must be reactive functions: either signals or closures that capture and read from signals.

 Note

This is actually the primary difference between the stable and nightly versions of Leptos. As you may know, using the nightly compiler and the `nightly` feature allows you to call a signal directly, as a function: so, `value()` instead of `value.get()`.

But this isn't just syntax sugar. It allows for an extremely consistent semantic model: Reactive things are functions. Signals are accessed by calling functions. To say "give me a signal as an argument" you can take anything that `impl Fn() -> T`. And this function-based interface makes no distinction between signals, memos, and derived signals: any of them can be accessed by calling them as functions.

Unfortunately implementing the `Fn` traits on arbitrary structs like signals requires nightly Rust, although this particular feature has mostly just languished and is not likely to change (or be stabilized) any time soon. Many people avoid nightly, for one reason or another. So, over time we've moved the defaults for things like documentation toward stable. Unfortunately, this makes the simple mental model of "signals are functions" a bit less straightforward.

Testing Your Components

Testing user interfaces can be relatively tricky, but really important. This article will discuss a couple principles and approaches for testing a Leptos app.

1. Test business logic with ordinary Rust tests

In many cases, it makes sense to pull the logic out of your components and test it separately. For some simple components, there's no particular logic to test, but for many it's worth using a testable wrapping type and implementing the logic in ordinary Rust `impl` blocks.

For example, instead of embedding logic in a component directly like this:

```
#[component]
pub fn TodoApp() -> impl IntoView {
    let (todos, set_todos) = signal(vec![Todo { /* ... */ }]);
    // ⚠️ this is hard to test because it's embedded in the component
    let num_remaining = move || todos.read().iter().filter(|todo|
!todo.completed).sum();
}
```

You could pull that logic out into a separate data structure and test it:

```
pub struct Todos(Vec<Todo>);

impl Todos {
    pub fn num_remaining(&self) -> usize {
        self.0.iter().filter(|todo| !todo.completed).sum()
    }
}

#[cfg(test)]
mod tests {
    #[test]
    fn test_remaining() {
        // ...
    }
}

#[component]
pub fn TodoApp() -> impl IntoView {
    let (todos, set_todos) = signal(Todos(vec![Todo { /* ... */ }]));
    // ✅ this has a test associated with it
    let num_remaining = move || todos.read().num_remaining();
}
```

In general, the less of your logic is wrapped into your components themselves, the more idiomatic your code will feel and the easier it will be to test.

2. Test components with end-to-end (e2e) testing

Our [examples](#) directory has several examples with extensive end-to-end testing, using different testing tools.

The easiest way to see how to use these is to take a look at the test examples themselves:

`wasm-bindgen-test` with `counter`

This is a fairly simple manual testing setup that uses the `wasm-pack test` command.

Sample Test

```
#[wasm_bindgen_test]
async fn clear() {
    let document = document();
    let test_wrapper = document.create_element("section").unwrap();
    let _ = document.body().unwrap().append_child(&test_wrapper);

    // start by rendering our counter and mounting it to the DOM
    // note that we start at the initial value of 10
    let _dispose = mount_to(
        test_wrapper.clone().unchecked_into(),
        || view! { <SimpleCounter initial_value=10 step=1/> },
    );

    // now we extract the buttons by iterating over the DOM
    // this would be easier if they had IDs
    let div = test_wrapper.query_selector("div").unwrap().unwrap();
    let clear = test_wrapper
        .query_selector("button")
        .unwrap()
        .unwrap()
        .unchecked_into::<web_sys::HtmlElement>();

    // now let's click the `clear` button
    clear.click();

    // the reactive system is built on top of the async system, so changes are
    not reflected
    // synchronously in the DOM
    // in order to detect the changes here, we'll just yield for a brief time
    after each change,
    // allowing the effects that update the view to run
    tick().await;

    // now let's test the <div> against the expected value
    // we can do this by testing its `outerHTML`
    assert_eq!(div.outer_html(), {
        // it's as if we're creating it with a value of 0, right?
        let (value, _set_value) = signal(0);

        // we can remove the event listeners because they're not rendered to
        HTML
        view! {
            <div>
                <button>"Clear"</button>
                <button>"-1"</button>
                <span>"Value: " {value} "!"</span>
                <button>" +1"</button>
            </div>
        }
        // Leptos supports multiple backend renderers for HTML elements
        // .into_view() here is just a convenient way of specifying "use the
        regular DOM renderer"
        .into_view()
        // views are lazy -- they describe a DOM tree but don't create it yet
        // calling .build() will actually build the DOM elements
    });
}
```

```

        .build()
        // .build() returned an ElementState, which is a smart pointer for
        // a DOM element. So we can still just call .outer_html(), which access
the outerHTML on
        // the actual DOM element
        .outer_html()
    });

    // There's actually an easier way to do this...
    // We can just test against a <SimpleCounter/> with the initial value 0
    assert_eq!(test_wrapper.inner_html(), {
        let comparison_wrapper = document.create_element("section").unwrap();
        let _dispose = mount_to(
            comparison_wrapper.clone().unchecked_into(),
            || view! { <SimpleCounter initial_value=0 step=1/> },
        );
        comparison_wrapper.inner_html()
    });
}

```

Playwright with counters

These tests use the common JavaScript testing tool Playwright to run end-to-end tests on the same example, using a library and testing approach familiar to many who have done frontend development before.

Sample Test

```

test.describe("Increment Count", () => {
    test("should increase the total count", async ({ page }) => {
        const ui = new CountersPage(page);
        await ui.goto();
        await ui.addCounter();

        await ui.incrementCount();
        await ui.incrementCount();
        await ui.incrementCount();

        await expect(ui.total).toHaveText("3");
    });
});

```

Gherkin/Cucumber Tests with todo_app_sqlite

You can integrate any testing tool you'd like into this flow. This example uses Cucumber, a testing framework based on natural language.

```

@add_todo
Feature: Add Todo

    Background:
        Given I see the app

    @add_todo-see
    Scenario: Should see the todo
        Given I set the todo as Buy Bread
        When I click the Add button
        Then I see the todo named Buy Bread

    # @allow.skipped
    @add_todo-style
    Scenario: Should see the pending todo
        When I add a todo as Buy Oranges
        Then I see the pending todo

```

The definitions for these actions are defined in Rust code.

```

use crate::fixtures::{action, world::AppWorld};
use anyhow::{Ok, Result};
use cucumber::{given, when};

#[given("I see the app")]
#[when("I open the app")]
async fn i_open_the_app(world: &mut AppWorld) -> Result<()> {
    let client = &world.client;
    action::goto_path(client, "").await?;

    Ok(())
}

#[given(regex = "^I add a todo as (.*)$")]
#[when(regex = "^I add a todo as (.*)$")]
async fn i_add_a_todo_titled(world: &mut AppWorld, text: String) -> Result<()>
{
    let client = &world.client;
    action::add_todo(client, text.as_str()).await?;

    Ok(())
}

// etc.

```

Learning More

Feel free to check out the CI setup in the Leptos repo to learn more about how to use these tools in your own application. All of these testing methods are run regularly against actual Leptos example apps.

Working with async

So far we've only been working with synchronous user interfaces: You provide some input, the app immediately processes it and updates the interface. This is great, but is a tiny subset of what web applications do. In particular, most web apps have to deal with some kind of asynchronous data loading, usually loading something from an API.

Asynchronous data is notoriously hard to integrate with the synchronous parts of your code because of problems of “function coloring.”

In the following chapters, we'll see a few reactive primitives for working with async data. But it's important to note at the very beginning: If you just want to do some asynchronous work, Leptos provides a cross-platform `spawn_local` function that makes it easy to run a `Future`. If one of the primitives discussed in the rest of this section doesn't seem to do what you want, consider combining `spawn_local` with setting a signal.

While the primitives to come are very useful, and even necessary in some cases, people sometimes run into situations in which they really just need to spawn a task and wait for it to finish before doing something else. Use `spawn_local` in those situations!

Loading Data with Resources

Resources are reactive wrappers for asynchronous tasks, which allow you to integrate an asynchronous `Future` into the synchronous reactive system.

They effectively allow you to load some async data, and then reactively access it either synchronously or asynchronously. You can `.await` a resource like an ordinary `Future`, and this will track it. But you can also access a resource with `.get()` and other signal access methods, as if a resource were a signal that returns `Some(T)` if it has resolved, and `None` if it's still pending.

Resources come in two primary flavors: `Resource` and `LocalResource`. If you're using server-side rendering (which this book will discuss later), you should default to using `Resource`. If you're using client-side rendering with a `!Send` API (like many of the browser APIs), or if you are using SSR but have some async task that can only be done on the browser (for example, accessing an async browser API) then you should use `LocalResource`.

Local Resources

`LocalResource::new()` takes a single argument: a “fetcher” function that returns a `Future`.

The `Future` can be an `async` block, the result of an `async fn` call, or any other Rust `Future`. The function will work like a derived signal or the other reactive closures that we've seen so far: you can read signals inside it, and whenever the signal changes, the function will run again, creating a new `Future` to run.

```
// this count is our synchronous, local state
let (count, set_count) = signal(0);

// tracks `count`, and reloads by calling `load_data`
// whenever it changes
let async_data = LocalResource::new(move || load_data(count.get()));
```

Creating a resource immediately calls its fetcher and begins polling the `Future`. Reading from a resource will return `None` until the async task completes, at which point it will notify its subscribers, and now have `Some(value)`.

You can also `.await` a resource. This might seem pointless—Why would you create a wrapper around a `Future`, only to then `.await` it? We'll see why in the next chapter.

Resources

If you're using SSR, you should be using `Resource` instead of `LocalResource` in most cases.

This API is slightly different. `Resource::new()` takes two functions as its arguments:

1. a source function, which contains the "input." This input is memoized, and whenever its value changes, the fetcher will be called.
2. a fetcher function, which takes the data from the source function and returns a `Future`

Unlike a `LocalResource`, a `Resource` serializes its value from the server to the client. Then, on the client, when first loading the page, the initial value will be deserialized rather than the async task running again. This is extremely important and very useful: It means that rather than waiting for the client WASM bundle to load and begin running the application, data loading begins on the server. (There will be more to say about this in later chapters.)

This is also why the API is split into two parts: signals in the *source* function are tracked, but signals in the *fetcher* are untracked, because this allows the resource to maintain reactivity without needing to run the fetcher again during initial hydration on the client.

Here's the same example, using `Resource` instead of `LocalResource`

```
// this count is our synchronous, local state
let (count, set_count) = signal(0);

// our resource
let async_data = Resource::new(
    move || count.get(),
    // every time `count` changes, this will run
    |count| load_data(count)
);
```

Resources also provide a `refetch()` method that allows you to manually reload the data (for example, in response to a button click).

To create a resource that simply runs once, you can use `OnceResource`, which simply takes a `Future`, and adds some optimizations that come from knowing it will only load once.

```
let once = OnceResource::new(load_data(42));
```

Accessing Resources

Both `LocalResource` and `Resource` implement the various signal access methods (`.read()`, `.with()`, `.get()`), but return `Option<T>` instead of `T`; they will be `None` until

the async data has loaded.

 [Live example](#)



► [CodeSandbox Source](#)

<Suspense/>

In the previous chapter, we showed how you can create a simple loading screen to show some fallback while a resource is loading.

```
let (count, set_count) = signal(0);
let once = Resource::new(move || count.get(), |count| async move {
  load_a(count).await });

view! {
  <h1>"My Data"</h1>
  {move || match once.get() {
    None => view! { <p>"Loading..."</p> }.into_any(),
    Some(data) => view! { <ShowData data/> }.into_any()
  }}
}
```

But what if we have two resources, and want to wait for both of them?

```
let (count, set_count) = signal(0);
let (count2, set_count2) = signal(0);
let a = Resource::new(move || count.get(), |count| async move {
  load_a(count).await });
let b = Resource::new(move || count2.get(), |count| async move {
  load_b(count).await });

view! {
  <h1>"My Data"</h1>
  {move || match (a.get(), b.get()) {
    (Some(a), Some(b)) => view! {
      <ShowA a/>
      <ShowA b/>
    }.into_any(),
    _ => view! { <p>"Loading..."</p> }.into_any()
  }}
}
```

That's not so bad, but it's kind of annoying. What if we could invert the flow of control?

The `<Suspense/>` component lets us do exactly that. You give it a `fallback` prop and children, one or more of which usually involves reading from a resource. Reading from a resource “under” a `<Suspense/>` (i.e., in one of its children) registers that resource with the `<Suspense/>`. If it's still waiting for resources to load, it shows the `fallback`. When they've all loaded, it shows the children.

```

let (count, set_count) = signal(0);
let (count2, set_count2) = signal(0);
let a = Resource::new(count, |count| async move { load_a(count).await });
let b = Resource::new(count2, |count| async move { load_b(count).await });

view! {
  <h1>"My Data"</h1>
  <Suspense
    fallback=move || view! { <p>"Loading..."</p> }
  >
    <h2>"My Data"</h2>
    <h3>"A"</h3>
    {move || {
      a.get()
        .map(|a| view! { <ShowA a/> })
    }}
    <h3>"B"</h3>
    {move || {
      b.get()
        .map(|b| view! { <ShowB b/> })
    }}
  </Suspense>
}

```

Every time one of the resources is reloading, the "Loading..." fallback will show again.

This inversion of the flow of control makes it easier to add or remove individual resources, as you don't need to handle the matching yourself. It also unlocks some massive performance improvements during server-side rendering, which we'll talk about during a later chapter.

Using `<Suspense/>` also gives us access to a useful way to directly `.await` resources, allowing us to remove a level of nesting, above. The `suspend` type lets us create a renderable `Future` which can be used in the view:

```

view! {
  <h1>"My Data"</h1>
  <Suspense
    fallback=move || view! { <p>"Loading..."</p> }
  >
    <h2>"My Data"</h2>
    {move || Suspend::new(async move {
      let a = a.await;
      let b = b.await;
      view! {
        <h3>"A"</h3>
        <ShowA a/>
        <h3>"B"</h3>
        <ShowB b/>
      }
    })}
  </Suspense>
}

```

`Suspend` allows us to avoid null-checking each resource, and removes some additional complexity from the code.

<Await/>

If you're simply trying to wait for some `Future` to resolve before rendering, you may find the `<Await/>` component helpful in reducing boilerplate. `<Await/>` essentially combines a `OnceResource` with a `<Suspense/>` with no fallback.

In other words:

1. It only polls the `Future` once, and does not respond to any reactive changes.
2. It does not render anything until the `Future` resolves.
3. After the `Future` resolves, it binds its data to whatever variable name you choose and then renders its children with that variable in scope.

```
async fn fetch_monkeys(monkey: i32) -> i32 {
    // maybe this didn't need to be async
    monkey * 2
}
view! {
    <Await
        // `future` provides the `Future` to be resolved
        future=fetch_monkeys(3)
        // the data is bound to whatever variable name you provide
        let:data
    >
        // you receive the data by reference and can use it in your view here
        <p>{*data} " little monkeys, jumping on the bed."</p>
    </Await>
}
```

[□ Live example](#)



► [CodeSandbox Source](#)

<Transition/>

You'll notice in the `<Suspense/>` example that if you keep reloading the data, it keeps flickering back to `"Loading..."`. Sometimes this is fine. For other times, there's `<Transition/>`.

`<Transition/>` behaves exactly the same as `<Suspense/>`, but instead of falling back every time, it only shows the fallback the first time. On all subsequent loads, it continues showing the old data until the new data are ready. This can be really handy to prevent the flickering effect, and to allow users to continue interacting with your application.

This example shows how you can create a simple tabbed contact list with `<Transition/>`. When you select a new tab, it continues showing the current contact until the new data loads. This can be a much better user experience than constantly falling back to a loading message.

[□ Live example](#)



► [CodeSandbox Source](#)

Mutating Data with Actions

We've talked about how to load `async` data with resources. Resources immediately load data and work closely with `<Suspense/>` and `<Transition/>` components to show whether data is loading in your app. But what if you just want to call some arbitrary `async` function and keep track of what it's doing?

Well, you could always use `spawn_local`. This allows you to just spawn an `async` task in a synchronous environment by handing the `Future` off to the browser (or, on the server, Tokio or whatever other runtime you're using). But how do you know if it's still pending? Well, you could just set a signal to show whether it's loading, and another one to show the result...

All of this is true. Or you could use the final `async` primitive: `Action`.

Actions and resources seem similar, but they represent fundamentally different things. If you're trying to load data by running an `async` function, either once or when some other value changes, you probably want to use a resource. If you're trying to occasionally run an `async` function in response to something like a user clicking a button, you probably want to use an `Action`.

Say we have some `async` function we want to run.

```
async fn add_todo_request(new_title: &str) -> Uuid {
    /* do some stuff on the server to add a new todo */
}
```

`Action::new()` takes an `async` function that takes a reference to a single argument, which you could think of as its "input type."

The input is always a single type. If you want to pass in multiple arguments, you can do it with a struct or tuple.

```
// if there's a single argument, just use that
let action1 = Action::new(|input: &String| {
    let input = input.clone();
    async move { todo!() }
});

// if there are no arguments, use the unit type `()`
let action2 = Action::new(|input: &()| async { todo!() });

// if there are multiple arguments, use a tuple
let action3 = Action::new(
    |input: &(usize, String)| async { todo!() }
);
```

Because the action function takes a reference but the `Future` needs to have a 'static' lifetime, you'll usually need to clone the value to pass it into the `Future`. This is admittedly awkward but it unlocks some powerful features like optimistic UI. We'll see a little more about that in future chapters.

So in this case, all we need to do to create an action is

```
let add_todo_action = Action::new(|input: &String| {  
    let input = input.to_owned();  
    async move { add_todo_request(&input).await }  
});
```

Rather than calling `add_todo_action` directly, we'll call it with `.dispatch()`, as in

```
add_todo_action.dispatch("Some value".to_string());
```

You can do this from an event listener, a timeout, or anywhere; because `.dispatch()` isn't an `async` function, it can be called from a synchronous context.

Actions provide access to a few signals that synchronize between the asynchronous action you're calling and the synchronous reactive system:

```
let submitted = add_todo_action.input(); // RwSignal<Option<String>>  
let pending = add_todo_action.pending(); // ReadSignal<bool>  
let todo_id = add_todo_action.value(); // RwSignal<Option<Uuid>>
```

This makes it easy to track the current state of your request, show a loading indicator, or do "optimistic UI" based on the assumption that the submission will succeed.

```
let input_ref = NodeRef::<Input>::new();

view! {
  <form
    on:submit=move |ev| {
      ev.preventDefault(); // don't reload the page...
      let input = input_ref.get().expect("input to exist");
      add_todo_action.dispatch(input.value());
    }
  >
  <label>
    "What do you need to do?"
    <input type="text"
      node_ref=input_ref
    />
  </label>
  <button type="submit">"Add Todo"</button>
</form>
// use our loading state
<p>{move || pending.get().then_some("Loading...")}</p>
}
```

Now, there's a chance this all seems a little over-complicated, or maybe too restricted. I wanted to include actions here, alongside resources, as the missing piece of the puzzle. In a real Leptos app, you'll actually most often use actions alongside server functions, [ServerAction](#), and the `<ActionForm/>` component to create really powerful progressively-enhanced forms. So if this primitive seems useless to you... Don't worry! Maybe it will make sense later. (Or check out our [todo_app_sqlite](#) example now.)

[□ Live example](#)[▶ CodeSandbox Source](#)

Projecting Children

As you build components you may occasionally find yourself wanting to “project” children through multiple layers of components.

The Problem

Consider the following:

```
pub fn NestedShow<F, IV>(fallback: F, children: ChildrenFn) -> impl IntoView
where
    F: Fn() -> IV + Send + Sync + 'static,
    IV: IntoView + 'static,
{
    view! {
        <Show
            when=|| todo!()
            fallback=|| ()
        >
            <Show
                when=|| todo!()
                fallback=fallback
            >
                {children()}
            </Show>
        </Show>
    }
}
```

This is pretty straightforward: if the inner condition is `true`, we want to show `children`. If not, we want to show `fallback`. And if the outer condition is `false`, we just render `()`, i.e., nothing.

In other words, we want to pass the children of `<NestedShow/>` *through* the outer `<Show/>` component to become the children of the inner `<Show/>`. This is what I mean by “projection.”

This won’t compile.

```
error[E0525]: expected a closure that implements the `Fn` trait, but this
closure only implements `FnOnce`
```

Each `<Show/>` needs to be able to construct its `children` multiple times. The first time you construct the outer `<Show/>`’s children, it takes `fallback` and `children` to move them into the invocation of the inner `<Show/>`, but then they’re not available for future outer-`<Show/>` children construction.

The Details

Feel free to skip ahead to the solution.

If you want to really understand the issue here, it may help to look at the expanded `view` macro. Here's a cleaned-up version:

```
Show(
  ShowProps::builder()
    .when(|| todo!())
    .fallback(|| ())
    .children({
      // children and fallback are moved into a closure here
      ::leptos::children::ToChildren::to_children(move || {
        Show(
          ShowProps::builder()
            .when(|| todo!())
            // fallback is consumed here
            .fallback(fallback)
            .children({
              // children is captured here
              ::leptos::children::ToChildren::to_children(
                move || children(),
              )
            })
          .build(),
        )
      })
    })
    .build(),
)
```

All components own their props; so the `<Show/>` in this case can't be called because it only has captured references to `fallback` and `children`.

Solution

However, both `<Suspense/>` and `<Show/>` take `ChildrenFn`, i.e., their children should implement the `Fn` type so they can be called multiple times with only an immutable reference. This means we don't need to own `children` or `fallback`; we just need to be able to pass `'static` references to them.

We can solve this problem by using the [StoredValue](#) primitive. This essentially stores a value in the reactive system, handing ownership off to the framework in exchange for a reference that is, like signals, `copy` and `'static`, which we can access or modify through certain methods.

In this case, it's really simple:

```
pub fn NestedShow<F, IV>(fallback: F, children: ChildrenFn) -> impl IntoView
where
    F: Fn() -> IV + Send + Sync + 'static,
    IV: IntoView + 'static,
{
    let fallback = StoredValue::new(fallback);
    let children = StoredValue::new(children);

    view! {
        <Show
            when=|| todo!()
            fallback=|| ()
        >
            <Show
                // check whether user is verified
                // by reading from the resource
                when=move || todo!()
                fallback=move || fallback.read_value()()
            >
                {children.read_value()()}
            </Show>
        </Show>
    }
}
```

At the top level, we store both `fallback` and `children` in the reactive scope owned by `NestedShow`. Now we can simply move those references down through the other layers into the `<Show/>` component and call them there.

A Final Note

Note that this works because `<Show/>` only needs an immutable reference to their children (which `.read_value` can give), not ownership.

In other cases, you may need to project owned props through a function that takes `ChildrenFn` and therefore needs to be called more than once. In this case, you may find the `clone:` helper in the `view` macro helpful.

Consider this example

```

#[component]
pub fn App() -> impl IntoView {
    let name = "Alice".to_string();
    view! {
        <Outer>
            <Inner>
                <Inmost name=name.clone()/>
            </Inner>
        </Outer>
    }
}

#[component]
pub fn Outer(children: ChildrenFn) -> impl IntoView {
    children()
}

#[component]
pub fn Inner(children: ChildrenFn) -> impl IntoView {
    children()
}

#[component]
pub fn Inmost(name: String) -> impl IntoView {
    view! {
        <p>{name}</p>
    }
}

```

Even with `name=name.clone()` , this gives the error

```
cannot move out of `name`, a captured variable in an `Fn` closure
```

It's captured through multiple levels of children that need to run more than once, and there's no obvious way to clone it *into* the children.

In this case, the `clone: syntax` comes in handy. Calling `clone:name` will clone `name` *before* moving it into `<Inner/>`'s children, which solves our ownership issue.

```

view! {
    <Outer>
        <Inner clone:name>
            <Inmost name=name.clone()/>
        </Inner>
    </Outer>
}

```

These issues can be a little tricky to understand or debug, because of the opacity of the `view` macro. But in general, they can always be solved.

Global State Management

So far, we've only been working with local state in components, and we've seen how to coordinate state between parent and child components. On occasion, there are times when people look for a more general solution for global state management that can work throughout an application.

In general, **you do not need this chapter**. The typical pattern is to compose your application out of components, each of which manages its own local state, not to store all state in a global structure. However, there are some cases (like theming, saving user settings, or sharing data between components in different parts of your UI) in which you may want to use some kind of global state management.

The three best approaches to global state are

1. Using the router to drive global state via the URL
2. Passing signals through context
3. Creating a global state struct using stores

Option #1: URL as Global State

In many ways, the URL is actually the best way to store global state. It can be accessed from any component, anywhere in your tree. There are native HTML elements like `<form>` and `<a>` that exist solely to update the URL. And it persists across page reloads and between devices; you can share a URL with a friend or send it from your phone to your laptop and any state stored in it will be replicated.

The next few sections of the tutorial will be about the router, and we'll get much more into these topics.

But for now, we'll just look at options #2 and #3.

Option #2: Passing Signals through Context

In the section on [parent-child communication](#), we saw that you can use `provide_context` to pass signal from a parent component to a child, and `use_context` to read it in the child. But `provide_context` works across any distance. If you want to create a global signal that holds some piece of state, you can provide it and access it via context anywhere in the descendants of the component where you provide it.

A signal provided via context only causes reactive updates where it is read, not in any of the components in between, so it maintains the power of fine-grained reactive updates, even at a distance.

We start by creating a signal in the root of the app and providing it to all its children and descendants using `provide_context`.

```
#[component]
fn App() -> impl IntoView {
    // here we create a signal in the root that can be consumed
    // anywhere in the app.
    let (count, set_count) = signal(0);
    // we'll pass the setter to specific components,
    // but provide the count itself to the whole app via context
    provide_context(count);

    view! {
        // SetterButton is allowed to modify the count
        <SetterButton set_count/>
        // These consumers can only read from it
        // But we could give them write access by passing `set_count` if we
wanted
        <FancyMath/>
        <ListItems/>
    }
}
```

`<SetterButton/>` is the kind of counter we've written several times now.

`<FancyMath/>` and `<ListItems/>` both consume the signal we're providing via `use_context` and do something with it.

```
/// A component that does some "fancy" math with the global count
#[component]
fn FancyMath() -> impl IntoView {
    // here we consume the global count signal with `use_context`
    let count = use_context::<ReadSignal<u32>>()
        // we know we just provided this in the parent component
        .expect("there to be a `count` signal provided");
    let is_even = move || count.get() & 1 == 0;

    view! {
        <div class="consumer blue">
            "The number "
            <strong>{count}</strong>
            {move || if is_even() {
                " is"
            } else {
                " is not"
            }}
            " even."
        </div>
    }
}
```

Option #3: Create a Global State Store

Some of this content is duplicated from the section on complex iteration with stores [here](#). Both sections are intermediate/optional content, so I thought some duplication couldn't hurt.

Stores are a new reactive primitive, available in Leptos 0.7 through the accompanying `reactive_stores` crate. (This crate is shipped separately for now so we can continue to develop it without requiring a version change to the whole framework.)

Stores allow you to wrap an entire struct, and reactively read from and update individual fields without tracking changes to other fields.

They are used by adding `#[derive(Store)]` onto a struct. (You can use `reactive_stores::Store;` to import the macro.) This creates an extension trait with a getter for each field of the struct, when the struct is wrapped in a `Store<_>`.

```
#[derive(Clone, Debug, Default, Store)]
struct GlobalState {
    count: i32,
    name: String,
}
```

This creates a trait named `GlobalStateStoreFields` which adds with methods `count` and `name` to a `Store<GlobalState>`. Each method returns a reactive store *field*.

```

#[component]
fn App() -> impl IntoView {
    provide_context(Store::new(GlobalState::default()));

    // etc.
}

/// A component that updates the count in the global state.
#[component]
fn GlobalStateCounter() -> impl IntoView {
    let state = expect_context::<Store<GlobalState>>();

    // this gives us reactive access to the `count` field only
    let count = state.count();

    view! {
        <div class="consumer blue">
            <button
                on:click=move |_| {
                    *count.write() += 1;
                }
            >
                "Increment Global Count"
            </button>
            <br/>
            <span>"Count is: " {move || count.get()}</span>
        </div>
    }
}

```

Clicking this button only updates `state.count`. If we read from `state.name` somewhere else, click the button won't notify it. This allows you to combine the benefits of a top-down data flow and of fine-grained reactive updates.

Check out the [stores example](#) in the repo for a more extensive example.

Routing

The Basics

Routing drives most websites. A router is the answer to the question, “Given this URL, what should appear on the page?”

A URL consists of many parts. For example, the URL `https://my-cool-blog.com/blog/search?q=Search#results` consists of

- a *scheme*: `https`
- a *domain*: `my-cool-blog.com`
- a **path**: `/blog/search`
- a **query** (or **search**): `?q=Search`
- a *hash*: `#results`

The Leptos Router works with the path and query (`/blog/search?q=Search`). Given this piece of the URL, what should the app render on the page?

The Philosophy

In most cases, the path should drive what is displayed on the page. From the user’s perspective, for most applications, most major changes in the state of the app should be reflected in the URL. If you copy and paste the URL and open it in another tab, you should find yourself more or less in the same place.

In this sense, the router is really at the heart of the global state management for your application. More than anything else, it drives what is displayed on the page.

The router handles most of this work for you by mapping the current location to particular components.

Defining Routes

Getting Started

It's easy to get started with the router.

First things first, make sure you've added the `leptos_router` package to your dependencies. Unlike `leptos`, this does not have separate `csr` and `hydrate` features; it does have an `ssr` feature, intended for use only on the server side, so activate that for your server-side build.

It's important that the router is a separate package from `leptos` itself. This means that everything in the router can be defined in user-land code. If you want to create your own router, or use no router, you're completely free to do that!

And import the relevant types from the router, either with something like

```
use leptos_router::components::{Router, Route, Routes};
```

Providing the `<Router />`

Routing behavior is provided by the `<Router />` component. This should usually be somewhere near the root of your application, wrapping the rest of the app.

You shouldn't try to use multiple `<Router />` s in your app. Remember that the router drives global state: if you have multiple routers, which one decides what to do when the URL changes?

Let's start with a simple `<App />` component using the router:

```

use leptos::prelude::*;
use leptos_router::components::Router;

#[component]
pub fn App() -> impl IntoView {
    view! {
        <Router>
            <nav>
                /* ... */
            </nav>
            <main>
                /* ... */
            </main>
        </Router>
    }
}

```

Defining <Routes/>

The `<Routes/>` component is where you define all the routes to which a user can navigate in your application. Each possible route is defined by a `<Route/>` component.

You should place the `<Routes/>` component at the location within your app where you want routes to be rendered. Everything outside `<Routes/>` will be present on every page, so you can leave things like a navigation bar or menu outside the `<Routes/>`.

```

use leptos::prelude::*;
use leptos_router::components::*;

#[component]
pub fn App() -> impl IntoView {
    view! {
        <Router>
            <nav>
                /* ... */
            </nav>
            <main>
                // all our routes will appear inside <main>
                <Routes fallback=|| "Not found.">
                    /* ... */
                </Routes>
            </main>
        </Router>
    }
}

```

`<Routes/>` should also have a `fallback`, a function that defines what should be shown if no route is matched.

Individual routes are defined by providing children to `<Routes/>` with the `<Route/>` component. `<Route/>` takes a `path` and a `view`. When the current location matches `path`, the `view` will be created and displayed.

The `path` is most easily defined using the `path` macro, and can include

- a static path (`/users`),
- dynamic, named parameters beginning with a colon (`/:id`),
- and/or a wildcard beginning with an asterisk (`/user/*any`)

The `view` is a function that returns a view. Any component with no props works here, as does a closure that returns some view.

```
<Routes fallback=|| "Not found.">
  <Route path=path!("/") view=Home/>
  <Route path=path!("/users") view=Users/>
  <Route path=path!("/users/:id") view=UserProfile/>
  <Route path=path!("/*any") view=|| view! { <h1>"Not Found"</h1> }/>
</Routes>
```

`view` takes a `Fn() -> impl IntoView`. If a component has no props, it can be passed directly into the `view`. In this case, `view=Home` is just a shorthand for `view=|| view! { <Home/> }`.

Now if you navigate to `/` or to `/users` you'll get the home page or the `<Users/>`. If you go to `/users/3` or `/blahblah` you'll get a user profile or your 404 page (`<NotFound/>`). On every navigation, the router determines which `<Route/>` should be matched, and therefore what content should be displayed where the `<Routes/>` component is defined.

Simple enough?

Nested Routing

We just defined the following set of routes:

```
<Routes fallback=|| "Not found.">
  <Route path=path!("/") view=Home/>
  <Route path=path!("/users") view=Users/>
  <Route path=path!("/users/:id") view=UserProfile/>
  <Route path=path!("/any") view=|| view! { <h1>"Not Found"</h1> }/>
</Routes>
```

There's a certain amount of duplication here: `/users` and `/users/:id`. This is fine for a small app, but you can probably already tell it won't scale well. Wouldn't it be nice if we could nest these routes?

Well... you can!

```
<Routes fallback=|| "Not found.">
  <Route path=path!("/") view=Home/>
  <ParentRoute path=path!("/users") view=Users>
    <Route path=path!("/:id") view=UserProfile/>
  </ParentRoute>
  <Route path=path!("/any") view=|| view! { <h1>"Not Found"</h1> }/>
</Routes>
```

You can nest a `<Route/>` inside a `<ParentRoute/>`. Seems straightforward.

But wait. We've just subtly changed what our application does.

The next section is one of the most important in this entire routing section of the guide. Read it carefully, and feel free to ask questions if there's anything you don't understand.

Nested Routes as Layout

Nested routes are a form of layout, not a method of route definition.

Let me put that another way: The goal of defining nested routes is not primarily to avoid repeating yourself when typing out the paths in your route definitions. It is actually to tell the router to display multiple `<Route/>`s on the page at the same time, side by side.

Let's look back at our practical example.

```
<Routes fallback=|| "Not found.">
  <Route path=path!("/users") view=Users/>
  <Route path=path!("/users/:id") view=UserProfile/>
</Routes>
```

This means:

- If I go to `/users`, I get the `<Users/>` component.
- If I go to `/users/3`, I get the `<UserProfile/>` component (with the parameter `id` set to `3`; more on that later)

Let's say I use nested routes instead:

```
<Routes fallback=|| "Not found.">
  <ParentRoute path=path!("/users") view=Users>
    <Route path=path!(":id") view=UserProfile/>
  </ParentRoute>
</Routes>
```

This means:

- If I go to `/users/3`, the path matches two `<Route/>`s: `<Users/>` and `<UserProfile/>`.
- If I go to `/users`, the path is not matched.

I actually need to add a fallback route

```
<Routes>
  <ParentRoute path=path!("/users") view=Users>
    <Route path=path!(":id") view=UserProfile/>
    <Route path=path!("") view=NoUser/>
  </ParentRoute>
</Routes>
```

Now:

- If I go to `/users/3`, the path matches `<Users/>` and `<UserProfile/>`.
- If I go to `/users`, the path matches `<Users/>` and `<NoUser/>`.

When I use nested routes, in other words, each **path** can match multiple **routes**: each URL can render the views provided by multiple `<Route/>` components, at the same time, on the same page.

This may be counter-intuitive, but it's very powerful, for reasons you'll hopefully see in a few minutes.

Why Nested Routing?

Why bother with this?

Most web applications contain levels of navigation that correspond to different parts of the layout. For example, in an email app you might have a URL like `/contacts/greg`, which

shows a list of contacts on the left of the screen, and contact details for Greg on the right of the screen. The contact list and the contact details should always appear on the screen at the same time. If there's no contact selected, maybe you want to show a little instructional text.

You can easily define this with nested routes

```
<Routes fallback=|| "Not found.">
  <ParentRoute path=path!("/contacts") view=ContactList>
    <Route path=path!(":id") view=ContactInfo/>
    <Route path=path!("") view=|| view! {
      <p>"Select a contact to view more info."</p>
    }/>
  </ParentRoute>
</Routes>
```

You can go even deeper. Say you want to have tabs for each contact's address, email/phone, and your conversations with them. You can add *another* set of nested routes inside `:id`:

```
<Routes fallback=|| "Not found.">
  <ParentRoute path=path!("/contacts") view=ContactList>
    <ParentRoute path=path!(":id") view=ContactInfo>
      <Route path=path!("") view=EmailAndPhone/>
      <Route path=path!("address") view=Address/>
      <Route path=path!("messages") view=Messages/>
    </ParentRoute>
    <Route path=path!("") view=|| view! {
      <p>"Select a contact to view more info."</p>
    }/>
  </ParentRoute>
</Routes>
```

The main page of the [Remix website](https://remix.run), a React framework from the creators of React Router, has a great visual example if you scroll down, with three levels of nested routing: Sales > Invoices > an invoice.

<Outlet/>

Parent routes do not automatically render their nested routes. After all, they are just components; they don't know exactly where they should render their children, and "just stick it at the end of the parent component" is not a great answer.

Instead, you tell a parent component where to render any nested components with an `<Outlet/>` component. The `<Outlet/>` simply renders one of two things:

- if there is no nested route that has been matched, it shows nothing

- if there is a nested route that has been matched, it shows its `view`

That's all! But it's important to know and to remember, because it's a common source of "Why isn't this working?" frustration. If you don't provide an `<outlet/>`, the nested route won't be displayed.

```
#[component]
pub fn ContactList() -> impl IntoView {
    let contacts = todo!();

    view! {
        <div style="display: flex">
            // the contact list
            <For each=contacts
                key=|contact| contact.id
                children=|contact| todo!()
            />
            // the nested child, if any
            // don't forget this!
            <Outlet/>
        </div>
    }
}
```

Refactoring Route Definitions

You don't need to define all your routes in one place if you don't want to. You can refactor any `<Route/>` and its children out into a separate component.

For example, you can refactor the example above to use two separate components:

```
#[component]
pub fn App() -> impl IntoView {
    view! {
        <Router>
            <Routes fallback=|| "Not found.">
                <ParentRoute path=path!("/contacts") view=ContactList>
                    <ContactInfoRoutes/>
                    <Route path=path!("") view=|| view! {
                        <p>"Select a contact to view more info."</p>
                    }/>
                </ParentRoute>
            </Routes>
        </Router>
    }
}

#[component(transparent)]
fn ContactInfoRoutes() -> impl MatchNestedRoutes + Clone {
    view! {
        <ParentRoute path=path!(":id") view=ContactInfo>
            <Route path=path!("") view=EmailAndPhone/>
            <Route path=path!("address") view=Address/>
            <Route path=path!("messages") view=Messages/>
        </ParentRoute>
    }
    .into_inner()
    .into_any_nested_route()
}
```

This second component is a `#[component(transparent)]`, meaning it just returns its data, not a view; likewise, it uses `.into_inner()` to remove some debug info added by the `view` macro and just return the route definitions created by `<ParentRoute/>`.

Nested Routing and Performance

All of this is nice, conceptually, but again—what’s the big deal?

Performance.

In a fine-grained reactive library like Leptos, it’s always important to do the least amount of rendering work you can. Because we’re working with real DOM nodes and not diffing a virtual DOM, we want to “rerender” components as infrequently as possible. Nested routing makes this extremely easy.

Imagine my contact list example. If I navigate from Greg to Alice to Bob and back to Greg, the contact information needs to change on each navigation. But the `<ContactList/>` should never be rerendered. Not only does this save on rendering performance, it also maintains state in the UI. For example, if I have a search bar at the top of `<ContactList/>`, navigating from Greg to Alice to Bob won’t clear the search.

In fact, in this case, we don't even need to rerender the `<Contact/>` component when moving between contacts. The router will just reactively update the `:id` parameter as we navigate, allowing us to make fine-grained updates. As we navigate between contacts, we'll update single text nodes to change the contact's name, address, and so on, without doing *any* additional rerendering.

This sandbox includes a couple features (like nested routing) discussed in this section and the previous one, and a couple we'll cover in the rest of this chapter. The router is such an integrated system that it makes sense to provide a single example, so don't be surprised if there's anything you don't understand.

□ [Live example](#)



► [CodeSandbox Source](#)

Params and Queries

Static paths are useful for distinguishing between different pages, but almost every application wants to pass data through the URL at some point.

There are two ways you can do this:

1. named route **params** like `id` in `/users/:id`
2. named route **queries** like `q` in `/search?q=Foo`

Because of the way URLs are built, you can access the query from *any* `<Route/>` view. You can access route params from the `<Route/>` that defines them or any of its nested children.

Accessing params and queries is pretty simple with a couple of hooks:

- `use_query` or `use_query_map`
- `use_params` or `use_params_map`

Each of these comes with a typed option (`use_query` and `use_params`) and an untyped option (`use_query_map` and `use_params_map`).

The untyped versions hold a simple key-value map. To use the typed versions, derive the `Params` trait on a struct.

`Params` is a very lightweight trait to convert a flat key-value map of strings into a struct by applying `FromStr` to each field. Because of the flat structure of route params and URL queries, it's significantly less flexible than something like `serde`; it also adds much less weight to your binary.

```
use leptos::Params;
use leptos_router::params::Params;

#[derive(Params, PartialEq)]
struct ContactParams {
    id: Option<usize>,
}

#[derive(Params, PartialEq)]
struct ContactSearch {
    q: Option<String>,
}
```

Note: The `Params` derive macro is located at `leptos_router::params::Params`.

Using stable, you can only use `option<T>` in params. If you are using the `nightly` feature, you can use either `T` or `Option<T>`.

Now we can use them in a component. Imagine a URL that has both params and a query, like `/contacts/:id?q=Search`.

The typed versions return `Memo<Result<T, _>>`. It's a `Memo` so it reacts to changes in the URL. It's a `Result` because the params or query need to be parsed from the URL, and may or may not be valid.

```
use leptos_router::hooks::{use_params, use_query};

let params = use_params::<ContactParams>();
let query = use_query::<ContactSearch>();

// id: || -> use
let id = move || {
    params
        .read()
        .as_ref()
        .ok()
        .and_then(|params| params.id)
        .unwrap_or_default()
};
```

The untyped versions return `Memo<ParamsMap>`. Again, it's a `Memo` to react to changes in the URL. `ParamsMap` behaves a lot like any other map type, with a `.get()` method that returns `Option<String>`.

```
use leptos_router::hooks::{use_params_map, use_query_map};

let params = use_params_map();
let query = use_query_map();

// id: || -> Option<String>
let id = move || params.read().get("id");
```

This can get a little messy: deriving a signal that wraps an `Option<_>` or `Result<_>` can involve a couple steps. But it's worth doing this for two reasons:

1. It's correct, i.e., it forces you to consider the cases, "What if the user doesn't pass a value for this query field? What if they pass an invalid value?"
2. It's performant. Specifically, when you navigate between different paths that match the same `<Route/>` with only params or the query changing, you can get fine-grained updates to different parts of your app without rerendering. For example, navigating between different contacts in our contact-list example does a targeted update to the name field (and eventually contact info) without needing to replace or rerender the wrapping `<Contact/>`. This is what fine-grained reactivity is for.

This is the same example from the previous section. The router is such an integrated system that it makes sense to provide a single example highlighting multiple features, even if we haven't explained them all yet.

□ [Live example](#)



► [CodeSandbox Source](#)

The `<A/>` Component

Client-side navigation works perfectly fine with ordinary HTML `<a>` elements. The router adds a listener that handles every click on a `<a>` element and tries to handle it on the client side, i.e., without doing another round trip to the server to request HTML. This is what enables the snappy “single-page app” navigations you’re probably familiar with from most modern web apps.

The router will bail out of handling an `<a>` click under a number of situations

- the click event has had `prevent_default()` called on it
- the `Meta`, `Alt`, `Ctrl`, or `Shift` keys were held during click
- the `<a>` has a `target` or `download` attribute, or `rel="external"`
- the link has a different origin from the current location

In other words, the router will only try to do a client-side navigation when it’s pretty sure it can handle it, and it will upgrade every `<a>` element to get this special behavior.

This also means that if you need to opt out of client-side routing, you can do so easily. For example, if you have a link to another page on the same domain, but which isn’t part of your Leptos app, you can just use `<a rel="external">` to tell the router it isn’t something it can handle.

The router also provides an `<A>` component, which does two additional things:

1. Correctly resolves relative nested routes. Relative routing with ordinary `<a>` tags can be tricky. For example, if you have a route like `/post/:id`, `<A href="1">` will generate the correct relative route, but `<a href="1">` likely will not (depending on where it appears in your view.) `<A/>` resolves routes relative to the path of the nested route within which it appears.
2. Sets the `aria-current` attribute to `page` if this link is the active link (i.e., it’s a link to the page you’re on). This is helpful for accessibility and for styling. For example, if you want to set the link a different color if it’s a link to the page you’re currently on, you can match this attribute with a CSS selector.

Navigating Programmatically

Your most-used methods of navigating between pages should be with `<a>` and `<form>` elements or with the enhanced `<A/>` and `<Form/>` components. Using links and forms to navigate is the best solution for accessibility and graceful degradation.

On occasion, though, you'll want to navigate programmatically, i.e., call a function that can navigate to a new page. In that case, you should use the `use_navigate` function.

```
let navigate = leptos_router::hooks::use_navigate();
navigate("/somewhere", Default::default());
```

You should almost never do something like `<button on:click=move |_| navigate(/* ... */)>`. Any `on:click` that navigates should be an `<a>`, for reasons of accessibility.

The second argument here is a set of `NavigateOptions`, which includes options to resolve the navigation relative to the current route as the `<A/>` component does, replace it in the navigation stack, include some navigation state, and maintain the current scroll state on navigation.

Once again, this is the same example. Check out the relative `<A/>` components, and take a look at the CSS in `index.html` to see the ARIA-based styling.

[□ Live example](#)



► [CodeSandbox Source](#)

The `<Form/>` Component

Links and forms sometimes seem completely unrelated. But, in fact, they work in very similar ways.

In plain HTML, there are three ways to navigate to another page:

1. An `<a>` element that links to another page: Navigates to the URL in its `href` attribute with the `GET` HTTP method.
2. A `<form method="GET">` : Navigates to the URL in its `action` attribute with the `GET` HTTP method and the form data from its inputs encoded in the URL query string.
3. A `<form method="POST">` : Navigates to the URL in its `action` attribute with the `POST` HTTP method and the form data from its inputs encoded in the body of the request.

Since we have a client-side router, we can do client-side link navigations without reloading the page, i.e., without a full round-trip to the server and back. It makes sense that we can do client-side form navigations in the same way.

The router provides a `<Form>` component, which works like the HTML `<form>` element, but uses client-side navigations instead of full page reloads. `<Form/>` works with both `GET` and `POST` requests. With `method="GET"` , it will navigate to the URL encoded in the form data. With `method="POST"` it will make a `POST` request and handle the server's response.

`<Form/>` provides the basis for some components like `<ActionForm/>` and `<MultiActionForm/>` that we'll see in later chapters. But it also enables some powerful patterns of its own.

For example, imagine that you want to create a search field that updates search results in real time as the user searches, without a page reload, but that also stores the search in the URL so a user can copy and paste it to share results with someone else.

It turns out that the patterns we've learned so far make this easy to implement.

```

async fn fetch_results() {
    // some async function to fetch our search results
}

#[component]
pub fn FormExample() -> impl IntoView {
    // reactive access to URL query strings
    let query = use_query_map();
    // search stored as ?q=
    let search = move || query.read().get("q").unwrap_or_default();
    // a resource driven by the search string
    let search_results = Resource::new(search, |_| fetch_results());

    view! {
        <Form method="GET" action="">
            <input type="search" name="q" value=search/>
            <input type="submit"/>
        </Form>
        <Transition fallback=move || ()>
            /* render search results */
            {todo!()}
        </Transition>
    }
}

```

Whenever you click `Submit`, the `<Form/>` will “navigate” to `?q={search}`. But because this navigation is done on the client side, there’s no page flicker or reload. The URL query string changes, which triggers `search` to update. Because `search` is the source signal for the `search_results` resource, this triggers `search_results` to reload its resource. The `<Transition/>` continues displaying the current search results until the new ones have loaded. When they are complete, it switches to displaying the new result.

This is a great pattern. The data flow is extremely clear: all data flows from the URL to the resource into the UI. The current state of the application is stored in the URL, which means you can refresh the page or text the link to a friend and it will show exactly what you’re expecting. And once we introduce server rendering, this pattern will prove to be really fault-tolerant, too: because it uses a `<form>` element and URLs under the hood, it actually works really well without even loading your WASM on the client.

We can actually take it a step further and do something kind of clever:

```

view! {
    <Form method="GET" action="">
        <input type="search" name="q" value=search
            oninput="this.form.requestSubmit()"
        />
    </Form>
}

```

You’ll notice that this version drops the `Submit` button. Instead, we add an `oninput` attribute to the input. Note that this is *not* `on:input`, which would listen for the `input`

event and run some Rust code. Without the colon, `oninput` is the plain HTML attribute. So the string is actually a JavaScript string. `this.form` gives us the form the input is attached to. `requestSubmit()` fires the `submit` event on the `<form>`, which is caught by `<Form/>` just as if we had clicked a `Submit` button. Now the form will “navigate” on every keystroke or input to keep the URL (and therefore the search) perfectly in sync with the user’s input as they type.

[□ Live example](#)[▶ CodeSandbox Source](#)

Interlude: Styling

Anyone creating a website or application soon runs into the question of styling. For a small app, a single CSS file is probably plenty to style your user interface. But as an application grows, many developers find that plain CSS becomes increasingly hard to manage.

Some frontend frameworks (like Angular, Vue, and Svelte) provide built-in ways to scope your CSS to particular components, making it easier to manage styles across a whole application without styles meant to modify one small component having a global effect. Other frameworks (like React or Solid) don't provide built-in CSS scoping, but rely on libraries in the ecosystem to do it for them. Leptos is in this latter camp: the framework itself has no opinions about CSS at all, but provides a few tools and primitives that allow others to build styling libraries.

Here are a few different approaches to styling your Leptos app, starting with plain CSS.

Plain CSS

Client-Side Rendering with Trunk

`trunk` can be used to bundle CSS files and images with your site. To do this, you can add them as Trunk assets by defining them in your `index.html` in the `<head>`. For example, to add a CSS file located at `style.css` you can add the tag `<link data-trunk rel="css" href="/style.css"/>`.

You can find more information in the Trunk documentation for [assets](#).

Server-Side Rendering with `cargo-leptos`

The `cargo-leptos` templates are configured by default to use SASS to bundle CSS files and output them at `/pkg/{project_name}.css`. If you want to load additional CSS files, you can do so either by importing them into that `style.scss` file, or by adding them to the `public` directory. (A file at `public/foo.css`, for example, is served at `/foo.css`.)

To load stylesheets in a component, you can use the [Stylesheet](#) component.

TailwindCSS: Utility-first CSS

[TailwindCSS](#) is a popular utility-first CSS library. It allows you to style your application by using inline utility classes, with a custom CLI tool that scans your files for Tailwind class names and bundles the necessary CSS.

This allows you to write components like this:

```
#[component]
fn Home() -> impl IntoView {
    let (count, set_count) = signal(0);

    view! {
        <main class="my-0 mx-auto max-w-3xl text-center">
            <h2 class="p-6 text-4xl">"Welcome to Leptos with Tailwind"</h2>
            <p class="px-10 pb-10 text-left">"Tailwind will scan your Rust
files for Tailwind class names and compile them into a CSS file."</p>
            <button
                class="bg-sky-600 hover:bg-sky-700 px-5 py-3 text-white
rounded-lg"
                on:click=move |_| *set_count.write() += 1
            >
                {move || if count.get() == 0 {
                    "Click me!".to_string()
                } else {
                    count.get().to_string()
                }}
            </button>
        </main>
    }
}
```

It can be a little complicated to set up the Tailwind integration at first, but you can check out our two examples of how to use Tailwind with a [client-side-rendered trunk application](#) or with a [server-rendered cargo-leptos application](#). `cargo-leptos` also has some [built-in Tailwind support](#) that you can use as an alternative to Tailwind's CLI.

Stylers: Compile-time CSS Extraction

[Stylers](#) is a compile-time scoped CSS library that lets you declare scoped CSS in the body of your component. Stylers will extract this CSS at compile time into CSS files that you can then import into your app, which means that it doesn't add anything to the WASM binary size of your application.

This allows you to write components like this:

```

use stylers::style;

#[component]
pub fn App() -> impl IntoView {
    let styler_class = style! { "App",
        #two{
            color: blue;
        }
        div.one{
            color: red;
            content: raw_str(r#"hello"#);
            font: "1.3em/1.2" Arial, Helvetica, sans-serif;
        }
        div {
            border: 1px solid black;
            margin: 25px 50px 75px 100px;
            background-color: lightblue;
        }
        h2 {
            color: purple;
        }
        @media only screen and (max-width: 1000px) {
            h3 {
                background-color: lightblue;
                color: blue
            }
        }
    };

    view! { class = styler_class,
        <div class="one">
            <h1 id="two">"Hello"</h1>
            <h2>"World"</h2>
            <h2>"and"</h2>
            <h3>"friends!"</h3>
        </div>
    }
}

```

Stylance: Scoped CSS Written in CSS Files

Stylers lets you write CSS inline in your Rust code, extracts it at compile time, and scopes it. [Stylance](#) allows you to write your CSS in CSS files alongside your components, import those files into your components, and scope the CSS classes to your components.

This works well with the live-reloading features of `trunk` and `cargo-leptos` because edited CSS files can be updated immediately in the browser.

```
import_style!(style, "app.module.scss");

#[component]
fn HomePage() -> impl IntoView {
    view! {
        <div class=style::jumbotron/>
    }
}
```

You can edit the CSS directly without causing a Rust recompile.

```
.jumbotron {
    background: blue;
}
```

Styled: Runtime CSS Scoping

[Styled](#) is a runtime scoped CSS library that integrates well with Leptos. It lets you declare scoped CSS in the body of your component function, and then applies those styles at runtime.

```
use styled::style;

#[component]

pub fn MyComponent() -> impl IntoView {

    let styles = style!(

        div {

            background-color: red;

            color: white;

        }

    );

    styled::view! { styles,

        <div>"This text should be red with white text."</div>

    }

}
```

Contributions Welcome

Leptos has no opinions on how you style your website or app, but we're very happy to provide support to any tools you're trying to create to make it easier. If you're working on a CSS or styling approach that you'd like to add to this list, please let us know!

Metadata

So far, everything we've rendered has been inside the `<body>` of the HTML document. And this makes sense. After all, everything you can see on a web page lives inside the `<body>`.

However, there are plenty of occasions where you might want to update something inside the `<head>` of the document using the same reactive primitives and component patterns you use for your UI.

That's where the `leptos_meta` package comes in.

Metadata Components

`leptos_meta` provides special components that let you inject data from inside components anywhere in your application into the `<head>`:

`<Title/>` allows you to set the document's title from any component. It also takes a `formatter` function that can be used to apply the same format to the title set by other pages. So, for example, if you put `<Title formatter=|text| format!("{text} – My Awesome Site")/>` in your `<App/>` component, and then `<Title text="Page 1"/>` and `<Title text="Page 2"/>` on your routes, you'll get `Page 1 – My Awesome Site` and `Page 2 – My Awesome Site`.

`<Link/>` injects a `<link>` element into the `<head>`.

`<Stylesheet/>` creates a `<link rel="stylesheet">` with the `href` you give.

`<Style/>` creates a `<style>` with the children you pass in (usually a string). You can use this to import some custom CSS from another file at compile time `<Style>{include_str!("my_route.css")}</Style>`.

`<Meta/>` lets you set `<meta>` tags with descriptions and other metadata.

Warning

Note: These components should be used in the body of your application, somewhere in a component. They should not be used in the `<head>` (for example, if you are using server-side rendering.) Rather than putting `leptos_meta` components into the `<head>`, you can and should simply use the corresponding HTML elements.

<Script/> and <script>

`leptos_meta` also provides a `<Script/>` component, and it's worth pausing here for a second. All of the other components we've considered inject `<head>`-only elements in the `<head>`. But a `<script>` can also be included in the body.

There's a very simple way to determine whether you should use a capital-S `<Script/>` component or a lowercase-s `<script>` element: the `<Script/>` component will be rendered in the `<head>`, and the `<script>` element will be rendered wherever in the `<body>` of your user interface you put it in, alongside other normal HTML elements. These cause JavaScript to load and run at different times, so use whichever is appropriate to your needs.

<Body/> and <Html/>

There are even a couple elements designed to make semantic HTML and styling easier. `<Body/>` and `<Html/>` are designed to allow you to add arbitrary attributes to the `<html>` and `<body>` tags on your page. You can add any number of attributes using the usual Leptos syntax after the spread operator (`{..}`) and those will be added directly to the appropriate element.

```
<Html
  {..}
  lang="he"
  dir="rtl"
  data-theme="dark"
/>
```

Metadata and Server Rendering

Now, some of this is useful in any scenario, but some of it is especially important for search-engine optimization (SEO). Making sure you have things like appropriate `<title>` and `<meta>` tags is crucial. Modern search engine crawlers do handle client-side rendering, i.e., apps that are shipped as an empty `index.html` and rendered entirely in JS/WASM. But they prefer to receive pages in which your app has been rendered to actual HTML, with metadata in the `<head>`.

This is exactly what `leptos_meta` is for. And in fact, during server rendering, this is exactly what it does: collect all the `<head>` content you've declared by using its components throughout your application, and then inject it into the actual `<head>`.

But I'm getting ahead of myself. We haven't actually talked about server-side rendering yet. The next chapter will talk about integrating with JavaScript libraries. Then we'll wrap up the discussion of the client side, and move onto server side rendering.

Integrating with JavaScript: `wasm-bindgen`, `web_sys` and `HtmlElement`

Leptos provides a variety of tools to allow you to build declarative web applications without leaving the world of the framework. Things like the reactive system, `component` and `view` macros, and `router` allow you to build user interfaces without directly interacting with the Web APIs provided by the browser. And they let you do it all directly in Rust, which is great—assuming you like Rust. (And if you’ve gotten this far in the book, we assume you like Rust.)

Ecosystem crates like the fantastic set of utilities provided by `leptos-use` can take you even further, by providing Leptos-specific reactive wrappers around many Web APIs.

Nevertheless, in many cases you will need to access JavaScript libraries or Web APIs directly. This chapter can help.

Using JS Libraries with `wasm-bindgen`

Your Rust code can be compiled to a WebAssembly (WASM) module and loaded to run in the browser. However, WASM does not have direct access to browser APIs. Instead, the Rust/WASM ecosystem depends on generating bindings from your Rust code to the JavaScript browser environment that hosts it.

The `wasm-bindgen` crate is at the center of that ecosystem. It provides both an interface for marking parts of Rust code with annotations telling it how to call JS, and a CLI tool for generating the necessary JS glue code. You’ve been using this without knowing it all along: both `trunk` and `cargo-leptos` rely on `wasm-bindgen` under the hood.

If there is a JavaScript library that you want to call from Rust, you should refer to the `wasm-bindgen` docs on [importing functions from JS](#). It is relatively easy to import individual functions, classes, or values from JavaScript to use in your Rust app.

It is not always easy to integrate JS libraries into your app directly. In particular, any library that depends on a particular JS framework like React may be hard to integrate. Libraries that manipulate DOM state in some way (for example, rich text editors) should also be used with care: both Leptos and the JS library will probably assume that they are the ultimate source of truth for the app’s state, so you should be careful to separate their responsibilities.

Accessing Web APIs with web-sys

If you just need to access some browser APIs without pulling in a separate JS library, you can do so using the `web_sys` crate. This provides bindings for all of the Web APIs provided by the browser, with 1:1 mappings from browser types and functions to Rust structs and methods.

In general, if you're asking "how do I *do X* with Leptos?" where *do X* is accessing some Web API, looking up a vanilla JavaScript solution and translating it to Rust using the [web-sys docs](#) is a good approach.

After this section, you might find [the wasm-bindgen guide chapter on web-sys](#) useful for additional reading.

Enabling features

`web_sys` is heavily feature-gated to keep compile times low. If you would like to use one of its many APIs, you may need to enable a feature to use it.

The features required to use an item are always listed in its documentation. For example, to use `Element::get_bounding_rect_client`, you need to enable the `DomRect` and `Element` features.

Leptos already enables [a whole bunch](#) of features - if the required feature is already enabled here, you won't have to enable it in your own app. Otherwise, add it to your `Cargo.toml` and you're good to go!

```
[dependencies.web-sys]
version = "0.3"
features = ["DomRect"]
```

However, as the JavaScript standard evolves and APIs are being written, you may want to use browser features that are technically not fully stable yet, such as [WebGPU](#). `web_sys` will follow the (potentially frequently changing) standard, which means that no stability guarantees are made.

In order to use this, you need to add `RUSTFLAGS=--cfg=web_sys_unstable_apis` as an environment variable. This can either be done by adding it to every command, or add it to `.cargo/config.toml` in your repository.

As part of a command:

```
RUSTFLAGS=--cfg=web_sys_unstable_apis cargo # ...
```

In `.cargo/config.toml`:

```
[env]
RUSTFLAGS = "--cfg=web_sys_unstable_apis"
```

Accessing raw `HtmlElements` from your view

The declarative style of the framework means that you don't need to directly manipulate DOM nodes to build up your user interface. However, in some cases you want direct access to the underlying DOM element that represents part of your view. The section of the book on [“uncontrolled inputs”](#) showed how to do this using the `NodeRef` type.

`NodeRef::get` returns a correctly-typed `web-sys` element that can be directly manipulated.

For example, consider the following:

```
#[component]
pub fn App() -> impl IntoView {
    let node_ref = NodeRef::<Input>::new();

    Effect::new(move |_| {
        if let Some(node) = node_ref.get() {
            leptos::logging::log!("value = {}", node.value());
        }
    });

    view! {
        <input node_ref=node_ref/>
    }
}
```

Inside the effect here, `node` is simply a `web_sys::HtmlInputElement`. This allows us to call any appropriate methods.

(Note that `.get()` returns an `Option` here, because the `NodeRef` is empty until it is filled when the DOM elements are actually created. Effects run a tick after the component runs, so in most cases the `<input>` will already have been created by the time the effect runs.)

Wrapping Up Part 1: Client-Side Rendering

So far, everything we've written has been rendered almost entirely in the browser. When we create an app using Trunk, it's served using a local development server. If you build it for production and deploy it, it's served by whatever server or CDN you're using. In either case, what's served is an HTML page with

1. the URL of your Leptos app, which has been compiled to WebAssembly (WASM)
2. the URL of the JavaScript used to initialize this WASM blob
3. an empty `<body>` element

When the JS and WASM have loaded, Leptos will render your app into the `<body>`. This means that nothing appears on the screen until JS/WASM have loaded and run. This has some drawbacks:

1. It increases load time, as your user's screen is blank until additional resources have been downloaded.
2. It's bad for SEO, as load times are longer and the HTML you serve has no meaningful content.
3. It's broken for users for whom JS/WASM don't load for some reason (e.g., they're on a train and just went into a tunnel before WASM finished loading; they're using an older device that doesn't support WASM; they have JavaScript or WASM turned off for some reason; etc.)

These downsides apply across the web ecosystem, but especially to WASM apps.

However, depending on the requirements of your project, you may be fine with these limitations.

If you just want to deploy your Client-Side Rendered website, skip ahead to the chapter on "[Deployment](#)" - there, you'll find directions on how best to deploy your Leptos CSR site.

But what do you do if you want to return more than just an empty `<body>` tag in your `index.html` page? Use "Server-Side Rendering"!

Whole books could be (and probably have been) written about this topic, but at its core, it's really simple: rather than returning an empty `<body>` tag, with SSR, you'll return an initial HTML page that reflects the actual starting state of your app or site, so that while JS/WASM are loading, and until they load, the user can access the plain HTML version.

Part 2 of this book, on Leptos SSR, will cover this topic in some detail!

Part 2: Server Side Rendering

As you read in the last chapter, there are some limitations to using client-side rendered web applications. This second part of the book will discuss how to use server-side rendering to overcome these limitations and get the best performance and SEO out of your Leptos apps.

Info

When working with Leptos on the server side, you're free to choose either an officially supported Actix or Axum integrations, or one of our community supported choices. The full feature set of Leptos is available with the official choices, the community ones may support less. Check their documentation for details.

We have a variety of community supported choices, including WinterCG-compatible runtimes like Deno or Cloudflare and server-side WASM runtimes like Spin. Community-supported integrations for Viz and Pavex offer more traditional server choices. Writing an integration yourself isn't recommended as a beginner, but medium/advanced Rust users may wish to. Feel free to reach out if you have questions about that on our Discord or Github.

I'd recommend either Axum or Actix for beginners. Both are fully functional and choosing between them is a matter of personal preference. There is no wrong choice there, but if you're looking for a recommendation, the Leptos team currently defaults to Axum for new projects.

Introducing cargo-leptos

So far, we've just been running code in the browser and using Trunk to coordinate the build process and run a local development process. If we're going to add server-side rendering, we'll need to run our application code on the server as well. This means we'll need to build two separate binaries, one compiled to native code and running the server, the other compiled to WebAssembly (WASM) and running in the user's browser. Additionally, the server needs to know how to serve this WASM version (and the JavaScript required to initialize it) to the browser.

This is not an insurmountable task but it adds some complication. For convenience and an easier developer experience, we built the [cargo-leptos](#) build tool. `cargo-leptos` basically exists to coordinate the build process for your app, handling recompiling the server and client halves when you make changes, and adding some built-in support for things like Tailwind, SASS, and testing.

Getting started is pretty easy. Just run

```
cargo install --locked cargo-leptos
```

And then to create a new project, you can run either

```
# for an Actix template
cargo leptos new --git https://github.com/leptos-rs/start-actix
```

or

```
# for an Axum template
cargo leptos new --git https://github.com/leptos-rs/start-axum
```

Make sure you've added the `wasm32-unknown-unknown` target so that Rust can compile your code to WebAssembly to run in the browser.

```
rustup target add wasm32-unknown-unknown
```

Now `cd` into the directory you've created and run

```
cargo leptos watch
```

Once your app has compiled you can open up your browser to <http://localhost:3000> to see it.

`cargo-leptos` has lots of additional features and built in tools. You can learn more [in its README](#).

But what exactly is happening when you open your browser to `localhost:3000`? Well, read on to find out.

The Life of a Page Load

Before we get into the weeds it might be helpful to have a higher-level overview. What exactly happens between the moment you type in the URL of a server-rendered Leptos app, and the moment you click a button and a counter increases?

I'm assuming some basic knowledge of how the Internet works here, and won't get into the weeds about HTTP or whatever. Instead, I'll try to show how different parts of the Leptos APIs map onto each part of the process.

This description also starts from the premise that your app is being compiled for two separate targets:

1. A server version, often running on Actix or Axum, compiled with the Leptos `ssr` feature
2. A browser version, compiled to WebAssembly (WASM) with the Leptos `hydrate` feature

The `cargo-leptos` build tool exists to coordinate the process of compiling your app for these two different targets.

On the Server

- Your browser makes a `GET` request for that URL to your server. At this point, the browser knows almost nothing about the page that's going to be rendered. (The question "How does the browser know where to ask for the page?" is an interesting one, but out of the scope of this tutorial!)
- The server receives that request, and checks whether it has a way to handle a `GET` request at that path. This is what the `.leptos_routes()` methods in `leptos_axum` and `leptos_actix` are for. When the server starts up, these methods walk over the routing structure you provide in `<Routes/>`, generating a list of all possible routes your app can handle and telling the server's router "for each of these routes, if you get a request... hand it off to Leptos."
- The server sees that this route can be handled by Leptos. So it renders your root component (often called something like `<App/>`), providing it with the URL that's being requested and some other data like the HTTP headers and request metadata.
- Your application runs once on the server, building up an HTML version of the component tree that will be rendered at that route. (There's more to be said here about resources and `<Suspense/>` in the next chapter.)
- The server returns this HTML page, also injecting information on how to load the version of your app that has been compiled to WASM so that it can run in the browser.

The HTML page that's returned is essentially your app, "dehydrated" or "freeze-dried": it is HTML without any of the reactivity or event listeners you've added. The browser will "rehydrate" this HTML page by adding the reactive system and attaching event listeners to that server-rendered HTML. Hence the two feature flags that apply to the two halves of this process: `ssr` on the server for "server-side rendering", and `hydrate` in the browser for that process of rehydration.

In the Browser

- The browser receives this HTML page from the server. It immediately goes back to the server to begin loading the JS and WASM necessary to run the interactive, client side version of the app.
- In the meantime, it renders the HTML version.
- When the WASM version has reloaded, it does the same route-matching process that the server did. Because the `<Routes/>` component is identical on the server and in the client, the browser version will read the URL and render the same page that was already returned by the server.
- During this initial "hydration" phase, the WASM version of your app doesn't re-create the DOM nodes that make up your application. Instead, it walks over the existing HTML tree, "picking up" existing elements and adding the necessary interactivity.

Note that there are some trade-offs here. Before this hydration process is complete, the page will *appear* interactive but won't actually respond to interactions. For example, if you have a counter button and click it before WASM has loaded, the count will not increment, because the necessary event listeners and reactivity have not been added yet. We'll look at some ways to build in "graceful degradation" in future chapters.

Client-Side Navigation

The next step is very important. Imagine that the user now clicks a link to navigate to another page in your application.

The browser will *not* make another round trip to the server, reloading the full page as it would for navigating between plain HTML pages or an application that uses server rendering (for example with PHP) but without a client-side half.

Instead, the WASM version of your app will load the new page, right there in the browser, without requesting another page from the server. Essentially, your app upgrades itself from

a server-loaded “multi-page app” into a browser-rendered “single-page app.” This yields the best of both worlds: a fast initial load time due to the server-rendered HTML, and fast secondary navigations because of the client-side routing.

Some of what will be described in the following chapters—like the interactions between server functions, resources, and `<Suspense/>`—may seem overly complicated. You might find yourself asking, “If my page is being rendered to HTML on the server, why can’t I just `.await` this on the server? If I can just call library X in a server function, why can’t I call it in my component?” The reason is pretty simple: to enable the upgrade from server rendering to client rendering, everything in your application must be able to run on both the client and the server.

This is not the only way to create a website or web framework, of course. But it’s the most common way, and we happen to think it’s quite a good way, to create the smoothest possible experience for your users.

Async Rendering and SSR “Modes”

Server-rendering a page that uses only synchronous data is pretty simple: You just walk down the component tree, rendering each element to an HTML string. But this is a pretty big caveat: it doesn’t answer the question of what we should do with pages that includes asynchronous data, i.e., the sort of stuff that would be rendered under a `<Suspense/>` node on the client.

When a page loads async data that it needs to render, what should we do? Should we wait for all the async data to load, and then render everything at once? (Let’s call this “async” rendering) Should we go all the way in the opposite direction, just sending the HTML we have immediately down to the client and letting the client load the resources and fill them in? (Let’s call this “synchronous” rendering) Or is there some middle-ground solution that somehow beats them both? (Hint: There is.)

If you’ve ever listened to streaming music or watched a video online, I’m sure you realize that HTTP supports streaming, allowing a single connection to send chunks of data one after another without waiting for the full content to load. You may not realize that browsers are also really good at rendering partial HTML pages. Taken together, this means that you can actually enhance your users’ experience by **streaming HTML**: and this is something that Leptos supports out of the box, with no configuration at all. And there’s actually more than one way to stream HTML: you can stream the chunks of HTML that make up your page in order, like frames of a video, or you can stream them... well, out of order.

Let me say a little more about what I mean.

Leptos supports all the major ways of rendering HTML that includes asynchronous data:

1. [Synchronous Rendering](#)
2. [Async Rendering](#)
3. [In-Order streaming](#)
4. [Out-of-Order Streaming](#) (and a partially-blocked variant)

Synchronous Rendering

1. **Synchronous:** Serve an HTML shell that includes `fallback` for any `<Suspense/>`. Load data on the client using `create_local_resource`, replacing `fallback` once resources are loaded.
 - *Pros:* App shell appears very quickly: great TTFB (time to first byte).
 - *Cons*
 - Resources load relatively slowly; you need to wait for JS + WASM to load before even making a request.

- No ability to include data from async resources in the `<title>` or other `<meta>` tags, hurting SEO and things like social media link previews.

If you're using server-side rendering, the synchronous mode is almost never what you actually want, from a performance perspective. This is because it misses out on an important optimization. If you're loading async resources during server rendering, you can actually begin loading the data on the server. Rather than waiting for the client to receive the HTML response, then loading its JS + WASM, *then* realize it needs the resources and begin loading them, server rendering can actually begin loading the resources when the client first makes the request. In this sense, during server rendering an async resource is like a `Future` that begins loading on the server and resolves on the client. As long as the resources are actually serializable, this will always lead to a faster total load time.

This is why a `Resource` needs its data to be serializable, and why you should use `LocalResource` for any async data that is not serializable and should therefore only be loaded in the browser itself. Creating a local resource when you could create a serializable resource is always a deoptimization.

Async Rendering

0:00 / 0:02



2. **async** : Load all resources on the server. Wait until all data are loaded, and render HTML in one sweep.
 - *Pros*: Better handling for meta tags (because you know async data even before you render the `<head>`). Faster complete load than **synchronous** because async resources begin loading on server.
 - *Cons*: Slower load time/TTFB: you need to wait for all async resources to load before displaying anything on the client. The page is totally blank until everything is loaded.

In-Order Streaming

0:00 / 0:02



3. **In-order streaming**: Walk through the component tree, rendering HTML until you hit a `<Suspense/>` . Send down all the HTML you've got so far as a chunk in the stream, wait for all the resources accessed under the `<Suspense/>` to load, then render it to HTML and keep walking until you hit another `<Suspense/>` or the end of the page.
 - *Pros*: Rather than a blank screen, shows at least *something* before the data are ready.
 - *Cons*
 - Loads the shell more slowly than synchronous rendering (or out-of-order streaming) because it needs to pause at every `<Suspense/>` .
 - Unable to show fallback states for `<Suspense/>` .

- Can't begin hydration until the entire page has loaded, so earlier pieces of the page will not be interactive until the suspended chunks have loaded.

Out-of-Order Streaming

0:00 / 0:02



4. **Out-of-order streaming:** Like synchronous rendering, serve an HTML shell that includes `fallback` for any `<Suspense/>` . But load data on the **server**, streaming it down to the client as it resolves, and streaming down HTML for `<Suspense/>` nodes, which is swapped in to replace the fallback.
- *Pros:* Combines the best of **synchronous** and **async** .
 - Fast initial response/TTFB because it immediately sends the whole synchronous shell
 - Fast total time because resources begin loading on the server.
 - Able to show the fallback loading state and dynamically replace it, instead of showing blank sections for un-loaded data.
 - *Cons:* Requires JavaScript to be enabled for suspended fragments to appear in correct order. (This small chunk of JS streamed down in a `<script>` tag alongside the `<template>` tag that contains the rendered `<Suspense/>` fragment, so it does not need to load any additional JS files.)

5. **Partially-blocked streaming:** “Partially-blocked” streaming is useful when you have multiple separate `<Suspense/>` components on the page. It is triggered by setting `ssr=SsrMode::PartiallyBlocked` on a route, and depending on blocking resources within the view. If one of the `<Suspense/>` components reads from one or more “blocking resources” (see below), the fallback will not be sent; rather, the server will wait until that `<Suspense/>` has resolved and then replace the fallback with the resolved fragment on the server, which means that it is included in the initial HTML response and appears even if JavaScript is disabled or not supported. Other `<Suspense/>` stream in out of order, similar to the `SsrMode::OutOfOrder` default.

This is useful when you have multiple `<Suspense/>` on the page, and one is more important than the other: think of a blog post and comments, or product information and reviews. It is *not* useful if there’s only one `<Suspense/>`, or if every `<Suspense/>` reads from blocking resources. In those cases it is a slower form of `async` rendering.

- *Pros:* Works if JavaScript is disabled or not supported on the user’s device.
- *Cons*
 - Slower initial response time than out-of-order.
 - Marginally slower overall response due to additional work on the server.
 - No fallback state shown.

Using SSR Modes

Because it offers the best blend of performance characteristics, Leptos defaults to out-of-order streaming. But it’s really simple to opt into these different modes. You do it by adding an `ssr` property onto one or more of your `<Route/>` components, like in the [ssr_modes example](#).

```
<Routes fallback=|| "Not found.">
  // We'll load the home page with out-of-order streaming and <Suspense/>
  <Route path=path!("") view=HomePage/>

  // We'll load the posts with async rendering, so they can set
  // the title and metadata *after* loading the data
  <Route
    path=path!("/post/:id")
    view=BlogPost
    ssr=SsrMode::Async
  />
</Routes>
```

For a path that includes multiple nested routes, the most restrictive mode will be used: i.e., if even a single nested route asks for `async` rendering, the whole initial request will be rendered `async`. `async` is the most restricted requirement, followed by in-order, and then out-of-order. (This probably makes sense if you think about it for a few minutes.)

Blocking Resources

Blocking resources can be created with `Resource::new_blocking`. A blocking resource still loads asynchronously like any other `async / .await` in Rust. It doesn't block a server thread, or anything like that. Instead, reading from a blocking resource under a `<Suspense/>` blocks the HTML *stream* from returning anything, including its initial synchronous shell, until that `<Suspense/>` has resolved.

From a performance perspective, this is not ideal. None of the synchronous shell for your page will load until that resource is ready. However, rendering nothing means that you can do things like set the `<title>` or `<meta>` tags in your `<head>` in actual HTML. This sounds a lot like `async` rendering, but there's one big difference: if you have multiple `<Suspense/>` sections, you can block on *one* of them but still render a placeholder and then stream in the other.

For example, think about a blog post. For SEO and for social sharing, I definitely want my blog post's title and metadata in the initial HTML `<head>`. But I really don't care whether comments have loaded yet or not; I'd like to load those as lazily as possible.

With blocking resources, I can do something like this:

```
#[component]
pub fn BlogPost() -> impl IntoView {
    let post_data = Resource::new_blocking(/* load blog post */);
    let comments_data = Resource::new(/* load blog comments */);
    view! {
        <Suspense fallback=|| ()>
            {move || Suspend::new(async move {
                let data = post_data.await;
                view! {
                    <Title text=data.title/>
                    <Meta name="description" content=data.excerpt/>
                    <article>
                        /* render the post content */
                    </article>
                }
            })}
        </Suspense>
        <Suspense fallback=|| "Loading comments...">
            {move || Suspend::new(async move {
                let comments = comments_data.await;
                todo!()
            })}
        </Suspense>
    }
}
```

The first `<Suspense/>`, with the body of the blog post, will block my HTML stream, because it reads from a blocking resource. Meta tags and other head elements awaiting the blocking resource will be rendered before the stream is sent.

Combined with the following route definition, which uses `SsrMode::PartiallyBlocked`, the blocking resource will be fully rendered on the server side, making it accessible to users who disable WebAssembly or JavaScript.

```
<Routes fallback=|| "Not found.">
  // We'll load the home page with out-of-order streaming and <Suspense/>
  <Route path=path!("") view=HomePage/>

  // We'll load the posts with async rendering, so they can set
  // the title and metadata *after* loading the data
  <Route
    path=path!("/post/:id")
    view=BlogPost
    ssr=SsrMode::PartiallyBlocked
  />
</Routes>
```

The second `<Suspense/>`, with the comments, will not block the stream. Blocking resources gave me exactly the power and granularity I needed to optimize my page for SEO and user experience.

Hydration Bugs (*and how to avoid them*)

A Thought Experiment

Let's try an experiment to test your intuitions. Open up an app you're server-rendering with `cargo-leptos`. (If you've just been using `trunk` so far to play with examples, go [clone a cargo-leptos template](#) just for the sake of this exercise.)

Put a log somewhere in your root component. (I usually call mine `<App/>`, but anything will do.)

```
#[component]
pub fn App() -> impl IntoView {
    logging::log!("where do I run?");
    // ... whatever
}
```

And let's fire it up

```
cargo leptos watch
```

Where do you expect `where do I run?` to log?

- In the command line where you're running the server?
- In the browser console when you load the page?
- Neither?
- Both?

Try it out.

...

...

...

Okay, consider the spoiler alerted.

You'll notice of course that it logs in both places, assuming everything goes according to plan. In fact on the server it logs twice—first during the initial server startup, when Leptos renders your app once to extract the route tree, then a second time when you make a request. Each time you reload the page, `where do I run?` should log once on the server and once on the client.

If you think about the description in the last couple sections, hopefully this makes sense. Your application runs once on the server, where it builds up a tree of HTML which is sent to the client. During this initial render, `where do I run?` logs on the server.

Once the WASM binary has loaded in the browser, your application runs a second time, walking over the same user interface tree and adding interactivity.

Does that sound like a waste? It is, in a sense. But reducing that waste is a genuinely hard problem. It's what some JS frameworks like Qwik are intended to solve, although it's probably too early to tell whether it's a net performance gain as opposed to other approaches.

The Potential for Bugs

Okay, hopefully all of that made sense. But what does it have to do with the title of this chapter, which is "Hydration bugs (and how to avoid them)"?

Remember that the application needs to run on both the server and the client. This generates a few different sets of potential issues you need to know how to avoid.

Mismatches between server and client code

One way to create a bug is by creating a mismatch between the HTML that's sent down by the server and what's rendered on the client. It's actually fairly hard to do this unintentionally, I think (at least judging by the bug reports I get from people.) But imagine I do something like this

```
#[component]
pub fn App() -> impl IntoView {
    let data = if cfg!(target_arch = "wasm32") {
        vec![0, 1, 2]
    } else {
        vec![]
    };
    data.into_iter()
        .map(|value| view! { <span>{value}</span> })
        .collect_view()
}
```

In other words, if this is being compiled to WASM, it has three items; otherwise it's empty.

When I load the page in the browser, I see nothing. If I open the console I see a panic:

```
ssr_modes.js:423 panicked at ../../tachys/src/html/element/mod.rs:352:14:  
called `Option::unwrap()` on a `None` value
```

The WASM version of your app, running in the browser, is expecting to find an element (in fact, it's expecting three elements!) But the HTML sent from the server has none.

Solution

It's pretty rare that you do this intentionally, but it could happen from somehow running different logic on the server and in the browser. If you're seeing warnings like this and you don't think it's your fault, it's much more likely that it's a bug with `<Suspense/>` or something. Feel free to go ahead and open an [issue](#) or [discussion](#) on GitHub for help.

Invalid/edge-case HTML, and mismatches between HTML and the DOM

Servers respond to requests with HTML. The browser then parses that HTML into a tree called the Document Object Model (DOM). During hydration, Leptos walks over the view tree of your application, hydrating an element, then moving into its children, hydrating the first child, then moving to its siblings, and so on. This assumes that the tree of HTML produced by the your application on the server maps directly onto the DOM tree into which the browser parses that HTML.

There are a few cases to be aware of in which the tree of HTML created by your `view` and the DOM tree might not correspond exactly: these can cause hydration errors.

Invalid HTML

Here's a very simple application that causes a hydration error:

```
#[component]  
pub fn App() -> impl IntoView {  
    let count = RwSignal::new(0);  
  
    view! {  
        <p>  
            <div class:blue=move || count.get() == 2>  
                "First"  
            </div>  
        </p>  
    }  
}
```

This will give an error message like

A hydration error occurred while trying to hydrate an element defined at `src/app.rs:6:14`.

The framework expected a text node, but found this instead: `<p></p>`

The hydration mismatch may have occurred slightly earlier, but this is the first time the framework found a node of an unexpected type.

(In most browser devtools, you can right-click on that `<p></p>` to show where it appears in the DOM, which is handy.)

If you look in the DOM inspector, you'll see that it instead of a `<div>` inside a `<p>`, it shows:

```
<p></p>
<div>First</div>
<p></p>
```

That's because this is invalid HTML! A `<div>` cannot go inside a `<p>`. When the browser parses that `<div>`, it actually closes the preceding `<p>`, then opens the `<div>`; then, when it sees the (now-unmatched) closing `</p>`, it treats it as a new, empty `<p>`.

As a result, our DOM tree no longer matches the expected view tree, and a hydration error ensues.

Unfortunately, it is difficult to ensure the validity of HTML in the view at compile time using our current model, and without an effect on compile times across the board. For now, if you run into issues like this, consider running the HTML output through a validator. (In the case above, the W3C HTML Validator does in fact show an error!)

Info

You may notice some bugs of this arise when migrating from 0.6 to 0.7. This is due to a change in how hydration works.

Leptos 0.1-0.6 used a method of hydration in which each HTML element was given a unique ID, which was then used to find it in the DOM by ID. Leptos 0.7 instead began walking over the DOM directly, hydrating each element as it came. This has much better performance characteristics (shorter, cleaner HTML output and faster hydration times) but is less resilient to the invalid or edge-case HTML examples above. Perhaps more importantly, this approach also fixes a number of *other* edge cases and bugs in hydration, making the framework more resilient on net.

`<table>` without `<tbody>`

There's one additional edge case I'm aware of, in which *valid* HTML produces a DOM tree that differs from the view tree, and that's `<table>`. When (most) browsers parse an HTML

`<table>` , they insert a `<tbody>` into the DOM, whether you included one or not.

```
#[component]
pub fn App() -> impl IntoView {
    let count = RwSignal::new(0);

    view! {
        <table>
            <tr>
                <td class:blue=move || count.get() == 0>"First"</td>
            </tr>
        </table>
    }
}
```

Again, this generates a hydration error, because the browser has inserted an additional `<tbody>` into the DOM tree that was not in your view.

Here, the fix is simple: adding `<tbody>` :

```
#[component]
pub fn App() -> impl IntoView {
    let count = RwSignal::new(0);

    view! {
        <table>
            <tbody>
                <tr>
                    <td class:blue=move || count.get() == 0>"First"</td>
                </tr>
            </tbody>
        </table>
    }
}
```

(It would be worth exploring in the future whether we can lint for this particular quirk more easily than linting for valid HTML.)

General Advice

These kind of mismatches can be tricky. In general, my recommendation for debugging:

1. Right-click on the element in the message to see where the framework first *notices* the problem.
2. Compare the DOM at that point and above it, checking for mismatches with your view tree. Are there extra elements? Missing elements?

Not all client code can run on the server

Imagine you happily import a dependency like `gloo-net` that you've been used to using to make requests in the browser, and use it in a `create_resource` in a server-rendered app.

You'll probably instantly see the dreaded message

```
panicked at 'cannot call wasm-bindgen imported functions on non-wasm targets'
```

Uh-oh.

But of course this makes sense. We've just said that your app needs to run on the client and the server.

Solution

There are a few ways to avoid this:

1. Only use libraries that can run on both the server and the client. `request`, for example, works for making HTTP requests in both settings.
2. Use different libraries on the server and the client, and gate them using the `#[cfg]` macro. ([Click here for an example.](#))
3. Wrap client-only code in `Effect::new`. Because effects only run on the client, this can be an effective way to access browser APIs that are not needed for initial rendering.

For example, say that I want to store something in the browser's `localStorage` whenever a signal changes.

```
#[component]
pub fn App() -> impl IntoView {
    use gloo_storage::Storage;
    let storage = gloo_storage::LocalStorage::raw();
    logging::log!("{storage:?}");
}
```

This panics because I can't access `LocalStorage` during server rendering.

But if I wrap it in an effect...

```
#[component]
pub fn App() -> impl IntoView {
    use gloo_storage::Storage;
    Effect::new(move |_| {
        let storage = gloo_storage::LocalStorage::raw();
        log!("{storage:?}");
    });
}
```

It's fine! This will render appropriately on the server, ignoring the client-only code, and then access the storage and log a message on the browser.

Not all server code can run on the client

WebAssembly running in the browser is a pretty limited environment. You don't have access to a file-system or to many of the other things the standard library may be used to having. Not every crate can even be compiled to WASM, let alone run in a WASM environment.

In particular, you'll sometimes see errors about the crate `mio` or missing things from `core`. This is generally a sign that you are trying to compile something to WASM that can't be compiled to WASM. If you're adding server-only dependencies, you'll want to mark them `optional = true` in your `Cargo.toml` and then enable them in the `ssr` feature definition. (Check out one of the template `Cargo.toml` files to see more details.)

You can create an `Effect` to specify that something should only run on the client, and not on the server. Is there a way to specify that something should run only on the server, and not the client?

In fact, there is. The next chapter will cover the topic of server functions in some detail. (In the meantime, you can check out their docs [here](#).)

Working with the Server

The previous section described the process of server-side rendering, using the server to generate an HTML version of the page that will become interactive in the browser. So far, everything has been “isomorphic”; in other words, your app has had the “same (*iso*) shape (*morphe*)” on the client and the server.

But a server can do a lot more than just render HTML! In fact, a server can do a whole bunch of things your browser *can't*, like reading from and writing to a SQL database.

If you're used to building JavaScript frontend apps, you're probably used to calling out to some kind of REST API to do this sort of server work. If you're used to building sites with PHP or Python or Ruby (or Java or C# or...), this server-side work is your bread and butter, and it's the client-side interactivity that tends to be an afterthought.

With Leptos, you can do both: not only in the same language, not only sharing the same types, but even in the same files!

This section will talk about how to build the uniquely-server-side parts of your application.

Server Functions

If you're creating anything beyond a toy app, you'll need to run code on the server all the time: reading from or writing to a database that only runs on the server, running expensive computations using libraries you don't want to ship down to the client, accessing APIs that need to be called from the server rather than the client for CORS reasons or because you need a secret API key that's stored on the server and definitely shouldn't be shipped down to a user's browser.

Traditionally, this is done by separating your server and client code, and by setting up something like a REST API or GraphQL API to allow your client to fetch and mutate data on the server. This is fine, but it requires you to write and maintain your code in multiple separate places (client-side code for fetching, server-side functions to run), as well as creating a third thing to manage, which is the API contract between the two.

Leptos is one of a number of modern frameworks that introduce the concept of **server functions**. Server functions have two key characteristics:

1. Server functions are **co-located** with your component code, so that you can organize your work by feature, not by technology. For example, you might have a "dark mode" feature that should persist a user's dark/light mode preference across sessions, and be applied during server rendering so there's no flicker. This requires a component that needs to be interactive on the client, and some work to be done on the server (setting a cookie, maybe even storing a user in a database.) Traditionally, this feature might end up being split between two different locations in your code, one in your "frontend" and one in your "backend." With server functions, you'll probably just write them both in one `dark_mode.rs` and forget about it.
2. Server functions are **isomorphic**, i.e., they can be called either from the server or the browser. This is done by generating code differently for the two platforms. On the server, a server function simply runs. In the browser, the server function's body is replaced with a stub that actually makes a fetch request to the server, serializing the arguments into the request and deserializing the return value from the response. But on either end, the function can simply be called: you can create an `add_todo` function that writes to your database, and simply call it from a click handler on a button in the browser!

Using Server Functions

Actually, I kind of like that example. What would it look like? It's pretty simple, actually.

```
// todo.rs

#[server]
pub async fn add_todo(title: String) -> Result<(), ServerFnError> {
    let mut conn = db().await?;

    match sqlx::query("INSERT INTO todos (title, completed) VALUES ($1,
false)")
        .bind(title)
        .execute(&mut conn)
        .await
    {
        Ok(_row) => Ok(()),
        Err(e) => Err(ServerFnError::ServerError(e.to_string())),
    }
}

#[component]
pub fn BusyButton() -> impl IntoView {
    view! {
        <button on:click=move |_| {
            spawn_local(async {
                add_todo("So much to do!".to_string()).await;
            });
        }>
        "Add Todo"
    </button>
    }
}
```

You'll notice a couple things here right away:

- Server functions can use server-only dependencies, like `sqlx`, and can access server-only resources, like our database.
- Server functions are `async`. Even if they only did synchronous work on the server, the function signature would still need to be `async`, because calling them from the browser *must* be asynchronous.
- Server functions return `Result<T, ServerFnError>`. Again, even if they only do infallible work on the server, this is true, because `ServerFnError`'s variants include the various things that can be wrong during the process of making a network request.
- Server functions can be called from the client. Take a look at our click handler. This is code that will *only ever* run on the client. But it can call the function `add_todo` (using `spawn_local` to run the `Future`) as if it were an ordinary `async` function:

```
move |_| {
    spawn_local(async {
        add_todo("So much to do!".to_string()).await;
    });
}
```

- Server functions are top-level functions defined with `fn`. Unlike event listeners, derived signals, and most everything else in Leptos, they are not closures! As `fn` calls, they have no access to the reactive state of your app or anything else that is not passed in as an argument. And again, this makes perfect sense: When you make a request to the server, the server doesn't have access to client state unless you send it explicitly. (Otherwise we'd have to serialize the whole reactive system and send it across the wire with every request. This would not be a great idea.)
- Server function arguments and return values both need to be serializable. Again, hopefully this makes sense: while function arguments in general don't need to be serialized, calling a server function from the browser means serializing the arguments and sending them over HTTP.

There are a few things to note about the way you define a server function, too.

- Server functions are created by using the `#[server]` macro to annotate a top-level function, which can be defined anywhere.

Server functions work by using conditional compilation. On the server, the server function creates an HTTP endpoint that receives its arguments as an HTTP request, and returns its result as an HTTP response. For the client-side/browser build, the body of the server function is stubbed out with an HTTP request.

Warning

An Important Note about Security

Server functions are a cool technology, but it's very important to remember. **Server functions are not magic; they're syntax sugar for defining a public API.** The *body* of a server function is never made public; it's just part of your server binary. But the server function is a publicly accessible API endpoint, and its return value is just a JSON or similar blob. Do not return information from a server function unless it is public, or you've implemented proper security procedures. These procedures might include authenticating incoming requests, ensuring proper encryption, rate limiting access, and more.

Customizing Server Functions

By default, server functions encode their arguments as an HTTP POST request (using `serde_qs`) and their return values as JSON (using `serde_json`). This default is intended to promote compatibility with the `<form>` element, which has native support for making POST

requests, even when WASM is disabled, unsupported, or has not yet loaded. They mount their endpoints at a hashed URL intended to prevent name collisions.

However, there are many ways to customize server functions, with a variety of supported input and output encodings, the ability to set specific endpoints, and so on.

Take a look at the docs for the `#[server] macro` and `server_fn crate`, and the extensive `server_fns_axum example` in the repo for more information and examples.

Using Custom Errors

Server functions can return any kind of errors that implement the `FromServerFnError` trait. This makes error handling much more ergonomic and allows you to provide domain-specific error information to your clients:

```
use leptos::prelude::*;
use server_fn::codec::JsonEncoding;
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Deserialize, Serialize)]
pub enum AppError {
    ServerFnError(ServerFnErrorErr),
    DbError(String),
}

impl FromServerFnError for AppError {
    type Encoder = JsonEncoding;

    fn from_server_fn_error(value: ServerFnErrorErr) -> Self {
        AppError::ServerFnError(value)
    }
}

#[server]
pub async fn create_user(name: String, email: String) -> Result<User, AppError> {
    // Try to create user in database
    match insert_user_into_db(&name, &email).await {
        Ok(user) => Ok(user),
        Err(e) => Err(AppError::DbError(e.to_string())),
    }
}
```

Quirks to Note

Server functions come with a few quirks that are worth noting:

- Using pointer-sized integer types such as `isize` and `usize` can lead to errors when making calls between the 32-bit WASM architecture and a 64-bit server architecture; if the server responds with a value that doesn't fit in 32 bits, this will lead to a deserialization error. Use fixed size types such as `i32` or `i64` to mitigate this problem.
- Arguments sent to the server are URL-encoded using `serde_qs` by default. This allows them to work well with `<form>` elements, but can have some quirks: for example, the current version of `serde_qs` does not always work well with optional types (see [here](#) or [here](#)) or with enums that have tuple variants (see [here](#)). You can use the workarounds described in those issues, or [switch to an alternate input encoding](#).

Integrating Server Functions with Leptos

So far, everything I've said is actually framework agnostic. (And in fact, the Leptos server function crate has been integrated into Dioxus as well!) Server functions are simply a way of defining a function-like RPC call that leans on Web standards like HTTP requests and URL encoding.

But in a way, they also provide the last missing primitive in our story so far. Because a server function is just a plain Rust async function, it integrates perfectly with the async Leptos primitives we discussed [earlier](#). So you can easily integrate your server functions with the rest of your applications:

- Create **resources** that call the server function to load data from the server
- Read these resources under `<Suspense/>` or `<Transition/>` to enable streaming SSR and fallback states while data loads.
- Create **actions** that call the server function to mutate data on the server

The final section of this book will make this a little more concrete by introducing patterns that use progressively-enhanced HTML forms to run these server actions.

But in the next few chapters, we'll actually take a look at some of the details of what you might want to do with your server functions, including the best ways to integrate with the powerful extractors provided by the Actix and Axum server frameworks.

Extractors

The server functions we looked at in the last chapter showed how to run code on the server, and integrate it with the user interface you're rendering in the browser. But they didn't show you much about how to actually use your server to its full potential.

Server Frameworks

We call Leptos a “full-stack” framework, but “full-stack” is always a misnomer (after all, it never means everything from the browser to your power company.) For us, “full stack” means that your Leptos app can run in the browser, and can run on the server, and can integrate the two, drawing together the unique features available in each; as we've seen in the book so far, a button click on the browser can drive a database read on the server, both written in the same Rust module. But Leptos itself doesn't provide the server (or the database, or the operating system, or the firmware, or the electrical cables...)

Instead, Leptos provides integrations for the two most popular Rust web server frameworks, Actix Web (`leptos_actix`) and Axum (`leptos_axum`). We've built integrations with each server's router so that you can simply plug your Leptos app into an existing server with `.leptos_routes()` , and easily handle server function calls.

If you haven't seen our [Actix](#) and [Axum](#) templates, now's a good time to check them out.

Using Extractors

Both Actix and Axum handlers are built on the same powerful idea of **extractors**. Extractors “extract” typed data from an HTTP request, allowing you to access server-specific data easily.

Leptos provides `extract` helper functions to let you use these extractors directly in your server functions, with a convenient syntax very similar to handlers for each framework.

Actix Extractors

The `extract` function in `leptos_actix` takes a handler function as its argument. The handler follows similar rules to an Actix handler: it is an async function that receives arguments that will be extracted from the request and returns some value. The handler

function receives that extracted data as its arguments, and can do further `async` work on them inside the body of the `async move` block. It returns whatever value you return back out into the server function.

```
use serde::Deserialize;

#[derive(Deserialize, Debug)]
struct MyQuery {
    foo: String,
}

#[server]
pub async fn actix_extract() -> Result<String, ServerFnError> {
    use actix_web::dev::ConnectionInfo;
    use actix_web::web::Query;
    use leptos_actix::extract;

    let (Query(search), connection): (Query<MyQuery>, ConnectionInfo) =
        extract().await?;
    Ok(format!("search = {search:?}\nconnection = {connection:?}"))
}
```

Axum Extractors

The syntax for the `leptos_axum::extract` function is very similar.

```
use serde::Deserialize;

#[derive(Deserialize, Debug)]
struct MyQuery {
    foo: String,
}

#[server]
pub async fn axum_extract() -> Result<String, ServerFnError> {
    use axum::{extract::Query, http::Method};
    use leptos_axum::extract;

    let (method, query): (Method, Query<MyQuery>) = extract().await?;

    Ok(format!("{method:?} and {query:?}"))
}
```

These are relatively simple examples accessing basic data from the server. But you can use extractors to access things like headers, cookies, database connection pools, and more, using the exact same `extract()` pattern.

The Axum `extract` function only supports extractors for which the state is `()`. If you need an extractor that uses `state`, you should use `extract_with_state`. This requires you to provide the state. You can do this by extending the existing `LeptosOptions` state using the

Axum `FromRef` pattern, which providing the state as context during render and server functions with custom handlers.

```
use axum::extract::FromRef;

/// Derive FromRef to allow multiple items in state, using Axum's
/// SubStates pattern.
#[derive(FromRef, Debug, Clone)]
pub struct AppState{
    pub leptos_options: LeptosOptions,
    pub pool: SqlitePool
}
```

[Click here for an example of providing context in custom handlers.](#)

Axum State

Axum's typical pattern for dependency injection is to provide a `State`, which can then be extracted in your route handler. Leptos provides its own method of dependency injection via context. Context can often be used instead of `state` to provide shared server data (for example, a database connection pool).

```
let connection_pool = /* some shared state here */;

let app = Router::new()
    .leptos_routes_with_context(
        &leptos_options,
        routes,
        move || provide_context(connection_pool.clone()),
        {
            let leptos_options = leptos_options.clone();
            move || shell(leptos_options.clone())
        },
    )
// etc.
```

This context can then be accessed with a simple `use_context::<T>()` inside your server functions.

If you *need* to use `state` in a server function—for example, if you have an existing Axum extractor that requires `state`—that is also possible using Axum's `FromRef` pattern and `extract_with_state`. Essentially you'll need to provide the state both via context and via Axum router state:

```

#[derive(FromRef, Debug, Clone)]
pub struct MyData {
    pub value: usize,
    pub leptos_options: LeptosOptions,
}

let app_state = MyData {
    value: 42,
    leptos_options,
};

// build our application with a route
let app = Router::new()
    .leptos_routes_with_context(
        &app_state,
        routes,
        {
            let app_state = app_state.clone();
            move || provide_context(app_state.clone())
        },
        App,
    )
    .fallback(file_and_error_handler)
    .with_state(app_state);

// ...
#[server]
pub async fn uses_state() -> Result<(), ServerFnError> {
    let state = expect_context::<MyData>();
    let SomeStateExtractor(data) = extract_with_state(&state).await?;
    // todo
}

```

Generic State

In some cases, you may want to use a generic type for your state. Let's use the following example:

```

pub struct AppState<TS: ThingService> {
    pub thing_service: Arc<TS>,
}

```

In Axum you would typically use a generic parameter with your handler, like so:

```

pub async fn do_thing<TS: ThingService>(
    State(state): State<AppState<TS>>,
) -> Result<(), ThingError> {
    state.thing_service.do_thing()
}

```

Unfortunately, generic parameters are not currently supported in Leptos server functions. However, you can work around this limitation by using a concrete type in your server

function to call an inner generic function. Here's how you can do it:

```
pub async do_thing_inner<TS: ThingService>() -> Result<(), ServerFnError> {
    let state = expect_context::<AppState<TS>>(); // Works!
    state.thing_service.do_thing()
}

#[server]
pub async do_thing() -> Result<(), ServerFnError> {
    use crate::thing::service::Service as ConcreteThingService;
    use crate::thing::some_dep::SomeDep;

    do_thing_inner::<ConcreteThingService<SomeDep>>>().await
}
```

A Note about Data-Loading Patterns

Because Actix and (especially) Axum are built on the idea of a single round-trip HTTP request and response, you typically run extractors near the “top” of your application (i.e., before you start rendering) and use the extracted data to determine how that should be rendered. Before you render a `<button>`, you load all the data your app could need. And any given route handler needs to know all the data that will need to be extracted by that route.

But Leptos integrates both the client and the server, and it’s important to be able to refresh small pieces of your UI with new data from the server without forcing a full reload of all the data. So Leptos likes to push data loading “down” in your application, as far towards the leaves of your user interface as possible. When you click a `<button>`, it can refresh just the data it needs. This is exactly what server functions are for: they give you granular access to data to be loaded and reloaded.

The `extract()` functions let you combine both models by using extractors in your server functions. You get access to the full power of route extractors, while decentralizing knowledge of what needs to be extracted down to your individual components. This makes it easier to refactor and reorganize routes: you don’t need to specify all the data a route needs up front.

Responses and Redirects

Extractors provide an easy way to access request data inside server functions. Leptos also provides a way to modify the HTTP response, using the `ResponseOptions` type (see docs for [Actix](#) or [Axum](#) types) and the `redirect` helper function (see docs for [Actix](#) or [Axum](#)).

ResponseOptions

`ResponseOptions` is provided via context during the initial server rendering response and during any subsequent server function call. It allows you to easily set the status code for the HTTP response, or to add headers to the HTTP response, e.g., to set cookies.

```
#[server]
pub async fn tea_and_cookies() -> Result<(), ServerFnError> {
    use actix_web::{
        cookie::Cookie,
        http::header::HeaderValue,
        http::{header, StatusCode},
    };
    use leptos_actix::ResponseOptions;

    // pull ResponseOptions from context
    let response = expect_context::<ResponseOptions>();

    // set the HTTP status code
    response.set_status(StatusCode::IM_A_TEAPOT);

    // set a cookie in the HTTP response
    let cookie = Cookie::build("biscuits", "yes").finish();
    if let Ok(cookie) = HeaderValue::from_str(&cookie.to_string()) {
        response.insert_header(header::SET_COOKIE, cookie);
    }
    Ok(())
}
```

redirect

One common modification to an HTTP response is to redirect to another page. The Actix and Axum integrations provide a `redirect` function to make this easy to do.

```
#[server]
pub async fn login(
    username: String,
    password: String,
    remember: Option<String>,
) -> Result<(), ServerFnError> {
    const INVALID_CREDENTIALS: fn() -> ServerFnError = || -> ServerFnError {
        ServerFnError::ServerError("Invalid credentials".into())
    };

    // pull the DB pool and auth provider from context
    let pool = pool()?;
    let auth = auth()?;

    // check whether the user exists
    let user: User = User::get_from_username(username, &pool)
        .await
        .ok_or_else(INVALID_CREDENTIALS)?;

    // check whether the user has provided the correct password
    match verify(password, &user.password)? {
        // if the password is correct...
        true => {
            // log the user in
            auth.login_user(user.id);
            auth.remember_user(remember.is_some());

            // and redirect to the home page
            leptos_axum::redirect("/");
            Ok(())
        }
        // if not, return an error
        false => Err(INVALID_CREDENTIALS()),
    }
}
```

This server function can then be used from your application. This `redirect` works well with the progressively-enhanced `<ActionForm/>` component: without JS/WASM, the server response will redirect because of the status code and header. With JS/WASM, the `<ActionForm/>` will detect the redirect in the server function response, and use client-side navigation to redirect to the new page.

Reference: Asynchronous Closures/Futures

With all the different types of asynchronous closures and futures having been introduced, it may become difficult to keep track of where and when they execute in different contexts. The following table briefly describes when and where the code is executed for each of the introduced asynchronous closure types. Note how each asynchronous closure type behaves differently when executed on the server and on the client in conjunction with the relevant stage of execution (SSR/hydration/CSR).

	Server-side rendering	During hydration (preparing for CSR)	Client-side rendering
Server Functions (Server)	Invoked as a normal asynchronous function, with the code defined in the body directly executed and the results returned to the caller. When executed inside a component, the full reactive context is accessible.	-	(Invoked by responding to the HTTP request made to the server function's standalone endpoint, where the response will be the serialized result. Since this endpoint is fully standalone, reactive context is typically not available.)
Server Functions (Client)	-	Not executed. (When defined inside resource typed async closures.)	Makes an HTTP request to the server function's endpoint, the response body will be deserialized into the result. Take note that the code in the function body is never compiled into the client.
Resources (Server)	Their async closures are executed in <i>parallel</i> when possible by the async executor; results are encoded into the	-	-

	Server-side rendering	During hydration (preparing for CSR)	Client-side rendering
	response body inside <code><script></code> tags.		
Resources (Client)	-	Not executed; the <code><script></code> tags within the response body provides the results.	Their async closures are executed whenever required by the reactive system to return any new results.
Local Resources (Client Only)	-	Not executed. (Will be executed once after hydration to provide the initial value.)	Their async closures are executed whenever required by the reactive system to return any new results.
Suspend (Server)	These futures are executed in <i>parallel</i> when possible by the async executor. The selected <code>SsrMode</code> can modify how and when: 1) these futures are polled, 2) the ordering of the stream of outputs.	-	-
Suspend (Client)	-	These futures are fully executed once more in the client as part of hydration; returned <code>view! {}</code> will be used to hydrate the HTML.	These futures are executed whenever required by the reactive system; returned <code>view! {}</code> will be rendered wherever they are defined.

A useful note to keep in mind is that there are up to three different stages of execution, split between both the server and the client. One consequence of this are the potential for hydration bugs or unexpected program behavior if there are side effects inside a Resource or Suspend. These side effects may not be executed in the expected time or in the identical order across the different stages of execution, and this mismatch may lead to hydration errors.

For example, a side effect might arise from a resource that sets a signal. The resource might pass a value that's sourced asynchronously through a reactive signal in its asynchronous closure - this behaves perfectly fine under SSR and CSR, but will not behave in the expected manner if hydration is involved. During hydration, the body of the asynchronous closure *will not* be executed, thus resulting in the signal not being set during hydration; and since that closure under SSR would be executed and thus the signal would be set there, this mismatch between the signal being set under SSR and not set during hydration may lead to a hydration error. To correct for this, have the resource return the value, so that a Suspend may `.await` for it before passing it through the reactive signal, ensuring the signal is set under all three execution stages.

Another way this mismatch may arise is when a single signal is used by multiple Suspend. The Suspend may be polled in a different order on the server versus how they may be polled on the client; this is simply due to the different async executors being used in these two contexts. This effectively results in an unstable order of execution, leading to the changing of the order of which the values are passed through the signal. Ultimately the result is yet another potential for values being mismatched under SSR versus hydration, and in this particular instance the hydration error may become non-deterministic and thus difficult to narrow down.

As another example, during SSR, server functions will have access to the full set of contexts set up by the application, as they will be invoked as standard async functions during rendering of the application. On the otherhand, their API endpoint counterparts exist as much more simple standalone endpoint handlers, thus they will run without any of the contexts that the application have set up. Hence the use of `expect_context` and `use_context` on types that are set up by the application will not work, unless the values were provided to context by `.leptos_routes_with_context()`.

The above are just some examples to serve as a quick reference of the behavior of the different types under the typical documented configuration (i.e. server functions are only invoked/awaited inside Resources, and Resources are only awaited inside Suspend), as combining or nesting of these asynchronous closures in some other manner can cause other behaviors.

Progressive Enhancement (and Graceful Degradation)

I've been driving around Boston for about fifteen years. If you don't know Boston, let me tell you: Massachusetts has some of the most aggressive drivers (and pedestrians!) in the world. I've learned to practice what's sometimes called "defensive driving": assuming that someone's about to swerve in front of you at an intersection when you have the right of way, preparing for a pedestrian to cross into the street at any moment, and driving accordingly.

"Progressive enhancement" is the "defensive driving" of web design. Or really, that's "graceful degradation," although they're two sides of the same coin, or the same process, from two different directions.

Progressive enhancement, in this context, means beginning with a simple HTML site or application that works for any user who arrives at your page, and gradually enhancing it with layers of additional features: CSS for styling, JavaScript for interactivity, WebAssembly for Rust-powered interactivity; using particular Web APIs for a richer experience if they're available and as needed.

Graceful degradation means handling failure gracefully when parts of that stack of enhancement *aren't* available. Here are some sources of failure your users might encounter in your app:

- Their browser doesn't support WebAssembly because it needs to be updated.
- Their browser can't support WebAssembly because browser updates are limited to newer OS versions, which can't be installed on the device. (Looking at you, Apple.)
- They have WASM turned off for security or privacy reasons.
- They have JavaScript turned off for security or privacy reasons.
- JavaScript isn't supported on their device (for example, some accessibility devices only support HTML browsing)
- The JavaScript (or WASM) never arrived at their device because they walked outside and lost WiFi.
- They stepped onto a subway car after loading the initial page and subsequent navigations can't load data.
- ... and so on.

How much of your app still works if one of these holds true? Two of them? Three?

If the answer is something like "95%... okay, then 90%... okay, then 75%," that's graceful degradation. If the answer is "my app shows a blank screen unless everything works correctly," that's... rapid unscheduled disassembly.

Graceful degradation is especially important for WASM apps, because WASM is the newest and least-likely-to-be-supported of the four languages that run in the browser (HTML, CSS, JS, WASM).

Luckily, we've got some tools to help.

Defensive Design

There are a few practices that can help your apps degrade more gracefully:

1. **Server-side rendering.** Without SSR, your app simply doesn't work without both JS and WASM loading. In some cases this may be appropriate (think internal apps gated behind a login) but in others it's simply broken.
2. **Native HTML elements.** Use HTML elements that do the things that you want, without additional code: `<a>` for navigation (including to hashes within the page), `<details>` for an accordion, `<form>` to persist information in the URL, etc.
3. **URL-driven state.** The more of your global state is stored in the URL (as a route param or part of the query string), the more of the page can be generated during server rendering and updated by an `<a>` or a `<form>`, which means that not only navigations but state changes can work without JS/WASM.
4. **`SsrMode::PartiallyBlocked` OR `SsrMode::InOrder`.** Out-of-order streaming requires a small amount of inline JS, but can fail if 1) the connection is broken halfway through the response or 2) the client's device doesn't support JS. Async streaming will give a complete HTML page, but only after all resources load. In-order streaming begins showing pieces of the page sooner, in top-down order. "Partially-blocked" SSR builds on out-of-order streaming by replacing `<Suspense/>` fragments that read from blocking resources on the server. This adds marginally to the initial response time (because of the $O(n)$ string replacement work), in exchange for a more complete initial HTML response. This can be a good choice for situations in which there's a clear distinction between "more important" and "less important" content, e.g., blog post vs. comments, or product info vs. reviews. If you choose to block on all the content, you've essentially recreated async rendering.
5. **Leaning on `<form>`s.** There's been a bit of a `<form>` renaissance recently, and it's no surprise. The ability of a `<form>` to manage complicated POST or GET requests in an easily-enhanced way makes it a powerful tool for graceful degradation. The example in [the `<Form/>` chapter](#), for example, would work fine with no JS/WASM: because it uses a `<form method="GET">` to persist state in the URL, it works with pure HTML by making normal HTTP requests and then progressively enhances to use client-side navigations instead.

There's one final feature of the framework that we haven't seen yet, and which builds on this characteristic of forms to build powerful applications: the `<ActionForm/>`.

<ActionForm/>

`<ActionForm/>` is a specialized `<Form/>` that takes a server action, and automatically dispatches it on form submission. This allows you to call a server function directly from a `<form>`, even without JS/WASM.

The process is simple:

1. Define a server function using the `#[server]` macro (see [Server Functions](#).)
2. Create an action using `ServerAction::new()`, specifying the type of the server function you've defined.
3. Create an `<ActionForm/>`, providing the server action in the `action` prop.
4. Pass the named arguments to the server function as form fields with the same names.

Note: `<ActionForm/>` only works with the default URL-encoded `POST` encoding for server functions, to ensure graceful degradation/correct behavior as an HTML form.

```
#[server]
pub async fn add_todo(title: String) -> Result<(), ServerFnError> {
    todo!()
}

#[component]
fn AddTodo() -> impl IntoView {
    let add_todo = ServerAction::<AddTodo>::new();
    // holds the latest *returned* value from the server
    let value = add_todo.value();
    // check if the server has returned an error
    let has_error = move || value.with(|val| matches!(val, Some(Err(_))));

    view! {
        <ActionForm action=add_todo>
            <label>
                "Add a Todo"
                // `title` matches the `title` argument to `add_todo`
                <input type="text" name="title"/>
            </label>
            <input type="submit" value="Add"/>
        </ActionForm>
    }
}
```

It's really that easy. With JS/WASM, your form will submit without a page reload, storing its most recent submission in the `.input()` signal of the action, its pending status in `.pending()`, and so on. (See the [Action](#) docs for a refresher, if you need.) Without JS/WASM, your form will submit with a page reload. If you call a `redirect` function (from `leptos_axum` or `leptos_actix`) it will redirect to the correct page. By default, it will redirect

back to the page you're currently on. The power of HTML, HTTP, and isomorphic rendering mean that your `<ActionForm/>` simply works, even with no JS/WASM.

Client-Side Validation

Because the `<ActionForm/>` is just a `<form>`, it fires a `submit` event. You can use either HTML validation, or your own client-side validation logic in an `on:submit:capture` handler. Just call `ev.preventDefault()` to prevent submission.

The `FromFormData` trait can be helpful here, for attempting to parse your server function's data type from the submitted form.

```
let on_submit = move |ev| {
    let data = AddTodo::from_event(&ev);
    // silly example of validation: if the todo is "nope!", nope it
    if data.is_err() || data.unwrap().title == "nope!" {
        // ev.preventDefault() will prevent form submission
        ev.preventDefault();
    }
}

// ... add the `submit` handler to an `ActionForm`

<ActionForm on:submit:capture=on_submit /* ... */>
```

Note

Note the use of `on:submit:capture` rather than `on:submit`. This adds an event listener that will fire during the browser's "capture" phase of event handling, rather than during the "bubble" phase, which means that your event handler will run before the built-in `submit` handler of the `ActionForm`. For more information, check out [this issue](#).

Complex Inputs

Server function arguments that are structs with nested serializable fields should make use of indexing notation of `serde_qs`.

```
#[derive(serde::Serialize, serde::Deserialize, Debug, Clone)]
struct Settings {
    display_name: String,
}

#[derive(serde::Serialize, serde::Deserialize, Debug, Clone)]
struct HeftyData {
    first_name: String,
    last_name: String,
    settings: Settings,
}

#[component]
fn ComplexInput() -> impl IntoView {
    let submit = ServerAction::<VeryImportantFn>::new();

    view! {
        <ActionForm action=submit>
            <input type="text" name="hefty_arg[first_name]" value="leptos"/>
            <input
                type="text"
                name="hefty_arg[last_name]"
                value="closures-everywhere"
            />
            <input
                type="text"
                name="hefty_arg[settings][display_name]"
                value="my alias"
            />
            <input type="submit"/>
        </ActionForm>
    }
}

#[server]
async fn very_important_fn(hefty_arg: HeftyData) -> Result<(), ServerFnError> {
    assert_eq!(hefty_arg.first_name.as_str(), "leptos");
    assert_eq!(hefty_arg.last_name.as_str(), "closures-everywhere");
    assert_eq!(hefty_arg.settings.display_name.as_str(), "my alias");
    Ok(())
}
```

Deployment

There are as many ways to deploy a web application as there are developers, let alone applications. But there are a couple useful tips to keep in mind when deploying an app.

General Advice

1. Remember: Always deploy Rust apps built in `--release` mode, not debug mode. This has a huge effect on both performance and binary size.
2. Test locally in release mode as well. The framework applies certain optimizations in release mode that it does not apply in debug mode, so it's possible for bugs to surface at this point. (If your app behaves differently or you do encounter a bug, it's likely a framework-level bug and you should open a GitHub issue with a reproduction.)
3. See the chapter on "Optimizing WASM Binary Size" for additional tips and tricks to further improve the time-to-interactive metric for your WASM app on first load.

We asked users to submit their deployment setups to help with this chapter. I'll quote from them below, but you can read the full thread [here](#).

Deploying a Client-Side-Rendered App

If you've been building an app that only uses client-side rendering, working with Trunk as a dev server and build tool, the process is quite easy.

```
trunk build --release
```

`trunk build` will create a number of build artifacts in a `dist/` directory. Publishing `dist` somewhere online should be all you need to deploy your app. This should work very similarly to deploying any JavaScript application.

We've created several example repositories which show how to set up and deploy a Leptos CSR app to various hosting services.

Note: Leptos does not endorse the use of any particular hosting service - feel free to use any service that supports static site deploys.

Examples:

- [Github Pages](#)
- [Vercel](#)
- [Spin \(serverless WebAssembly\)](#)
- [Netlify](#)

Github Pages

Deploying a Leptos CSR app to Github pages is a simple affair. First, go to your Github repo's settings and click on "Pages" in the left side menu. In the "Build and deployment" section of the page, change the "source" to "Github Actions". Then copy the following into a file such as `.github/workflows/gh-pages-deploy.yml`

 **Example**

>

For more on deploying to Github Pages [see the example repo here](#)

Vercel

Step 1: Set Up Vercel

In the Vercel Web UI...

1. Create a new project
2. Ensure
 - The "Build Command" is left empty with Override on
 - The "Output Directory" is changed to dist (which is the default output directory for Trunk builds) and the Override is on

▼ Build and Output Settings

Build Command ?

``npm run vercel-build` or `npm run build``

OVERRIDE ☒

Output Directory ?

`dist`

OVERRIDE ☒

Install Command ?

``yarn install`, `pnpm install`, `npm install`, or `bun install``

OVERRIDE ☐

> Environment Variables

Deploy

Step 2: Add Vercel Credentials for GitHub Actions

Note: Both the preview and deploy actions will need your Vercel credentials setup in GitHub secrets

1. Retrieve your [Vercel Access Token](#) by going to "Account Settings" > "Tokens" and creating a new token - save the token to use in sub-step 5, below.
2. Install the [Vercel CLI](#) using the `npm i -g vercel` command, then run `vercel login` to login to your account.
3. Inside your folder, run `vercel link` to create a new Vercel project; in the CLI, you will be asked to 'Link to an existing project?' - answer yes, then enter the name you created in step 1. A new `.vercel` folder will be created for you.

4. Inside the generated `.vercel` folder, open the `project.json` file and save the "projectId" and "orgId" for the next step.
5. Inside GitHub, go the repo's "Settings" > "Secrets and Variables" > "Actions" and add the following as [Repository secrets](#):
 - save your Vercel Access Token (from sub-step 1) as the `VERCEL_TOKEN` secret
 - from the `.vercel/project.json` add "projectId" as `VERCEL_PROJECT_ID`
 - from the `.vercel/project.json` add "orgId" as `VERCEL_ORG_ID`

For full instructions see ["How can I use Github Actions with Vercel"](#)

Step 3: Add Github Action Scripts

Finally, you're ready to simply copy and paste the two files - one for deployment, one for PR previews - from below or from [the example repo's .github/workflows/](#) folder into your own github workflows folder - then, on your next commit or PR deploys will occur automatically.

Production deployment script: `vercel_deploy.yml`

 **Example**

>

Preview deployments script: `vercel_preview.yml`

 **Example**

>

See [the example repo here](#) for more.

Spin - Serverless WebAssembly

Another option is using a serverless platform such as Spin. Although [Spin](#) is open source and you can run it on your own infrastructure (eg. inside Kubernetes), the easiest way to get started with Spin in production is to use the Fermyon Cloud.

Start by installing the [Spin CLI using the instructions, here](#), and creating a Github repo for your Leptos CSR project, if you haven't done so already.

1. Open "Fermyon Cloud" > "User Settings". If you're not logged in, choose the Login With GitHub button.

2. In the “Personal Access Tokens”, choose “Add a Token”. Enter the name “gh_actions” and click “Create Token”.
3. Fermyon Cloud displays the token; click the copy button to copy it to your clipboard.
4. Go into your Github repo and open "Settings" > "Secrets and Variables" > "Actions" and add the Fermyon cloud token to "Repository secrets" using the variable name "FERMYON_CLOUD_TOKEN"
5. Copy and paste the following Github Actions scripts (below) into your `.github/workflows/<SCRIPT_NAME>.yaml` files
6. With the 'preview' and 'deploy' scripts active, Github Actions will now generate previews on pull requests & deploy automatically on updates to your 'main' branch.

Production deployment script: `spin_deploy.yaml`

 **Example**



Preview deployment script: `spin_preview.yaml`

 **Example**



See [the example repo here](#).

Netlify

All it takes to deploy a Leptos CSR app to Netlify is to create a project and to add two simple configuration files in your project root. Let's begin with the latter.

Configuration Files

Create a `netlify.toml` file in your project root with the following content:

```
[build]
command = "rustup target add wasm32-unknown-unknown && cargo install trunk --
locked && trunk build --release"
publish = "dist"

[build.environment]
RUST_VERSION = "stable"

[[redirects]]
from = "/*"
to = "/index.html"
status = 200
```

Create a `rust-toolchain.toml` file in your project root with the following content:

```
[toolchain]
channel = "stable"
targets = ["wasm32-unknown-unknown"]
```

Deployment

1. [Add your project to Netlify](#) by connecting your Git repository
2. Netlify will automatically detect your `netlify.toml` configuration
3. If you need additional environment variables, configure them in [Netlify's environment variables settings](#)

The `rust-toolchain.toml` ensures the correct Rust toolchain and WASM target are available during the build process. The redirect rule in `netlify.toml` ensures your SPA routes work correctly by serving `index.html` for all paths.

Deploying a Full-Stack SSR App

It's possible to deploy Leptos fullstack, SSR apps to any number of server or container hosting services. The most simple way to get a Leptos SSR app into production might be to use a VPS service and either run Leptos natively in a VM ([see here for more details](#)). Alternatively, you could containerize your Leptos app and run it in [Podman](#) or [Docker](#) on any colocated or cloud server.

There are a multitude of different deployment setups and hosting services, and in general, Leptos itself is agnostic to the deployment setup you use. With this diversity of deployment targets in mind, on this page we will go over:

- [creating a Containerfile \(or Dockerfile\)](#) for use with Leptos SSR apps
- Using a [Dockerfile](#) to [deploy to a cloud service](#) - for example, [Fly.io](#)
- Deploying Leptos to [serverless runtimes](#) - for example, [AWS Lambda](#) and [JS-hosted WASM runtimes like Deno & Cloudflare](#)
- [Platforms that have not yet gained Leptos SSR support](#)

Note: Leptos does not endorse the use of any particular method of deployment or hosting service.

Creating a Containerfile

The most popular way for people to deploy full-stack apps built with `cargo-leptos` is to use a cloud hosting service that supports deployment via a Podman or Docker build. Here's a sample `Containerfile` / `Dockerfile`, which is based on the one we use to deploy the Leptos website.

Debian

```
# Get started with a build env with Rust nightly
FROM rustlang/rust:nightly-trixie as builder

# If you're using stable, use this instead
# FROM rust:1.92.0-trixie as builder # See current official Rust tags here:
# https://hub.docker.com/_/rust

# Install cargo-binstall, which makes it easier to install other
# cargo extensions like cargo-leptos
RUN wget https://github.com/cargo-bins/cargo-
binstall/releases/latest/download/cargo-binstall-x86_64-unknown-linux-musl.tgz
RUN tar -xvf cargo-binstall-x86_64-unknown-linux-musl.tgz
RUN cp cargo-binstall /usr/local/cargo/bin

# Install required tools
RUN apt-get update -y \
    && apt-get install -y --no-install-recommends clang

# Install cargo-leptos
RUN cargo binstall cargo-leptos -y

# Add the WASM target
RUN rustup target add wasm32-unknown-unknown

# Make an /app dir, which everything will eventually live in
RUN mkdir -p /app
WORKDIR /app
COPY . .

# Build the app
RUN cargo leptos build --release -vv

FROM debian:trixie-slim as runtime
WORKDIR /app
RUN apt-get update -y \
    && apt-get install -y --no-install-recommends openssl ca-certificates \
    && apt-get autoremove -y \
    && apt-get clean -y \
    && rm -rf /var/lib/apt/lists/*

# -- NB: update binary name from "leptos_start" to match your app name in
Cargo.toml --
# Copy the server binary to the /app directory
COPY --from=builder /app/target/release/leptos_start /app/

# /target/site contains our JS/WASM/CSS, etc.
COPY --from=builder /app/target/site /app/site

# Copy Cargo.toml if it's needed at runtime
COPY --from=builder /app/Cargo.toml /app/

# Set any required env variables and
ENV RUST_LOG="info"
ENV LEPTOS_SITE_ADDR="0.0.0.0:8080"
ENV LEPTOS_SITE_ROOT="site"
```

```
EXPOSE 8080
```

```
# -- NB: update binary name from "leptos_start" to match your app name in
Cargo.toml --
# Run the server
CMD ["/app/leptos_start"]
```

Alpine

```
# Get started with a build env with Rust nightly
FROM rustlang/rust:nightly-alpine as builder

RUN apk update && \
    apk add --no-cache bash curl npm libc-dev binaryen

RUN npm install -g sass

RUN curl --proto '=https' --tlsv1.3 -LsSf https://github.com/leptos-rs/cargo-
leptos/releases/latest/download/cargo-leptos-installer.sh | sh

# Add the WASM target
RUN rustup target add wasm32-unknown-unknown

WORKDIR /work
COPY . .

RUN cargo leptos build --release -vv

FROM rustlang/rust:nightly-alpine as runner

WORKDIR /app

COPY --from=builder /work/target/release/leptos_start /app/
COPY --from=builder /work/target/site /app/site
COPY --from=builder /work/Cargo.toml /app/

ENV RUST_LOG="info"
ENV LEPTOS_SITE_ADDR="0.0.0.0:8080"
ENV LEPTOS_SITE_ROOT=./site
EXPOSE 8080

CMD ["/app/leptos_start"]
```

Read more: [gnu](#) and [musl](#) build files for Leptos apps.

A Note on Reverse Proxies

While you can expose your Leptos app directly, it's usually better to put it behind a reverse proxy. This allows you to handle SSL/TLS, compression, and security headers in a dedicated layer rather than in your Rust binary.

There are several popular reverse proxy options. Caddy is often chosen for its automatic HTTPS certificate management, while Nginx, Traefik, or Apache are also widely used depending on your requirements and familiarity.

If you are using Caddy, your configuration can be as simple as pointing a domain to your container name or IP:

```
# Simple setup
example.com {
    reverse_proxy leptos-app:8080
}

# Advanced: Basic auth and HSTS headers
app.example.com {
    # Protect a staging site with basic auth
    basic_auth {
        admin $2a$14$CIW9S...
    }

    header {
        Strict-Transport-Security "max-age=31536000; includeSubDomains;
preload"
    }

    reverse_proxy leptos-app:8080
}
```

For more details, see the [Caddy Reverse Proxy Quick-start](#) and the [Caddyfile concept documentation](#), among other resources.

Cloud Deployments

Deploy to Fly.io

One option for deploying your Leptos SSR app is to use a service like [Fly.io](#), which takes a Dockerfile definition of your Leptos app and runs it in a quick-starting micro-VM; Fly also offers a variety of storage options and managed DBs to use with your projects. The following example will show how to deploy a simple Leptos starter app, just to get you up and going; [see here for more about working with storage options on Fly.io](#) if and when required.

First, create a `Dockerfile` in the root of your application and fill it in with the suggested contents (above); make sure to update the binary names in the Dockerfile example to the name of your own application, and make other adjustments as necessary.

Also, ensure you have the `flyctl` CLI tool installed, and have an account set up at [Fly.io](https://fly.io). To install `flyctl` on MacOS, Linux, or Windows WSL, run:

```
curl -L https://fly.io/install.sh | sh
```

If you have issues, or for installing to other platforms [see the full instructions here](#)

Then login to Fly.io

```
fly auth login
```

and manually launch your app using the command

```
fly launch
```

The `flyctl` CLI tool will walk you through the process of deploying your app to Fly.io.

Note

By default, Fly.io will auto-stop machines that don't have traffic coming to them after a certain period of time. Although Fly.io's lightweight VM's start up quickly, if you want to minimize the latency of your Leptos app and ensure it's always swift to respond, go into the generated `fly.toml` file and change the `min_machines_running` to 1 from the default of 0.

[See this page in the Fly.io docs for more details.](#)

If you prefer to use Github Actions to manage your deployments, you will need to create a new access token via the [Fly.io](https://fly.io) web UI.

Go to "Account" > "Access Tokens" and create a token named something like "github_actions", then add the token to your Github repo's secrets by going into your project's Github repo, then clicking "Settings" > "Secrets and Variables" > "Actions" and creating a "New repository secret" with the name "FLY_API_TOKEN".

To generate a `fly.toml` config file for deployment to Fly.io, you must first run the following from within the project source directory

```
fly launch --no-deploy
```

to create a new Fly app and register it with the service. Git commit your new `fly.toml` file.

To set up the Github Actions deployment workflow, copy the following into a `.github/workflows/fly_deploy.yml` file:

Example



On the next commit to your Github `main` branch, your project will automatically deploy to Fly.io.

See [the example repo here](#).

Railway

Another provider for cloud deployments is [Railway](#). Railway integrates with GitHub to automatically deploy your code.

There is an opinionated community template that gets you started quickly:



The template has renovate setup to keep dependencies up to date and supports GitHub Actions to test your code before a deploy happens.

Railway has a free tier that does not require a credit card, and with how little resources Leptos needs that free tier should last a long time.

See [the example repo here](#).

Deploy to Serverless Runtimes

Leptos supports deploying to FaaS (Function as a Service) or 'serverless' runtimes such as AWS Lambda as well as [WinterCG](#)-compatible JS runtimes such as [Deno](#) and Cloudflare. Just be aware that serverless environments do place some restrictions on the functionality available to your SSR app when compared with VM or container type deployments (see notes, below).

AWS Lambda

With a little help from the [Cargo Lambda](#) tool, Leptos SSR apps can be deployed to AWS Lambda. A starter template repo using Axum as the server is available at [leptos-rs/start-aws](#); the instructions there can be adapted for you to use a Leptos+Actix-web server as well. The

starter repo includes a Github Actions script for CI/CD, as well as instructions for setting up your Lambda functions and getting the necessary credentials for cloud deployment.

However, please keep in mind that some native server functionality does not work with FaaS services like Lambda because the environment is not necessarily consistent from one request to the next. In particular, the ['start-aws' docs](#) state that "since AWS Lambda is a serverless platform, you'll need to be more careful about how you manage long-lived state. Writing to disk or using a state extractor will not work reliably across requests. Instead, you'll need a database or other microservices that you can query from the Lambda function."

The other factor to bear in mind is the 'cold-start' time for functions as a service - depending on your use case and the FaaS platform you use, this may or may not meet your latency requirements; you may need to keep one function running at all times to optimize the speed of your requests.

Deno & Cloudflare Workers

Currently, Leptos-Axum supports running in Javascript-hosted WebAssembly runtimes such as Deno, Cloudflare Workers, etc. This option requires some changes to the setup of your source code (for example, in `Cargo.toml` you must define your app using `crate-type = ["cdylib"]` and the "wasm" feature must be enabled for `leptos_axum`). [The Leptos HackerNews JS-fetch example](#) demonstrates the required modifications and shows how to run an app in the Deno runtime. Additionally, the [leptos_axum crate docs](#) are a helpful reference when setting up your own `Cargo.toml` file for JS-hosted WASM runtimes.

While the initial setup for JS-hosted WASM runtimes is not onerous, the more important restriction to keep in mind is that since your app will be compiled to WebAssembly (`wasm32-unknown-unknown`) on the server as well as the client, you must ensure that the crates you use in your app are all WASM-compatible; this may or may not be a deal-breaker depending on your app's requirements, as not all crates in the Rust ecosystem have WASM support.

If you're willing to live with the limitations of WASM server-side, the best place to get started right now is by checking out the [example of running Leptos with Deno](#) in the official Leptos Github repo.

Platforms Working on Leptos Support

Deploy to Spin Serverless WASI (with Leptos SSR)

WebAssembly on the server has been gaining steam lately, and the developers of the open source serverless WebAssembly framework Spin are working on natively supporting Leptos.

While the Leptos-Spin SSR integration is still in its early stages, there is a working example you may wish to try out.

The full set of instructions to get Leptos SSR & Spin working together are available as [a post on the Fermyon blog](#), or if you want to skip the article and just start playing around with a working starter repo, [see here](#).

Deploy to Shuttle.rs

Several Leptos users have asked about the possibility of using the Rust-friendly [Shuttle.rs](#) service to deploy Leptos apps. Unfortunately, Leptos is not officially supported by the Shuttle.rs service at the moment.

However, the folks at Shuttle.rs are committed to getting Leptos support in the future; if you would like to keep up-to-date on the status of that work, keep an eye on [this Github issue](#).

Additionally, some effort has been made to get Shuttle working with Leptos, but to date, deploys to the Shuttle cloud are still not working as expected. That work is available here, if you would like to investigate for yourself or contribute fixes: [Leptos Axum Starter Template for Shuttle.rs](#).

Deploying at Non-Root Paths

So far, the deployment steps have assumed that your application is deployed at the root path of your domain (/). However, it is also possible to deploy your application at a non-root path, such as /my-app .

If you are deploying at a non-root path, you'll need to take a few steps to tell the various parts of the application what that new base path is.

Update the Router base

The `<Router/>` component has a `base` prop that specifies the base path for routing. For example, if you are deploying an application with three pages / , /about , and /contact , and you want them to be deployed at /my-app so that the three routes are /my-app , /my-app/about , and /my-app/contact , you would set the `base` prop to /my-app :

```
<Router base="/my-app">
  <Routes fallback=|| "Not found.">
    <Route path=path!("/") view=Home/>
    <Route path=path!("/about") view=About/>
    <Route path=path!("/contact") view=Contact/>
  </Routes>
</Router>
```

If you are using a reverse proxy, it's likely that your server will *think* it's serving / when it is actually serving /my-app . But in the browser, the router will still see the URL as /my-app . In this situation, you should set the `base` prop conditionally using conditional compilation:

```
let base = if cfg!(feature = "hydrate") {
  "/my-app"
} else {
  "/"
};
// ...
<Router base> // ...
```

Update the `<HydrationScripts root/>`

If you're using server rendering, the `<HydrationScripts/>` component is responsible for loading the JS/WASM to hydrate the app. This has its own `root` prop that specifies the base

path for the hydration scripts. If they are also being served from a subdirectory, you should include that base path as the `root` prop.

Update the Server Function URL

If you are using server functions, they will default to sending requests to `/`. If your server function handler is mounted at a different path, you can set that with `set_server_url`.

Trunk configuration

If you're using client-side rendering with Trunk, [consult the Trunk docs](#) on how to set the public URL via `--public-url`.

Optimizing WASM Binary Size

WebAssembly binaries are significantly larger than the JavaScript bundles you'd expect for the equivalent application. Because the WASM format is designed for streaming compilation, WASM files are much faster to compile per kilobyte than JavaScript files. (For a deeper look, you can [read this great article from the Mozilla team](#) on streaming WASM compilation.) Still, it's important to ship the smallest WASM binary to users that you can, as it will reduce their network usage and make your app interactive as quickly as possible.

So what are some practical steps?

Things to Do

1. Make sure you're looking at a release build. (Debug builds are much, much larger.)
2. Add a release profile for WASM that optimizes for size, not speed.

For a `cargo-leptos` project, for example, you can add this to your `Cargo.toml`:

```
[profile.wasm-release]
inherits = "release"
opt-level = 'z'
lto = true
codegen-units = 1

# ....

[package.metadata.leptos]
# ....
lib-profile-release = "wasm-release"
```

This will hyper-optimize the WASM for your release build for size, while keeping your server build optimized for speed. (For a pure client-rendered app without server considerations, just use the `[profile.wasm-release]` block as your `[profile.release]`.)

3. Always serve compressed WASM in production. WASM tends to compress very well, typically shrinking to less than 50% its uncompressed size, and it's trivial to enable compression for static files being served from Actix or Axum.
4. If you're using nightly Rust, you can rebuild the standard library with this same profile rather than the prebuilt standard library that's distributed with the `wasm32-unknown-unknown` target.

To do this, create a file in your project at `.cargo/config.toml`

```
[unstable]
build-std = ["std", "panic_abort", "core", "alloc"]
build-std-features = ["panic_immediate_abort"]
```

Note that if you're using this with SSR too, the same Cargo profile will be applied. You'll need to explicitly specify your target:

```
[build]
target = "x86_64-unknown-linux-gnu" # or whatever
```

Also note that in some cases, the cfg feature `has_std` will not be set, which may cause build errors with some dependencies which check for `has_std`. You may fix any build errors due to this by adding:

```
[build]
rustflags = ["--cfg=has_std"]
```

And you'll need to add `panic = "abort"` to `[profile.release]` in `Cargo.toml`. Note that this applies the same `build-std` and `panic` settings to your server binary, which may not be desirable. Some further exploration is probably needed here.

5. One of the sources of binary size in WASM binaries can be `serde` serialization/deserialization code. Leptos uses `serde` by default to serialize and deserialize resources created with `Resource::new()`. `leptos_server` includes additional features to activate alternative encodings by adding additional `new_` methods. For example, activating the `miniserde` feature on the `leptos_server` crate adds a `Resource::new_miniserde()` method, and the `serde-lite` feature adds `new_serde_lite`. `miniserde` and `serde-lite` only implement subsets of `serde`'s functionality, but typically optimize for binary size over speed.

Things to Avoid

There are certain crates that tend to inflate binary sizes. For example, the `regex` crate with its default features adds about 500kb to a WASM binary (largely because it has to pull in Unicode table data!). In a size-conscious setting, you might consider avoiding regexes in general, or even dropping down and calling browser APIs to use the built-in regex engine instead. (This is what `leptos_router` does on the few occasions it needs a regular expression.)

In general, Rust's commitment to runtime performance is sometimes at odds with a commitment to a small binary. For example, Rust monomorphizes generic functions, meaning it creates a distinct copy of the function for each generic type it's called with. This is significantly faster than dynamic dispatch, but increases binary size. Leptos tries to balance

runtime performance with binary size considerations pretty carefully; but you might find that writing code that uses many generics tends to increase binary size. For example, if you have a generic component with a lot of code in its body and call it with four different types, remember that the compiler could include four copies of that same code. Refactoring to use a concrete inner function or helper can often maintain performance and ergonomics while reducing binary size.

Code Splitting

`cargo-leptos` and the Leptos framework and router have support for WASM binary splitting. (Note that this support was released during the summer of 2025; depending on when you're reading this, we may still be ironing out bugs.)

This can be used through the combination of three tools: `cargo leptos` (`serve|watch|build`) `--split`, the `#[lazy]` macro, and the `#[lazy_route]` macro (paired with the `LazyRoute` trait).

#[lazy]

The `#[lazy]` macro indicates that a function can be lazy-loaded from a separate WebAssembly (WASM) binary. It can be used to annotate a synchronous or async function; in either case, it will produce an async function. The first time you call the lazy-loaded function, that separate chunk of code will be loaded from the server and called. Subsequently, it will be called without an additional loading step.

```
#[lazy]
fn lazy_synchronous_function() -> String {
    "Hello, lazy world!".to_string()
}

#[lazy]
async fn lazy_async_function() -> String {
    /* do something that requires async work */
    "Hello, lazy async world!".to_string()
}

async fn use_lazy_functions() {
    // synchronous function has been converted to async
    let value1 = lazy_synchronous_function().await;

    // async function is still async
    let value1 = lazy_async_function().await;
}
```

This can be useful for one-off lazy functions. But lazy-loading is most powerful when it's paired with the router.

#[lazy_route]

Lazy routes allow you to split out the code for a route's view, and to lazily load it concurrently with data for that route while navigating. Through the use of nested routing, multiple lazy-loaded routes can be nested: each will load its own data and its own lazy view concurrently.

Splitting the data loading from the (lazy-loaded) view allows you to prevent a “waterfall,” in which you wait for the lazy view to load, then begin loading data.

```
use leptos::prelude::*;
use leptos_router::{lazy_route, LazyRoute};

// the route definition
#[derive(Debug)]
struct BlogListingRoute {
    titles: Resource<Vec<String>>
}

#[lazy_route]
impl LazyRoute for BlogListingRoute {
    fn data() -> Self {
        Self {
            titles: Resource::new(|| (), |_| async {
                vec![/* todo: load blog posts */]
            })
        }
    }

    // this function will be lazy-loaded, concurrently with data()
    fn view(this: Self) -> AnyView {
        let BlogListingRoute { titles } = this;

        // ... now you can use the `posts` resource with Suspense, etc.,
        // and return AnyView by calling .into_any() on a view
    }
}
```

Examples and More Information

You can find more in-depth discussion in [this YouTube video](#), and a full `lazy_routes` example in the repo.

Guide: Islands

Leptos 0.5 introduced the new `islands` feature. This guide will walk through the islands feature and core concepts, while implementing a demo app using the islands architecture.

The Islands Architecture

The dominant JavaScript frontend frameworks (React, Vue, Svelte, Solid, Angular) all originated as frameworks for building client-rendered single-page apps (SPAs). The initial page load is rendered to HTML, then hydrated, and subsequent navigations are handled directly in the client. (Hence “single page”: everything happens from a single page load from the server, even if there is client-side routing later.) Each of these frameworks later added server-side rendering to improve initial load times, SEO, and user experience.

This means that by default, the entire app is interactive. It also means that the entire app has to be shipped to the client as JavaScript in order to be hydrated. Leptos has followed this same pattern.

You can read more in the chapters on [server-side rendering](#).

But it’s also possible to work in the opposite direction. Rather than taking an entirely-interactive app, rendering it to HTML on the server, and then hydrating it in the browser, you can begin with a plain HTML page and add small areas of interactivity. This is the traditional format for any website or app before the 2010s: your browser makes a series of requests to the server and returns the HTML for each new page in response. After the rise of “single-page apps” (SPA), this approach has sometimes become known as a “multi-page app” (MPA) by comparison.

The phrase “islands architecture” has emerged recently to describe the approach of beginning with a “sea” of server-rendered HTML pages, and adding “islands” of interactivity throughout the page.

Additional Reading

The rest of this guide will look at how to use islands with Leptos. For more background on the approach in general, check out some of the articles below:

- Jason Miller, [“Islands Architecture”](#), Jason Miller
- Ryan Carniato, [“Islands & Server Components & Resumability, Oh My!”](#)

- [“Islands Architectures”](#) on patterns.dev
 - [Astro Islands](#)
-

Activating Islands Mode

Let's start with a fresh `cargo-leptos` app:

```
cargo leptos new --git leptos-rs/start-axum
```

There should be no real differences between Actix and Axum in this example.

I'm just going to run

```
cargo leptos build
```

in the background while I fire up my editor and keep writing.

The first thing I'll do is to add the `islands` feature in my `Cargo.toml`. I only need to add this to the `leptos` crate.

```
leptos = { version = "0.7", features = ["islands"] }
```

Next I'm going to modify the `hydrate` function exported from `src/lib.rs`. I'm going to remove the line that calls `leptos::mount::hydrate_body(App)` and replace it with

```
leptos::mount::hydrate_islands();
```

Rather than running the whole application and hydrating the view that it creates, this will hydrate each individual island, in order.

In `app.rs`, in the `shell` functions, we'll also need to add `islands=true` to the `HydrationScripts` component:

```
<HydrationScripts options islands=true/>
```

Okay, now fire up your `cargo leptos watch` and go to <http://localhost:3000> (or wherever).

Click the button, and...

Nothing happens!

Perfect.

Note

The starter templates include `use app::*;` in their `hydrate()` function definitions. Once you've switched over to islands mode, you are no longer using the imported `main App` function, so you might think you can delete this. (And in fact, Rust lint tools might issue warnings if you don't!)

However, this can cause issues if you are using a workspace setup. We use `wasm-bindgen` to independently export an entrypoint for each function. In my experience, if you are using a workspace setup and nothing in your `frontend` crate actually uses the `app` crate, those bindings will not be generated correctly. [See this discussion for more.](#)

Using Islands

Nothing happens because we've just totally inverted the mental model of our app. Rather than being interactive by default and hydrating everything, the app is now plain HTML by default, and we need to opt into interactivity.

This has a big effect on WASM binary sizes: if I compile in release mode, this app is a measly 24kb of WASM (uncompressed), compared to 274kb in non-islands mode. (274kb is quite large for a "Hello, world!" It's really just all the code related to client-side routing, which isn't being used in the demo.)

When we click the button, nothing happens, because our whole page is static.

So how do we make something happen?

Let's turn the `HomePage` component into an island!

Here was the non-interactive version:

```
#[component]
fn HomePage() -> impl IntoView {
    // Creates a reactive value to update the button
    let count = RwSignal::new(0);
    let on_click = move |_| *count.write() += 1;

    view! {
        <h1>"Welcome to Leptos!"</h1>
        <button on:click=on_click>"Click Me: " {count}</button>
    }
}
```

Here's the interactive version:

```
#[island]
fn HomePage() -> impl IntoView {
    // Creates a reactive value to update the button
    let count = RwSignal::new(0);
    let on_click = move |_| *count.write() += 1;

    view! {
        <h1>"Welcome to Leptos!"</h1>
        <button on:click=on_click>"Click Me: " {count}</button>
    }
}
```

Now when I click the button, it works!

The `#[island]` macro works exactly like the `#[component]` macro, except that in islands mode, it designates this as an interactive island. If we check the binary size again, this is 166kb uncompressed in release mode; much larger than the 24kb totally static version, but much smaller than the 355kb fully-hydrated version.

If you open up the source for the page now, you'll see that your `HomePage` island has been rendered as a special `<leptos-island>` HTML element which specifies which component should be used to hydrate it:

```
<leptos-island data-component="HomePage_7432294943247405892">
  <h1>Welcome to Leptos!</h1>
  <button>
    Click Me:
    <!--0
  </button>
</leptos-island>
```

Only code for what's inside this `<leptos-island>` is compiled to WASM, only that code runs when hydrating.

Using Islands Effectively

Remember that *only* code within an `#[island]` needs to be compiled to WASM and shipped to the browser. This means that islands should be as small and specific as possible. My `HomePage`, for example, would be better broken apart into a regular component and an island:

```
#[component]
fn HomePage() -> impl IntoView {
    view! {
        <h1>"Welcome to Leptos!"</h1>
        <Counter/>
    }
}

#[island]
fn Counter() -> impl IntoView {
    // Creates a reactive value to update the button
    let (count, set_count) = signal(0);
    let on_click = move |_| *set_count.write() += 1;

    view! {
        <button on:click=on_click>"Click Me: " {count}</button>
    }
}
```

Now the `<h1>` doesn't need to be included in the client bundle, or hydrated. This seems like a silly distinction now; but note that you can now add as much inert HTML content as you want to the `HomePage` itself, and the WASM binary size will remain exactly the same.

In regular hydration mode, your WASM binary size grows as a function of the size/complexity of your app. In islands mode, your WASM binary grows as a function of the amount of interactivity in your app. You can add as much non-interactive content as you want, outside islands, and it will not increase that binary size.

Unlocking Superpowers

So, this 50% reduction in WASM binary size is nice. But really, what's the point?

The point comes when you combine two key facts:

1. Code inside `#[component]` functions now *only* runs on the server, unless you use it in an island.*
2. Children and props can be passed from the server to islands, without being included in the WASM binary.

This means you can run server-only code directly in the body of a component, and pass it directly into the children. Certain tasks that take a complex blend of server functions and Suspense in fully-hydrated apps can be done inline in islands.

* This "unless you use it in an island" is important. It is *not* the case that `#[component]` components only run on the server. Rather, they are "shared components" that are

only compiled into the WASM binary if they're used in the body of an `#[island]` . But if you don't use them in an island, they won't run in the browser.

We're going to rely on a third fact in the rest of this demo:

3. Context can be passed between otherwise-independent islands.

So, instead of our counter demo, let's make something a little more fun: a tabbed interface that reads data from files on the server.

Passing Server Children to Islands

One of the most powerful things about islands is that you can pass server-rendered children into an island, without the island needing to know anything about them. Islands hydrate their own content, but not children that are passed to them.

As Dan Abramov of React put it (in the very similar context of RSCs), islands aren't really islands: they're donuts. You can pass server-only content directly into the "donut hole," as it were, allowing you to create tiny atolls of interactivity, surrounded on *both* sides by the sea of inert server HTML.

In the demo code included below, I added some styles to show all server content as a light-blue "sea," and all islands as light-green "land." Hopefully that will help picture what I'm talking about!

To continue with the demo: I'm going to create a `Tabs` component. Switching between tabs will require some interactivity, so of course this will be an island. Let's start simple for now:

```
#[island]
fn Tabs(labels: Vec<String>) -> impl IntoView {
    let buttons = labels
        .into_iter()
        .map(|label| view! { <button>{label}</button> })
        .collect_view();
    view! {
        <div style="display: flex; width: 100%; justify-content: space-between;">
            {buttons}
        </div>
    }
}
```

Oops. This gives me an error

```
error[E0463]: can't find crate for `serde`
--> src/app.rs:43:1
|
43 | #[island]
|   ^^^^^^^^^ can't find crate
```

Easy fix: let's cargo add serde --features=derive. The #[island] macro wants to pull in serde here because it needs to serialize and deserialize the labels prop.

Now let's update the HomePage to use Tabs.

```
#[component]
fn HomePage() -> impl IntoView {
    // these are the files we're going to read
    let files = ["a.txt", "b.txt", "c.txt"];
    // the tab labels will just be the file names
    let labels = files.iter().copied().map(Into::into).collect();
    view! {
        <h1>"Welcome to Leptos!"</h1>
        <p>"Click any of the tabs below to read a recipe."</p>
        <Tabs labels/>
    }
}
```

If you take a look in the DOM inspector, you'll see the island is now something like

```
<leptos-island
  data-component="Tabs_1030591929019274801"
  data-props='{"labels":["a.txt","b.txt","c.txt"]}'
>
  <div style="display: flex; width: 100%; justify-content: space-between;">
    <button>a.txt</button>
    <button>b.txt</button>
    <button>c.txt</button>
  <!-->
</div>
</leptos-island>
```

Our labels prop is getting serialized to JSON and stored in an HTML attribute so it can be used to hydrate the island.

Now let's add some tabs. For the moment, a Tab island will be really simple:

```
#[island]
fn Tab(index: usize, children: Children) -> impl IntoView {
    view! {
        <div>{children()}</div>
    }
}
```

Each tab, for now will just be a <div> wrapping its children.

Our `Tabs` component will also get some children: for now, let's just show them all.

```
#[island]
fn Tabs(labels: Vec<String>, children: Children) -> impl IntoView {
    let buttons = labels
        .into_iter()
        .map(|label| view! { <button>{label}</button> })
        .collect_view();
    view! {
        <div style="display: flex; width: 100%; justify-content: space-around;">
            {buttons}
        </div>
        {children()}
    }
}
```

Okay, now let's go back into the `HomePage`. We're going to create the list of tabs to put into our tab box.

```
#[component]
fn HomePage() -> impl IntoView {
    let files = ["a.txt", "b.txt", "c.txt"];
    let labels = files.iter().copied().map(Into::into).collect();
    let tabs = move || {
        files
            .into_iter()
            .enumerate()
            .map(|(index, filename)| {
                let content = std::fs::read_to_string(filename).unwrap();
                view! {
                    <Tab index>
                        <h2>{filename.to_string()}</h2>
                        <p>{content}</p>
                    </Tab>
                }
            })
            .collect_view()
    };

    view! {
        <h1>"Welcome to Leptos!"</h1>
        <p>"Click any of the tabs below to read a recipe."</p>
        <Tabs labels>
            <div>{tabs()}</div>
        </Tabs>
    }
}
```

Uh... What?

If you're used to using Leptos, you know that you just can't do this. All code in the body of components has to run on the server (to be rendered to HTML) and in the browser (to hydrate), so you can't just call `std::fs`; it will panic, because there's no access to the local

filesystem (and certainly not to the server filesystem!) in the browser. This would be a security nightmare!

Except... wait. We're in islands mode. This `HomePage` component *really does* only run on the server. So we can, in fact, just use ordinary server code like this.

Is this a dumb example? Yes! Synchronously reading from three different local files in a `.map()` is not a good choice in real life. The point here is just to demonstrate that this is, definitely, server-only content.

Go ahead and create three files in the root of the project called `a.txt`, `b.txt`, and `c.txt`, and fill them in with whatever content you'd like.

Refresh the page and you should see the content in the browser. Edit the files and refresh again; it will be updated.

You can pass server-only content from a `#[component]` into the children of an `#[island]`, without the island needing to know anything about how to access that data or render that content.

This is really important. Passing server `children` to islands means that you can keep islands small. Ideally, you don't want to slap an `#[island]` around a whole chunk of your page. You want to break that chunk out into an interactive piece, which can be an `#[island]`, and a bunch of additional server content that can be passed to that island as `children`, so that the non-interactive subsections of an interactive part of the page can be kept out of the WASM binary.

Passing Context Between Islands

These aren't really "tabs" yet: they just show every tab, all the time. So let's add some simple logic to our `Tabs` and `Tab` components.

We'll modify `Tabs` to create a simple `selected` signal. We provide the read half via context, and set the value of the signal whenever someone clicks one of our buttons.

```
#[island]
fn Tabs(labels: Vec<String>, children: Children) -> impl IntoView {
    let (selected, set_selected) = signal(0);
    provide_context(selected);

    let buttons = labels
        .into_iter()
        .enumerate()
        .map(|(index, label)| view! {
            <button on:click=move |_| set_selected.set(index)>
                {label}
            </button>
        })
        .collect_view();
    // ...
}
```

And let's modify the `Tab` island to use that context to show or hide itself:

```
#[island]
fn Tab(index: usize, children: Children) -> impl IntoView {
    let selected = expect_context::<ReadSignal<usize>>();
    view! {
        <div
            style:background-color="lightgreen"
            style:padding="10px"
            style:display=move || if selected.get() == index {
                "block"
            } else {
                "none"
            }
        >
            {children()}
        </div>
    }
}
```

Now the tabs behave exactly as I'd expect. `Tabs` passes the signal via context to each `Tab`, which uses it to determine whether it should be open or not.

That's why in `HomePage`, I made `let tabs = move ||` a function, and called it like `{tabs()}: creating the tabs lazily this way meant that the Tabs island would already have provided the selected context by the time each Tab went looking for it.`

Our complete tabs demo is about 200kb uncompressed: not the smallest demo in the world, but still significantly smaller than the "Hello, world" using client side routing that we started with! Just for kicks, I built the same demo without islands mode, using `#[server]` functions and `Suspense`. and it was over 400kb. So again, this was about a 50% savings in binary size. And this app includes quite minimal server-only content: remember that as we add additional server-only components and pages, this 200kb will not grow.

Overview

This demo may seem pretty basic. It is. But there are a number of immediate takeaways:

- **50% WASM binary size reduction**, which means measurable improvements in time to interactivity and initial load times for clients.
- **Reduced data serialization costs.** Creating a resource and reading it on the client means you need to serialize the data, so it can be used for hydration. If you've also read that data to create HTML in a `suspense`, you end up with "double data," i.e., the same exact data is both rendered to HTML and serialized as JSON, increasing the size of responses, and therefore slowing them down.
- **Easily use server-only APIs** inside a `#[component]` as if it were a normal, native Rust function running on the server—which, in islands mode, it is!
- **Reduced `#[server]` / `create_resource` / `Suspense` boilerplate** for loading server data.

Future Exploration

The `islands` feature reflects work at the cutting edge of what frontend web frameworks are exploring right now. As it stands, our islands approach is very similar to Astro (before its recent View Transitions support): it allows you to build a traditional server-rendered, multi-page app and pretty seamlessly integrate islands of interactivity.

There are some small improvements that will be easy to add. For example, we can do something very much like Astro's View Transitions approach:

- add client-side routing for islands apps by fetching subsequent navigations from the server and replacing the HTML document with the new one
- add animated transitions between the old and new document using the View Transitions API
- support explicit persistent islands, i.e., islands that you can mark with unique IDs (something like `persist:searchbar` on the component in the view), which can be copied over from the old to the new document without losing their current state

There are other, larger architectural changes that I'm [not sold on yet](#).

Additional Information

Check out the [islands example](#), [roadmap](#), and [Hackernews demo](#) for additional discussion.

Demo Code

```

use leptos::prelude::*;

#[component]
pub fn App() -> impl IntoView {
    view! {
        <main style="background-color: lightblue; padding: 10px">
            <HomePage/>
        </main>
    }
}

/// Renders the home page of your application.
#[component]
fn HomePage() -> impl IntoView {
    let files = ["a.txt", "b.txt", "c.txt"];
    let labels = files.iter().copied().map(Into::into).collect();
    let tabs = move || {
        files
            .into_iter()
            .enumerate()
            .map(|(index, filename)| {
                let content = std::fs::read_to_string(filename).unwrap();
                view! {
                    <Tab index>
                        <div style="background-color: lightblue; padding:
10px">
                            <h2>{filename.to_string()}</h2>
                            <p>{content}</p>
                        </div>
                    </Tab>
                }
            })
            .collect_view()
    };

    view! {
        <h1>"Welcome to Leptos!"</h1>
        <p>"Click any of the tabs below to read a recipe."</p>
        <Tabs labels>
            <div>{tabs()}</div>
        </Tabs>
    }
}

#[island]
fn Tabs(labels: Vec<String>, children: Children) -> impl IntoView {
    let (selected, set_selected) = signal(0);
    provide_context(selected);

    let buttons = labels
        .into_iter()
        .enumerate()
        .map(|(index, label)| {
            view! {

```

```

        <button on:click=move |_| set_selected.set(index)>
            {label}
        </button>
    }
})
.collect_view();
view! {
    <div
        style="display: flex; width: 100%; justify-content: space-around;\
        background-color: lightgreen; padding: 10px;"
    >
        {buttons}
    </div>
    {children()}
}
}

#[island]
fn Tab(index: usize, children: Children) -> impl IntoView {
    let selected = expect_context::<ReadSignal<usize>>();
    view! {
        <div
            style:background-color="lightgreen"
            style:padding="10px"
            style:display=move || if selected.get() == index {
                "block"
            } else {
                "none"
            }
        >
            {children()}
        </div>
    }
}
}

```

Appendix: How does the Reactive System Work?

You don't need to know very much about how the reactive system actually works in order to use the library successfully. But it's always useful to understand what's going on behind the scenes once you start working with the framework at an advanced level.

The reactive primitives you use are divided into three sets:

- **Signals** (`ReadSignal` / `WriteSignal` , `RwSignal` , `Resource` , `Trigger`) Values you can actively change to trigger reactive updates.
- **Computations** (`Memo`) Values that depend on signals (or other computations) and derive a new reactive value through some pure computation.
- **Effects** Observers that listen to changes in some signals or computations and run a function, causing some side effect.

Derived signals are a kind of non-primitive computation: as plain closures, they simply allow you to refactor some repeated signal-based computation into a reusable function that can be called in multiple places, but they are not represented in the reactive system itself.

All the other primitives actually exist in the reactive system as nodes in a reactive graph.

Most of the work of the reactive system consists of propagating changes from signals to effects, possibly through some intervening memos.

The assumption of the reactive system is that effects (like rendering to the DOM or making a network request) are orders of magnitude more expensive than things like updating a Rust data structure inside your app.

So the **primary goal** of the reactive system is to **run effects as infrequently as possible**.

Leptos does this through the construction of a reactive graph.

Leptos's current reactive system is based heavily on the [Reactivity](#) library for JavaScript. You can read Milo's article "[Super-Charging Fine-Grained Reactivity](#)" for an excellent account of its algorithm, as well as fine-grained reactivity in general—including some beautiful diagrams!

The Reactive Graph

Signals, memos, and effects all share three characteristics:

- **Value** They have a current value: either the signal's value, or (for memos and effects) the value returned by the previous run, if any.
- **Sources** Any other reactive primitives they depend on. (For signals, this is an empty set.)
- **Subscribers** Any other reactive primitives that depend on them. (For effects, this is an empty set.)

In reality then, signals, memos, and effects are just conventional names for one generic concept of a "node" in a reactive graph. Signals are always "root nodes," with no sources/parents. Effects are always "leaf nodes," with no subscribers. Memos typically have both sources and subscribers.

In the following examples, I'm going to use the `nightly` syntax, simply for the sake of reducing verbosity in a document that's intended for you to read, not to copy-and-paste from!

Simple Dependencies

So imagine the following code:

```
// A
let (name, set_name) = signal("Alice");

// B
let name_upper = Memo::new(move |_| name.with(|n| n.to_uppercase()));

// C
Effect::new(move |_| {
    log!("{}", name_upper());
});

set_name("Bob");
```

You can easily imagine the reactive graph here: `name` is the only signal/origin node, the `Effect::new` is the only effect/terminal node, and there's one intervening memo.

```
A    (name)
|
B    (name_upper)
|
C    (the effect)
```

Splitting Branches

Let's make it a little more complex.

```
// A
let (name, set_name) = signal("Alice");

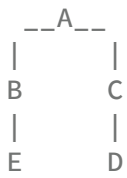
// B
let name_upper = Memo::new(move |_| name.with(|n| n.to_uppercase()));

// C
let name_len = Memo::new(move |_| name.len());

// D
Effect::new(move |_| {
    log!("len = {}", name_len());
});

// E
Effect::new(move |_| {
    log!("name = {}", name_upper());
});
```

This is also pretty straightforward: a signal source `signal ( name / A )` divides into two parallel tracks: `name_upper / B` and `name_len / C`, each of which has an effect that depends on it.



Now let's update the signal.

```
set_name("Bob");
```

We immediately log

```
len = 3
name = BOB
```

Let's do it again.

```
set_name("Tim");
```

The log should shows

```
name = TIM
```

`len = 3` does not log again.

Remember: the goal of the reactive system is to run effects as infrequently as possible. Changing `name` from `"Bob"` to `"Tim"` will cause each of the memos to re-run. But they will

only notify their subscribers if their value has actually changed. "BOB" and "TIM" are different, so that effect runs again. But both names have the length 3, so they do not run again.

Reuniting Branches

One more example, of what's sometimes called **the diamond problem**.

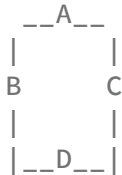
```
// A
let (name, set_name) = signal("Alice");

// B
let name_upper = Memo::new(move |_| name.with(|n| n.to_uppercase()));

// C
let name_len = Memo::new(move |_| name.len());

// D
Effect::new(move |_| {
    log!("{ } is { } characters long", name_upper(), name_len());
});
```

What does the graph look like for this?



You can see why it's called the "diamond problem." If I'd connected the nodes with straight lines instead of bad ASCII art, it would form a diamond: two memos, each of which depend on a signal, which feed into the same effect.

A naive, push-based reactive implementation would cause this effect to run twice, which would be bad. (Remember, our goal is to run effects as infrequently as we can.) For example, you could implement a reactive system such that signals and memos immediately propagate their changes all the way down the graph, through each dependency, essentially traversing the graph depth-first. In other words, updating A would notify B, which would notify D; then A would notify C, which would notify D again. This is both inefficient (D runs twice) and glitchy (D actually runs with the incorrect value for the second memo during its first run.)

Solving the Diamond Problem

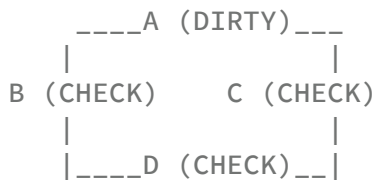
Any reactive implementation worth its salt is dedicated to solving this issue. There are a number of different approaches (again, [see Milo's article](#) for an excellent overview).

Here's how ours works, in brief.

A reactive node is always in one of three states:

- `clean` : it is known not to have changed
- `check` : it is possible it has changed
- `dirty` : it has definitely changed

Updating a signal `dirty` marks that signal `dirty`, and marks all its descendants `check`, recursively. Any of its descendants that are effects are added to a queue to be re-run.



Now those effects are run. (All of the effects will be marked `check` at this point.) Before re-running its computation, the effect checks its parents to see if they are dirty.

- So `D` goes to `B` and checks if it is `dirty`.
- But `B` is also marked `check`. So `B` does the same thing:
 - `B` goes to `A`, and finds that it is `dirty`.
 - This means `B` needs to re-run, because one of its sources has changed.
 - `B` re-runs, generating a new value, and marks itself `clean`
 - Because `B` is a memo, it then checks its prior value against the new value.
 - If they are the same, `B` returns "no change." Otherwise, it returns "yes, I changed."
- If `B` returned "yes, I changed," `D` knows that it definitely needs to run and re-runs immediately before checking any other sources.
- If `B` returned "no, I didn't change," `D` continues on to check `C` (see process above for `B`.)
- If neither `B` nor `C` has changed, the effect does not need to re-run.
- If either `B` or `C` did change, the effect now re-runs.

Because the effect is only marked `check` once and only queued once, it only runs once.

If the naive version was a "push-based" reactive system, simply pushing reactive changes all the way down the graph and therefore running the effect twice, this version could be called "push-pull." It pushes the `check` status all the way down the graph, but then "pulls" its way back up. In fact, for large graphs it may end up bouncing back up and down and left and right on the graph as it tries to determine exactly which nodes need to re-run.

Note this important trade-off: Push-based reactivity propagates signal changes more quickly, at the expense of over-re-running memos and effects. Remember: the reactive system is designed to minimize how often you re-run effects, on the (accurate) assumption that side effects are orders of magnitude more expensive than this kind of cache-friendly graph traversal happening entirely inside the library's Rust code. The measurement of a good reactive system is not how quickly it propagates changes, but how quickly it propagates changes *without over-notifying*.

Memos vs. Signals

Note that signals always notify their children; i.e., a signal is always marked `Dirty` when it updates, even if its new value is the same as the old value. Otherwise, we'd have to require `PartialEq` on signals, and this is actually quite an expensive check on some types. (For example, add an unnecessary equality check to something like `some_vec_signal.update(|n| n.pop())` when it's clear that it has in fact changed.)

Memos, on the other hand, check whether they change before notifying their children. They only run their calculation once, no matter how many times you `.get()` the result, but they run whenever their signal sources change. This means that if the memo's computation is *very* expensive, you may actually want to memoize its inputs as well, so that the memo only re-calculates when it is sure its inputs have changed.

Memos vs. Derived Signals

All of this is cool, and memos are pretty great. But most actual applications have reactive graphs that are quite shallow and quite wide: you might have 100 source signals and 500 effects, but no memos or, in rare case, three or four memos between the signal and the effect. Memos are extremely good at what they do: limiting how often they notify their subscribers that they have changed. But as this description of the reactive system should show, they come with overhead in two forms:

1. A `PartialEq` check, which may or may not be expensive.
2. Added memory cost of storing another node in the reactive system.
3. Added computational cost of reactive graph traversal.

In cases in which the computation itself is cheaper than this reactive work, you should avoid "over-wrapping" with memos and simply use derived signals. Here's a great example in which you should never use a memo:

```
let (a, set_a) = signal(1);  
// none of these make sense as memos  
let b = move || a() + 2;  
let c = move || b() % 2 == 0;  
let d = move || if c() { "even" } else { "odd" };  
  
set_a(2);  
set_a(3);  
set_a(5);
```

Even though memoizing would technically save an extra calculation of `d` between setting `a` to `3` and `5`, these calculations are themselves cheaper than the reactive algorithm.

At the very most, you might consider memoizing the final node before running some expensive side effect:

```
let text = Memo::new(move |_| {  
    d()  
});  
Effect::new(move |_| {  
    engrave_text_into_bar_of_gold(&text());  
});
```

Appendix: The Life Cycle of a Signal

Three questions commonly arise at the intermediate level when using Leptos:

1. How can I connect to the component lifecycle, running some code when a component mounts or unmounts?
2. How do I know when signals are disposed, and why do I get an occasional panic when trying to access a disposed signal?
3. How is it possible that signals are `copy` and can be moved into closures and other structures without being explicitly cloned?

The answers to these three questions are closely inter-related, and are each somewhat complicated. This appendix will try to give you the context for understanding the answers, so that you can reason correctly about your application's code and how it runs.

The Component Tree vs. The Decision Tree

Consider the following simple Leptos app:

```

use leptos::logging::log;
use leptos::prelude::*;

#[component]
pub fn App() -> impl IntoView {
    let (count, set_count) = signal(0);

    view! {
        <button on:click=move |_| *set_count.write() += 1>"+1"</button>
        {move || if count.get() % 2 == 0 {
            view! { <p>"Even numbers are fine."</p> }.into_any()
        } else {
            view! { <InnerComponent count/> }.into_any()
        }}
    }
}

#[component]
pub fn InnerComponent(count: ReadSignal<usize>) -> impl IntoView {
    Effect::new(move |_| {
        log!("count is odd and is {}", count.get());
    });

    view! {
        <OddDuck/>
        <p>{count}</p>
    }
}

#[component]
pub fn OddDuck() -> impl IntoView {
    view! {
        <p>"You're an odd duck."</p>
    }
}

```

All it does is show a counter button, and then one message if it's even, and a different message if it's odd. If it's odd, it also logs the values in the console.

One way to map out this simple application would be to draw a tree of nested components:

```

App
├─ InnerComponent
│  └─ OddDuck

```

Another way would be to draw the tree of decision points:

```

root
├─ is count even?
│  └─ yes
│     └─ no

```

If you combine the two together, you'll notice that they don't map onto one another perfectly. The decision tree slices the view we created in `InnerComponent` into three pieces, and combines part of `InnerComponent` with the `OddDuck` component:

DECISION	COMPONENT	DATA	SIDE EFFECTS
root	<code><App/></code>	(count)	render <code><button></code>
_ is count even?	<code><InnerComponent/></code>		
_ yes			render even <code><p></code>
_ no	<code><OddDuck/></code>		start logging the count
			render odd <code><p></code>
			render odd <code><p></code> (in
	<code><InnerComponent/>!</code>		

Looking at this table, I notice the following things:

1. The component tree and the decision tree don't match one another: the "is count even?" decision splits `<InnerComponent/>` into three parts (one that never changes, one if even, one if odd), and merges one of these with the `<OddDuck/>` component.
2. The decision tree and the list of side effects correspond perfectly: each side effect is created at a specific decision point.
3. The decision tree and the tree of data also line up. It's hard to see with only one signal in the table, but unlike a component, which is a function that can include multiple decisions or none, a signal is always created at a specific line in the tree of decisions.

Here's the thing: The structure of your data and the structure of side effects affect the actual functionality of your application. The structure of your components is just a convenience of authoring. You don't care, and you shouldn't care, which component rendered which `<p>` tag, or which component created the effect to log the values. All that matters is that they happen at the right times.

In Leptos, *components do not exist*. That is to say: You can write your application as a tree of components, because that's convenient, and we provide some debugging tools and logging built around components, because that's convenient too. But your components do not exist at runtime: Components are not a unit of change detection or of rendering. They are simply function calls. You can write your whole application in one big component, or split it into a hundred components, and it does not affect the runtime behavior, because components don't really exist.

The decision tree, on the other hand, *does exist*. And it's really important!

The Decision Tree, Rendering, and Ownership

Every decision point is some kind of reactive statement: a signal or a function that can change over time. When you pass a signal or a function into the renderer, it automatically

wraps it in an effect that subscribes to any signals it contains, and updates the view accordingly over time.

This means that when your application is rendered, it creates a tree of nested effects that perfectly mirrors the decision tree. In pseudo-code:

```
// root
let button = /* render the <button> once */;

// the renderer wraps an effect around the `move || if count() ...`
Effect::new(|_| {
    if count.get() % 2 == 0 {
        let p = /* render the even <p> */;
    } else {
        // the user created an effect to log the count
        Effect::new(|_| {
            log!("count is odd and is {}", count.get());
        });

        let p1 = /* render the <p> from OddDuck */;
        let p2 = /* render the second <p> */

        // the renderer creates an effect to update the second <p>
        Effect::new(|_| {
            // update the content of the <p> with the signal
            p2.set_text_content(count.get());
        });
    }
})
```

Each reactive value is wrapped in its own effect to update the DOM, or run any other side effects of changes to signals. But you don't need these effects to keep running forever. For example, when `count` switches from an odd number back to an even number, the second `<p>` no longer exists, so the effect to keep updating it is no longer useful. Instead of running forever, effects are canceled when the decision that created them changes. In other words, and more precisely: effects are canceled whenever the effect that was running when they were created re-runs. If they were created in a conditional branch, and re-running the effect goes through the same branch, the effect will be created again: if not, it will not.

From the perspective of the reactive system itself, your application's "decision tree" is really a reactive "ownership tree." Simply put, a reactive "owner" is the effect or memo that is currently running. It owns effects created within it, they own their own children, and so on. When an effect is going to re-run, it first "cleans up" its children, then runs again.

So far, this model is shared with the reactive system as it exists in JavaScript frameworks like S.js or Solid, in which the concept of ownership exists to automatically cancel effects.

What Leptos adds is that we add a second, similar meaning to ownership: a reactive owner not only owns its child effects, so that it can cancel them; it also owns its signals (memos, etc.) so that it can dispose of them.

Ownership and the Copy Arena

This is the innovation that allows Leptos to be usable as a Rust UI framework. Traditionally, managing UI state in Rust has been hard, because UI is all about shared mutability. (A simple counter button is enough to see the problem: You need both immutable access to set the text node showing the counter's value, and mutable access in the click handler, and every Rust UI framework is designed around the fact that Rust is designed to prevent exactly that!) Using something like an event handler in Rust traditionally relies on primitives for communicating via shared memory with interior mutability (`Rc<RefCell<_>>` , `Arc<Mutex<_>>`) or for shared memory by communicating via channels, either of which often requires explicit `.clone()` ing to be moved into an event listener. This is kind of fine, but also an enormous inconvenience.

Leptos has always used a form of arena allocation for signals instead. A signal itself is essentially an index into a data structure that's held elsewhere. It's a cheap-to-copy integer type that does not do reference counting on its own, so it can be copied around, moved into event listeners, etc. without explicit cloning.

Instead of Rust lifetimes or reference counting, the life cycles of these signals are determined by the ownership tree.

Just as all effects belong to an owning parent effect, and the children are canceled when the owner reruns, so too all signals belong to an owner, and are disposed of when the parent reruns.

In most cases, this is completely fine. Imagine that in our example above, `<OddDuck/>` created some other signal that it used to update part of its UI. In most cases, that signal will be used for local state in that component, or maybe passed down as a prop to another component. It's unusual for it to be hoisted up out of the decision tree and used somewhere else in the application. When the `count` switches back to an even number, it is no longer needed and can be disposed.

However, this means there are two possible issues that can arise.

Signals can be used after they are disposed

The `ReadSignal` or `WriteSignal` that you hold is just an integer: say, 3 if it's the 3rd signal in the application. (As always, the reality is a bit more complicated, but not much.) You can copy that number all over the place and use it to say, "Hey, get me signal 3." When the owner cleans up, the *value* of signal 3 will be invalidated; but the number 3 that you've copied all over the place can't be invalidated. (Not without a whole garbage collector!) That means that if you push signals back "up" the decision tree, and store them somewhere conceptually "higher" in your application than they were created, they can be accessed after being disposed.

If you try to *update* a signal after it was disposed, nothing bad really happens. The framework will just warn you that you tried to update a signal that no longer exists. But if you try to *access* one, there's no coherent answer other than panicking: there is no value that could be returned. (There are `try_` equivalents to the `.get()` and `.with()` methods that will simply return `None` if a signal has been disposed).

Signals can be leaked if you create them in a higher scope and never dispose of them

The opposite is also true, and comes up particularly when working with collections of signals, like an `RwSignal<Vec<RwSignal<_>>>`. If you create a signal at a higher level, and pass it down to a component at a lower level, it is not disposed until the higher-up owner is cleaned up.

For example, if you have a todo app that creates a new `RwSignal<Todo>` for each todo, stores it in an `RwSignal<Vec<RwSignal<Todo>>>`, and then passes it down to a `<Todo/>`, that signal is not automatically disposed when you remove the todo from the list, but must be manually disposed, or it will "leak" for as long as its owner is still alive. (See the [TodoMVC example](#) for more discussion.)

This is only an issue when you create signals, store them in a collection, and remove them from the collection without manually disposing of them as well.

Solving these Problems with Reference-Counted Signals

0.7 introduces a reference-counted equivalent for each of our arena-allocated primitive: for every `RwSignal` there is an `ArcRwSignal` (`ArcReadSignal`, `ArcWriteSignal`, `ArcMemo`, and so on).

These have their memory and disposal managed by reference counting, rather than the ownership tree.

This means that they can safely be used in situations in which the arena-allocated equivalents would either be leaked or used after being disposed.

This is especially useful when creating collections of signals: you might create `ArcRwSignal<_>` instead of `RwSignal<_>`, and then convert it into an `RwSignal<_>` in each row of a table, for example.

See the use of `ArcRwSignal<i32>` in the [counters example](#) for a more concrete example.

Connecting the Dots

The answers to the questions we started with should probably make some sense now.

Component Life-Cycle

There is no component life-cycle, because components don't really exist. But there is an ownership lifecycle, and you can use it to accomplish the same things:

- *before mount*: simply running code in the body of a component will run it "before the component mounts"
- *on mount*: `create_effect` runs a tick after the rest of the component, so it can be useful for effects that need to wait for the view to be mounted to the DOM.
- *on unmount*: You can use `on_cleanup` to give the reactive system code that should run while the current owner is cleaning up, before running again. Because an owner is around a "decision," this means that `on_cleanup` will run when your component unmounts: if something can unmount, the renderer must have created an effect that's unmounting it!

Issues with Disposed Signals

Generally speaking, problems can only arise here if you are creating a signal lower down in the ownership tree and storing it somewhere higher up. If you run into issues here, you should instead "hoist" the signal creation up into the parent, and then pass the created signals down—making sure to dispose of them on removal, if needed!

Copy signals

The whole system of copyable wrapper types (signals, `StoredValue`, and so on) uses the ownership tree as a close approximation of the life-cycle of different parts of your UI. In effect, it parallels the Rust language's system of lifetimes based on blocks of code with a system of lifetimes based on sections of UI. This can't always be perfectly checked at compile time, but overall we think it's a net positive.